Abstract

Practical Hash- and Lattice-based SNARKs

Psi Vesely

2026

Succinct, Non-interactive Arguments of Knowledge (SNARKs) produce small proofs that can be verified quickly while attesting to the correctness of arbitrary computations. First deployed in 2016, SNARKs have enabled private transactions, decentralized storage, anonymous credentials (e.g., private age verification), proofs about network sessions (e.g., email and TLS session contents), and more. This dissertation advances SNARKs in two ways: (1) it develops arithmetization techniques and interactive oracle proofs (IOPs) for mixed- and small-domain computations, enabling constraints to be written over their natural domains and reducing non-native overhead; and (2) it gives efficient post-quantum (hash- and lattice-based) polynomial commitments (PCs) for these specialized domains.

Zinc+ is a framework for proving computations specified across multiple rings and ideals simultaneously. We define the Universal Constraint System (UCS), capturing constraints over $\mathbb{Z}, \mathbb{Q}$, finite fields, and polynomial rings together with ideal-membership predicates. The Zinc+ compiler lifts finite-field polynomial IOPs (PIOPs) into round-by-round knowledge-sound PIOPs for UCS. We instantiate it with a new IOP of proximity, Zip+, that supports mixed-domain coefficients. *Microlotus* accelerates SNARKs in small-modulus fields and cyclotomic rings. We introduces a packed variant of sumcheck for small fields and a finite-field IOP for cyclotomic ring multiplication that, unlike previous results, has native prover complexity. We also study extensions of linear codes and proximity testing from fields to rings. Finally, *Orbweaver* gives a lattice-based functional commitment for linear relations over cyclotomic rings and the integers, achieving $\tilde{O}_\lambda(\log n)$

verification time and supporting public aggregation. We show it yields univariate and multilinear PCs, and a SNARK directly.

Practical Hash- and Lattice-based SNARKs

A Dissertation

Presented to the Faculty of the Graduate School

Of

Yale University

In Candidacy for the Degree of

Doctor of Philosophy

By

Psi Vesely

Dissertation Director: Ben Fisch

May 2026

© 2026 by Psi Vesely
All rights reserved.

Contents

1 Introduction **1**

1.1 Motivation . . . 1

2 Zinc+: SNARKs for Polynomial Rings **2**

2.1 Introduction . . . 2

2.1.1 Contributions . . . 3

2.1.2 Related work . . . 9

2.1.3 Acknowledgments . . . 11

2.2 Technical Overview . . . 11

2.2.1 Universal Constraint Systems (UCS) . . . 11

2.2.2 Zinc+: a PIOP for UCS from PIOPs over finite fields . . . 19

2.2.3 Efficiency and experimental results . . . 25

2.3 Preliminaries . . . 36

2.3.1 Basic notions . . . 36

2.3.2 Indexed relations and oracles. . . . 38

2.3.3 Polynomial Interactive Oracle Reductions (PIORs) . . . 39

2.4 Universal Constraint Systems (UCS) . . . 41

2.4.1 General UCS definition . . . 41

2.4.2 Lookup and permutation constraints as UCS constraints . . . 48

2.4.3 R1CS, CCS, and AIR as UCS instances . . . 52

2.5 Zinc+ Compiler: from finite field PIOPs to PIOPs for UCS . . . 53

2.5.1 Incomplete polynomial oracles . . . 53
2.5.2 Batching ideal membership checks and projecting to finite fields . . . 54
2.5.3 Strictly algebraic UCS . . . 56
2.5.4 Zinc+ Compiler . . . 57

3 Microlotus: Faster Proving Over Small Fields and Cyclotomic Rings 68

3.0.1 Contributions . . . 70
3.0.2 Simulating ring arithmetic in native time . . . 70
3.0.3 Multilinear polynomials . . . 72
3.0.4 Rings . . . 73
3.0.5 Circulant matrices . . . 75
3.0.6 Sumcheck . . . 76
3.1 PIOPs over fields for cyclotomic ring arithmetic . . . 76
3.1.1 A PIOP for cyclotomic matrix-vector multiplication . . . 79
3.1.2 A holographic variant . . . 82
3.1.3 Optimized inner product variant . . . 83
3.1.4 Reducing the number of commitments . . . 85
3.1.5 Interactive oracle reductions . . . 86
3.1.6 DP24 packing . . . 88

4 Orbweaver: Succinct Linear Functional Commitments from Lattices 93

4.1 Overview . . . 93
4.1.1 Our results . . . 96
4.1.2 Related work . . . 97
4.1.3 Technical overview . . . 98
4.2 Preliminaries . . . 101
4.2.1 Exceptional challenge sets . . . 102
4.2.2 Functional commitments . . . 103
4.2.3 Sampling Algorithm . . . 105

4.2.4 Cryptographic assumptions 105
4.3 Cryptanalysis of k-P-R-ISIS 109
4.4 Orbweaver: linear functional commitments for rings 111
4.4.1 Extensions 116
4.4.2 Sublogarithmic proofs in the input norm 119
4.4.3 Polynomial commitments for integers 122
4.5 Evaluation 123
4.5.1 Implementation optimizations 123
4.5.2 Proof and CRS sizes 125
.1 Sumcheck algorithms 127

Acknowledgment

This dissertation grew out of many conversations and collaborations, and I am grateful to everyone who helped shape the ideas, the writing, and my time at Yale. I thank my advisor, Ben Fisch, for his guidance, high standards, and incisive feedback on both the technical work and its presentation. I thank the rest of my dissertation committee—Katerina Sotiraki, Charalampos Papamanthou, and Benedikt Bünz—for their time and careful reading. I also thank my past advisors, Mary Maller and Alessandro Chiesa, for shaping how I approach research and preparing me for this endeavour.

I am grateful to the co-authors who worked with me on the papers included in this dissertation: Ben Fisch and Zeyu Liu on *Orbweaver*; Alexander Abdugafarov, Albert Garreta, Michał Osadnik, Ilia Vlasov, and Kai Zhe Zheng on *Zinc+*; and Hunter Lindauer and Hadas Zeilberger on *Microlotus*. Likewise, I am grateful for those who worked with me on papers published earlier in my PhD, but not included here: Benedikt Bünz, Mary Maller, Pratyush Mishra, and Nirvan Tyagi on *Proofs for Inner Pairing Products and Applications*; and Kobi Gurkan, Michael Straka, Ariel Gabizon, Philipp Jovanovic, Georgios Konstantopoulos, Asa Oines, Marek Olszewski, and Eran Tromer on *Plumo: An Ultralight Blockchain Client*. I learned a great deal from each collaboration about proof systems, working under real constraints, and sharpening rough ideas into precise results.

Finally, I thank my parents, friends, partner, and community for all the love, joy, and support they have given me, which has made this project possible.

Per scientiam ad justitiam

Chapter 1

Introduction

1.1 Motivation

Note on Authorship, Scope of Included Material, and Use of AI Tools

Each chapter of this dissertation is adapted from joint work with co-authors, in which both the writing and the technical development were collaborative. In preparing this dissertation, I have therefore distinguished between expository material and core technical development. The core technical sections included here are limited to material of which I was the sole author or principal author, so that the dissertation's principal technical content reflects my primary research and writing contributions. By contrast, although I contributed to all of the introduction and technical overview sections included here, I was not the principal author of all of them. I have included this expository material to preserve the motivation, structure, and overall technical narrative of the underlying joint works, so that each chapter remains intelligible in context even where the detailed technical presentation is limited to portions that principally reflect my own authorship.

In preparing this manuscript, I used an AI-based tool (Claude) for limited purposes such as grammar review, editing assistance, and latex code-generation assistance. I bear full responsibility for the accuracy, originality, and proper attribution of all content in this dissertation.

Chapter 2

Zinc+: SNARKs for Polynomial Rings

2.1 Introduction

Succinct Non-interactive Arguments of Knowledge (SNARKs) are proof systems that allow a prover to succinctly demonstrate the validity of arbitrary computations. More precisely, given a relation R consisting of input-witness tuples $(\mathbb{x}; \mathbb{w})$ and a public input $\mathbb{x}$, a SNARK is a non-interactive protocol that enables a prover, P, to convince a verifier, V, that they know a witness, $\mathbb{w}$, such that $(\mathbb{x}; \mathbb{w}) \in R$, where communication and verifier runtime are sublinear (i.e. the scheme is *succinct*). The most common approach to building SNARKs is to first choose a finite field $\mathbb{F}$ and then express R as a new relation $R'_{\mathbb{F}}$ consisting of a system of polynomial equations over $\mathbb{F}$ in a certain fixed form (e.g. R1CS, CCS, AIR, Plonkish, or lookups)[1]. This process is called *arithmetization* of the computation Comp, and it is by far the most common paradigm in both the SNARK literature and industry [**Groth16**; **GWC19**; **ethstark**; **Cairo**]. SNARKs based on this approach have experienced remarkable performance improvements over the past years.

A limitation of this approach is that most computations of interest (see table 2.2) are not naturally expressed as systems of equations over a single finite field. Having to express a computation naturally defined over multiple algebraic domains, or with non-algebraic operations, over a single finite field

[1]Lookup and permutation constraints can be reduced to (high degree) polynomial equations; see section 2.4.2 for details.

results in large blowups to the circuit size, i.e. given $(\mathbb{x}; \mathbb{w}) \in R$, the size of the arithmetized $(\mathbb{x}'; \mathbb{w}') \in R'_{\mathbb{F}}$ is typically much larger than $(\mathbb{x}; \mathbb{w})$ (as will the corresponding circuit for verifying membership) [**Zinc**; **jea_codes**; **EFpaper**; **binius**]. Accordingly, state-of-the-art SNARKs end up proving computations that are, in a sense, significantly larger than the original computation Comp. We call this relative size increase the *arithmetization overhead*, and note it detrimentally affects all performance metrics of the SNARK: prover time, proof size, and verifier time. Beyond performance, there are two other problems brought by non-native arithmetizations:

1) Need for expert arithmetization or generic arithmetizers (zkVMs). Designing efficient and correct arithmetizations over a finite field is challenging both conceptually and in terms of implementation [**PailoorUnderconstrained**; **WenZKPSecurity**]. This makes the arithmetization step inaccessible to non-experts, who are then bound to generic tools such as zkVMs [**Cairo**; **Jolt**]. In fact, current zkVM designs even incur a *double* arithmetization overhead: first from compiling the computation into VM cycles (typically composed of RISC-V instructions), and then from arithmetizing those cycles into a finite-field relation. We note that this double overhead is not inherent to the zkVM paradigm—one could design zkVMs whose underlying proof system avoids arithmetization overhead. We envision the present work as a step in this direction.

2) Correctness and auditability. The gap between the original computation and its arithmetized form makes the final proof system harder to audit and more prone to bugs, e.g., due to underconstrained systems.

2.1.1 Contributions

We next list the contributions of this work, which we elaborate on in the technical overview (section 2.2.2).

Universal Constraint Systems (UCS). We propose a new arithmetization framework which is tailored to express most computations of interest (as captured in table 2.2), with significantly reduced arithmetization overhead compared to single-field constraint systems (see table 2.2 for concrete examples). In particular, we design a relation $\mathsf{REL}_{\mathsf{UCS}}$ called *Universal Constraint System* (UCS) that efficiently encapsulates said computations (see section 2.2.1 and the formal definition in

definition 2.4.1).

UCS is a generalization of constraints such as R1CS, CCS, AIR, lookups, etc., along two main axes: the algebraic domains over which constraints are defined, and the types of predicates supported. First, UCS expresses constraints simultaneously over different rings such as $\mathbb{Q}$ (rationals), $\mathbb{Z}$ (integers), $\mathbb{F}_q$ (prime or prime power fields), and, importantly, polynomial rings such as $\mathbb{Q}[X], \mathbb{Z}[X], \mathbb{F}_q[X], R[X]$. In short, UCS can express R1CS, AIR, etc. relations over multiple such rings in the same instance. Additionally, it supports ideal-membership predicates, i.e. predicates of the form $Q(\dots) \in \mathfrak{I}$, where Q is a polynomial expression in the input and the witness, and $\mathfrak{I}$ is an ideal of $\mathbb{Q}[X]$ or $\mathbb{F}_q[X]$. This expressivity allows one to capture operations such as rotation of bitstrings (through the ideal $X^w - 1$), relating bitstrings and integers (through the ideal $(X - 2)$), constraints over multiple rings and fields (through quotienting $\mathbb{Z}[X]$ via an ideal), etc.

A SNARK for the UCS relation. We design a general framework for building SNARKs for $\mathsf{REL}_{\mathsf{UCS}}$, which we call Zinc+ (section 2.2.2). This framework consists of two components:

Zinc+ PIOP compiler. A compiler that takes existing PIOPs over finite fields, and applies a so-called *lifting* to adapt them to our constraint setting, which, as we mentioned, involves domains such as $\mathbb{Z}, \mathbb{Q}$, $\mathbb{F}_q$, and polynomial rings $R[X]$ (for $R \in \{\mathbb{Z}, \mathbb{Q}, \mathbb{F}_q\}$). The compiled PIOP involves polynomial oracles defined over all the domains in the relation. Outside the PIOP model (i.e. when building a final SNARK), the correct typing (i.e., that each oracle's coefficients lie in the declared ring or field and satisfy the prescribed degree and bit-size bounds) and evaluations of the oracles are enforced via Zip+ (explained below). We prove that the Zinc+ PIOP achieves *round-by-round knowledge soundness* (theorem 2.5.7), with knowledge error $2^{-\lambda}$ per round (the PIOR has two rounds of verifier randomness, contributing at most $2 \cdot 2^{-\lambda}$ total knowledge error), where λ is the security parameter. Since round-by-round knowledge soundness composes with the Fiat–Shamir transform in the random oracle model [**BCS16**], this yields a knowledge-sound non-interactive argument (SNARK) when the PIOP is compiled with Zip+ and Fiat–Shamir (**??**).

Zinc+ in a nutshell. In simple terms, the Zinc+ PIOP compiler works as follows: first, the prover sends an oracle (or oracles) $[[\tilde{\mathbb{w}}]]$ to the multilinear extensions (MLE) of the witness vector(s) $\mathbb{w}$. Second, all ideal membership constraints $Q(\dots) \in \mathfrak{I}$ are reduced to strict equalities (over

$\mathbb{Q}, \mathbb{Q}[X], \mathbb{F}_q, \mathbb{F}_q[X]$, etc.) by checking that the evaluation on a random point of the MLE of $Q(...)$ belongs to $\mathfrak{I}$. This way, $Q(...) \in \mathfrak{I}$ is replaced by an MLE evaluation claim, which is a strict equality. Third, the strict equality is projected onto a random finite field $\mathbb{F}_q$ via one of the following maps, depending on whether our initial constraint is over $\mathbb{Q}[X]$ or $\mathbb{F}_q[X]$, respectively:

$$\begin{aligned} \psi_{q',a} : \mathbb{Q}[X] &\to \mathbb{F}_{q'} & \qquad \psi_b : \mathbb{F}_q[X] &\to \mathbb{F}_q \\ f(X) &\mapsto f(a) \mod q' & \qquad f(X) &\mapsto f(b) \end{aligned} \tag{2.1}$$

where q' is a random prime when working over $\mathbb{Q}[X]$, a is a random integer, and b is a random element of $\mathbb{F}_q$ (or of a field extension of $\mathbb{F}_q$ if q is small).

Fourth, the resulting finite field equality is proved using any suitable PIOP $\Pi_{\mathbb{F}}$ for constraints over a finite field $\mathbb{F}$. During this step, every time the verifier of $\Pi_{\mathbb{F}}$ queries the MLE of the projected witness at a point $\mathbf{r}$, the Zinc+ verifier lifts $\mathbf{r}$ and queries $[[\tilde{\mathfrak{w}}]]$ at the lifted point, and then projects the resulting evaluation onto $\mathbb{F}$.

Zip+. An Interactive Oracle Proof of Proximity (IOPP) for handling commitments to multilinear polynomials whose coefficients belong to the rings mentioned above, i.e. $\mathbb{Q}, \mathbb{Q}[X], \mathbb{F}_q, \mathbb{F}_q[X]$. We call this IOPP Zip+ (**??**). Zip+ is round-by-round knowledge sound, supports batched evaluations, and it can handle distance parameters up to the Mutual Correlated Agreement (MCA) bound of the underlying code (**??**) –unlike its predecessor Zip [**Zinc**], a PCS for multilinear polynomials with coefficients in the set $\mathbb{Q}^{<B}$ of rationals of bit-size less than B. Zip+ is a modern version of the Brakedown/Ligero PCS [**Brakedown**; **Ligero**] adapted to our rings and fields.

In short, Zip+ allows one to commit to multilinear polynomials with coefficients in $\mathbb{Q}^{<d,B}[X]$, where $\mathbb{Q}^{<d,B}[X]$ denotes the set of rational polynomials of degree less than d and coefficients in $\mathbb{Q}^{<B}$. The commitment is done by using an improved version of Zip to commit to d multilinear polynomials with coefficients in $\mathbb{Q}^{<B}$.

In section 2.2.3 we discuss the efficiency of Zip+ in isolation and in the broader context of our compiled SNARK and use-cases.

Integer Pseudo Reed–Solomon (IPRS) codes. We introduce a new linear code over the field

of rational numbers $\mathbb{Q}$ and over the ring of integers $\mathbb{Z}$, which we call *Integer Pseudo Reed–Solomon (IPRS) code* (**??**), and which we believe is of independent interest. This code is not a Reed–Solomon (RS) code over the integers, but it presents similar characteristics, namely it is minimum distance separable and admits a FFT encoding process. Additionally, IPRS codes do not blowup the bit-size of encoded messages (see next).

Our IOPP and multilinear PCS Zip+ relies at its core on linear error correcting codes over $\mathbb{Q}, \mathbb{Q}[X], \mathbb{F}_q$, and $\mathbb{F}_q[X]$. As with other code-based IOPPs, the performance of Zip+ is heavily influenced by both the minimum distance of the underlying code and the efficiency of its encoding. Additionally, when working over $\mathbb{Q}$ (or fields of size much larger than the size of the witness entries), one also needs to ensure that the bit-size of the codewords does not blowup compared to the bit-size of the messages.

IPRS codes in a nutshell. When working over small fields, Reed-Solomon (RS) codes are a natural choice as underlying code. These codes have optimal minimum distance and support efficient FFT-based encoding. One could try to use RS codes over $\mathbb{Q}$ or $\mathbb{Z}$ as well — for example, by naively defining RS codes over $\mathbb{Q}$ as the set of vectors obtained by evaluating low-degree polynomials on integer points. However, for practical code dimensions, the bit-size of the resulting codewords would be in the thousands or even millions, which would result in an inefficient IOPP. Alternatively, one could "lift" a standard RS code $\mathsf{RS}[\mathbb{F}_q, n, k]$ from a small field $\mathbb{F}_q$ onto $\mathbb{Q}$ by looking at the Vandermonde generating matrix of $\mathsf{RS}[\mathbb{F}_q, n, k]$ as an integer matrix with entries in $\{0, \dots, q-1\}$, and defining the lifted code as the set of rational linear combinations of the rows of this matrix. However, this code does not have, a priori, an efficient FFT encoding.

IPRS codes constitute a middle ground between the two aforementioned RS-based constructions. In short, they are obtained by "lifting" to $\mathbb{Q}$ the FFT encoding algorithm for a standard RS code over a small field $\mathbb{F}_q$. Concretely, we fix such an FFT algorithm (by specifying its radix, twiddle factors, depth, etc.), and then we look at its twiddle factors and other constants as integers from $\{-(q-1)/2, \dots, (q-1)/2\}$, and perform the FFT algorithm over $\mathbb{Q}$, without performing modular reduction. The IPRS code is defined as the set of vectors obtained by applying such lifted FFT algorithm. In **??** we discuss the performance and properties of these codes.

RAA and JEA codes. Non RS-based codes such as RAA and JEA codes [**jea_codes**; **blaze**] can be defined over $\mathbb{Q}$ and $\mathbb{Z}$ without blowup issues and with efficient encoding. However, their *proved* minimum distance is significantly smaller than that of RS codes and our IPRS codes.

Implementation and Evaluation. We partially implement our proof system (section 2.2.3). Concretely, we implement the Zip+ IOPP and a component of our PIOP. We arithmetize the SHA-256 compression function and the ECDSA verification algorithm as an instance of our UCS relation. Then we evaluate the implemented components on inputs corresponding to said arithmetizations. We expect the IOPP to constitute the most expensive part of our scheme, in prover time, proof size, and verifier time. Hence our benchmarks for the IOPP may be taken as a proxy for the performance of the full proof system on the arithmetized computations.

Our partial benchmarks provide preliminary evidence that, when run on computations of practical interest (such as SHA-256 hashing followed by ECDSA signature verification—see table 2.2), a fully optimized end-to-end implementation of our schemes may approach the performance of state-of-the-art SNARKs in prover time, with comparable or smaller proof size, and a fast verifier. We emphasize that our current benchmarks are partial: they cover the Zip+ IOPP (which we expect to dominate the overall cost), together with most steps of the Zinc+ PIOP, but not the full end-to-end pipeline. Therefore, the figures reported below should be understood as *indicative estimates*, not direct apples-to-apples comparisons with fully deployed systems. The next table describes the costs of our Zip+ IOPP compiled with Merkle tree commitments, which yields a hash-based polynomial commitment scheme. Throughout this paper, all reported Zip+ IOPP times include the cost of Merkle tree hashing. In table 2.8 we provide almost full prover performance for the Zinc+ PIOP. See **??** for a detailed description of our experiments.

Zip+ IOPP on trace of 8x SHA-256 + ECDSA		
Prover (ms)	Verifier (ms)	Proof (KB)
10.71	2.30	211

Table 2.1: Benchmarks for our Zip+ IOPP compiled with Merkle tree commitments, on a batched set of multilinear polynomials whose sizes and types correspond to our arithmetization of 8 SHA-256 compressions, followed by one ECDSA signature verification. All reported times include the cost of Merkle tree hashing. The compiled IOPP is expected to dominate all costs of our proof system (see section 2.2.3), and so we view these benchmarks as a rough estimate of times Zinc+ could achieve on this task. We note that ours is a PoC implementation.

The following table lists some computations of practical interest in the context of SNARKs, and the types of operations they involve.

Computation	$\mathbb{Z}/2^w\mathbb{Z}$	**Bit**	$\mathbb{F}_q$	$\mathbb{F}_{2^w}$	$\mathbb{Z}/n\mathbb{Z}$	$\mathbb{Z}$ or $\mathbb{Q}$	R_{latt}
Cryptographic primitives							
Classic hashes (SHA, etc.)							
AES / symmetric encr.							
Elliptic curve crypt.							
Lattice crypt.							
RSA crypt.							
Applications							
zkVMs / CPU emulation							
zkID							
Blockchain transactions							
TLS (AEAD)							
Proof recursion (classic hash)							
Hash-to-curve							
FHE							
Machine learning							
Finance							

Table 2.2: Common computations and the types of arithmetic they require. Columns named with a ring R refer to "addition and multiplication R". The Bit column refers to operations such as XOR, AND, rotations, shifts, etc. For an integer $m \geq 1$, $\mathbb{Z}/m\mathbb{Z}$ denotes integers modulo m (where composite m are relevant in RSA cryptography and lattices, FHE, etc.). Further, $\mathbb{F}_q$ is a finite field (often prime); $\mathbb{F}_{2^w}$ denotes fields of size 2^w; and R_{latt} denotes rings relevant to lattice-based cryptography such as cyclotomic rings $\mathbb{Z}[X]/(X^n + 1)$.
zkID refers to performing a classic hash followed by a signature verification with, e.g. ECDSA or RSA. Proof recursion refers to proving acceptance of a SNARK verifier in a context where the original proof system uses classic hashes (due to Fiat-Shamir or Merkle tree commitments); it has $\mathbb{Z}/n\mathbb{Z}$ checkmarked as it sometimes requires using different moduli, e.g. in IVC recursion [**Nova**]. Hash-to-curve refers to algorithms that iteratively perform hashes and check membership of points on an elliptic curve. zkVMs may use so-called precompiles/builtins to perform computations such as ECDSA verification. In this case they also involve prime field arithmetic.

2.1.2 Related work

SNARKs for integer constraints. Zinc [**Zinc**] presents a framework for building proof systems for constraints over $\mathbb{Z}$ or $\mathbb{Q}$ and introduces a hash-based PCS for polynomials with coefficients in $\mathbb{Q}$, built with a tensor IOPP similar to the one introduced in Brakedown [**Brakedown**].

Our work improves upon [**Zinc**] significantly in the following ways. (1) our UCS constraints strictly generalize Zinc's, which is restricted to constraints over $\mathbb{Z}$ and $\mathbb{Q}$, by additionally supporting multiple finite fields, polynomial rings $R[X]$, and ideal-membership predicates. UCS supports simultaneously multiple rings and fields of the form $R^{<d}[X]$ and ideal membership constraints, which, as we demonstrate, enriches the expressivity of the constraint system substantially. (2) We design a PIOP framework for these more general constraints. Additionally, we prove round-by-round knowledge soundness, allowing for Fiat-Shamir compilation. (3) Our IOPP Zip+ is an improvement to the PCS from [**Zinc**] in that it supports polynomials over a wide range of rings and fields, can handle proximity parameters up to the MCA bound of the underlying code (**??**) (this greatly helps reduce proof size and verifier time), merges the so-called "test" and "evaluate" phases into one, is proved to be round-by-round knowledge sound, and has batching capabilities. (4) We improve upon the underlying linear code with our IPRS codes. (5) We demonstrate the practicality of our approach with our implementation and evaluation.

Zartan [**Zartan**] builds SNARKs over $\mathbb{Z}$ by compiling so-called mod-AHPs with a PCS for integral polynomials based on hidden-order groups [**BlockDARK**; **BunzFisch22**]. We adopt the idea from [**Zartan**] of working modulo a random prime, but replace the hidden-order-group PCS with a hash-based PCS (Zip+), thereby avoiding the stronger cryptographic assumptions (e.g. the adaptive root assumption) that hidden-order-group commitments require. Moreover, as in the comparison with Zinc [**Zinc**], our scheme works with constraint systems that are substantially more expressive than Zartan's.

Binary-field SNARKs. Binius [**binius**; **fribinius**] is a series of SNARKs for relations over binary fields $\mathbb{F}_{2^k}$. Binary fields are more friendly to bitstring operations than prime fields. For example, bitwise XOR can be modeled as addition in $\mathbb{F}_{2^k}$. However, AND, rotation, and modular-2^w

arithmetic remain non-native in binary fields. Despite this, Binius achieves significant improvements on computations involving bitstring operations and arithmetic modulo 2^w, when compared to prime-field SNARKs [**ethproofs**]. We believe this is in part due to $\mathbb{F}_{2^k}$ being more amenable to some of these operations, and due to the *pay-per-bit* capacity of Binius' IOPP.

Our benchmarks show that, for computations dominated by bitstring and modular-integer operations (e.g. SHA-256 hashing), Zinc+ achieves performance competitive with Binius. For computations that additionally involve prime-field arithmetic (e.g. SHA-256 followed by ECDSA signature verification), our results indicate a clear advantage, since Zinc+ handles prime-field constraints natively whereas Binius must emulate them over binary fields. While not evaluated in the benchmarks of this paper, Zinc+ can natively handle further constraints such as lattice-based encryption combined with classic hashing (see section 2.2.1).

Proof systems over rings. There are a number of works describing SNARKs over different types of specific rings, such as cyclotomic rings [**beullens2023labrador**], Galois rings [**new_rings_work_one**; **new_rings_work_two**], and abstract rings with large exceptional sets [**bootle2021sumcheck**; **attema2022vector**; **soria2022doubly**]. Approaches for proving statements modulo 2^n are based on VOLE techniques (see, e.g. [**C:LinXinYao24**; **baum2021appenzeller**; **baum2022moz**]) but yield non-succinct schemes. Unlike our work, none of these approaches support the wide range of rings and fields that we can handle with our framework, including constraints over $\mathbb{Z}$ and $\mathbb{Z}[X]$. The recent works FREPack [**FREPack**] and Swirl [**swirl**] treat the problem of allowing different constraint domains within the same proof system. FREPack focuses on highly repetitive computations and packs multiple field operations into a single CCS/R1CS constraint via the Chinese Remainder Theorem, achieving significant speedups for structured circuits but limiting its flexibility: the rings used must factor the chosen CRS modulus, and the approach does not natively support polynomial-ring constraints or ideal-membership predicates as UCS does. Swirl stacks WHIR with LogUp to handle lookups across different subfields within a single proof, but remains in a finite-field setting and does not address constraints over $\mathbb{Z}$, $\mathbb{Q}$, or polynomial rings $R[X]$. In contrast, Zinc+ supports constraints over $\mathbb{Z}, \mathbb{Q}, \mathbb{F}_q$ and their polynomial ring counterparts simultaneously, with ideal-membership predicates that natively capture operations such as bitstring rotation and modular

reduction without additional encoding.

Applications. In this work we discuss the example of SHA-256 hashing followed by ECDSA signature verification. Two recent approaches to this, which additionally include document parsing, are [**longfellow**; **vega**]. As future work, we plan to include document parsing in our arithmetizations and compare our scheme with these approaches.

2.1.3 Acknowledgments

We thank Benedikt Bünz, Giacomo Fenzi, Amit Kumar, Alin Tomescu, and Arantxa Zapico for helpful discussions and contributions to the implementation.

The authors used AI-based tools (GitHub Copilot, ChatGPT) for grammar checking, editing, and assisting with code generation during the preparation of this manuscript. The authors bear full responsibility for the correctness and originality of all content.

2.2 Technical Overview

We now provide an overview of the main contributions of this work.

2.2.1 Universal Constraint Systems (UCS)

To describe and motivate our framework we focus on two running examples. The first involves arithmetic modulo 2^w, bitstring operations such as XOR and rotation, and arithmetic modulo an arbitrary integer (possibly non-prime, e.g. a power of two). The second revolves around Hamming weight computations, lattice based operations, and a toy electronic voting application.

We introduce some notation. Further background can be found in section 4.2. Given a ring or field R, we write $R^{<d}[X]$ to denote polynomials on the variable X and of degree less than d. Note that $R^{<1}[X] = R$. Given a bit size $w \geq 1$ we let $[0\,..\,2^w - 1]$ denote the set of integers $\{0, \dots, 2^w - 1\}$. We define the set of so-called *bit-polynomials* of width w as

$$\{0,1\}^{<w}[X] \;:=\; \left\{ \sum_{i=0}^{w-1} b_i\, X^i \;\middle|\; b_i \in \{0,1\} \right\} \;\subseteq\; \mathbb{Z}[X]. \tag{2.2}$$

The set $\{0,1\}^{<w}[X]$ is in bijective correspondence with $[0\,..\,2^w-1]$ via the evaluation map

$$\psi_2 \colon \mathbb{Z}[X] \to \mathbb{Z} \qquad \text{defined by} \qquad \psi_2(u(X)) := u(2),$$

which restricts to a bijection $\{0,1\}^{<w}[X] \xrightarrow{\sim} [0\,..\,2^w-1]$. Note that ψ_2 is the canonical surjective ring homomorphism $\mathbb{Z}[X] \twoheadrightarrow \mathbb{Z}[X]/(X-2) \cong \mathbb{Z}$. Here, $(X-2)$ denotes the ideal generated by $X-2$ in the ring $\mathbb{Z}[X]$. See section 4.2 for background. We also let

$$\phi_2 \colon \mathbb{Z}[X] \to \mathbb{F}_2[X]$$

denote reduction modulo 2 coefficient-wise.

We emphasize that $\{0,1\}^{<w}[X]$ is formally a set of *integer* polynomials, and for the purposes of this paper this is not the same as $\mathbb{F}_2^{<w}[X]$. It is important that commitments to polynomials in $\{0,1\}^{<w}[X]$ are over $\mathbb{Z}[X]$, and cannot be taken modulo 2.

A motivating example: Simultaneous bitstring operations and arithmetic mod 2^w and mod primes

Our first example has the following function f at its core. Let $r \in [w-1]$ be a rotation parameter. Let $n \in [0\,..\,2^w]$ be a an integer, possibly non-prime. Now define

$$\begin{aligned} f : [0\,..\,2^w-1] &\to [0\,..\,2^w-1], \\ x &\mapsto (x^2 \bmod n) \;\; \mathsf{XOR} \;\; \mathsf{ROTR}^r(x), \end{aligned} \tag{2.3}$$

where XOR is the "bitwise exclusive or" (XOR) operation, and ROTR^r is right rotation by r bits. Precisely, for two integers $u, v \in [0\,..\,2^w-1]$ with binary representations $u = \sum_{i=0}^{w-1} u_i 2^i$ and $v = \sum_{i=0}^{w-1} v_i 2^i$, we have $u \;\mathsf{XOR}\; v = \sum_{i=0}^{w-1} (u_i + v_i \bmod 2) 2^i$, and $\mathsf{ROTR}^r(u) = \sum_{i=0}^{w-1} u_{(i+r) \bmod w} 2^i$. We extend these operations to the set $\{0,1\}^{<w}[X]$ by applying them coefficient wise. In the definition of f (eq. (2.3)), the expression $x^2 \bmod n$ means that x is understood as an integer in $[0\,..\,2^w-1]$, squared, and then the canonical representative of x^2 modulo 2^w in $[0, n-1]$ is returned.

We view f as a toy function roughly representative of computations involving some or all of the

following: arithmetic modulo 2^w, bitstring operations, and prime field operations. Concretely, we think of f as a proxy for some of the subroutines in computations such as classic hashing, CPU and zkVM instructions, zkID authentication systems, TLS, etc. See table 2.2.

Given a number of iterations $n \geq 1$, our goal is to design a proof system for the relation REL_f defined as

$$\mathsf{REL}_f = \left\{ (\mathbb{x}; \mathbb{w}) \;\middle|\; \begin{array}{l} \mathbb{x} = (u_1, u_n) \in [0 \,..\, 2^w - 1]^2, \\ \mathbb{w} = (u_2, \dots, u_{n-1}) \in [0 \,..\, 2^w - 1]^{n-2}, \\ u_{i+1} = f(u_i) \text{ for all } i \in [n-1] \end{array} \right\}. \tag{2.4}$$

Before diving into arithmetizing REL_f, we make a series of observations about the operations used by the function f, which will motivate our arithmetization framework.

XOR and AND For any $u, v \in \{0,1\}^{<w}[X]$, the following identity holds in $\mathbb{Z}[X]$:

$$u + v \;=\; (u \;\mathsf{XOR}\; v) + 2 \cdot (u \;\mathsf{AND}\; v)\,. \qquad \text{in } \mathbb{Z}[X], \tag{2.5}$$

The converse also holds: if $u, v, z, a \in \{0,1\}^{<w}[X]$ satisfy $u + v = z + 2 \cdot a$ in $\mathbb{Z}[X]$, then $z = u \;\mathsf{XOR}\; v$ and $a = u \;\mathsf{AND}\; v$. Both directions follow from the fact that $b + b' = (b \;\mathsf{XOR}\; b') + 2\,(b \;\mathsf{AND}\; b')$ for any two $b, b' \in \{0,1\}$, applied coefficient-wise.

Hence, informally speaking, provided the witness elements belong to $\{0,1\}^{<w}[X]$, the identity (2.5) arithmetizes both XOR and AND over $\mathbb{Z}[X]$ simultaneously.

A further consequence of (2.5) is that for any $u, v, z \in \{0,1\}^{<w}[X]$,

$$z = u \;\mathsf{XOR}\; v \quad \Longleftrightarrow z = u + v \mod 2, \tag{2.6}$$

In summary, one can expose the XOR of two elements of $\{0,1\}^{<w}[X]$ either via a single equality over $\mathbb{Z}[X]$ (which additionally yields the AND) or through addition modulo 2. We leverage both approaches in our arithmetization of SHA-256.

Rotation through ideal membership For any $u \in \{0,1\}^{<w}[X]$, the following ideal-membership relation holds in $\mathbb{Z}[X]$:

$$\mathsf{ROTR}^r(u) - X^{w-r} \cdot u \in (X^w - 1). \tag{2.7}$$

where $(X^w - 1)$ denotes the ideal in $\mathbb{Z}[X]$ generated by the polynomial $X^w - 1$. Conversely, for any two $u, v \in \{0,1\}^{<w}[X]$, if $v - X^{w-r} \cdot u \in (X^w - 1)$ then $v = \mathsf{ROTR}^r(u)$ (**??**). Hence rotation *for elements in* $\{0,1\}^{<w}[X]$ can be enforced by the ideal-membership predicate $v - X^{w-r}u \in (X^w - 1)$.

At this point one may now wonder whether such a predicate can be efficiently handled in proof systems. The answer is that this can be achieved by checking that certain appropriate multilinear extension, when evaluated at a random point, returns a value that belongs to the ideal. This is analogous to the technique in classical proof systems where the MLE of the polynomial constraints composed with the MLE of the witnesses evaluates to zero at a randomly sampled point. Here, instead, evaluation to zero (i.e. membership to the zero ideal) is replaced by ideal membership. See lemma 2.2.5.

Mixing XOR and rotation Given elements $u, v, z \in \{0,1\}^{<w}[X]$, we want to arithmetize the constraint $z = u \ \mathsf{XOR}\ \mathsf{ROTR}^r(v)$. To do so, observe that as long as u, v, z belong to $\{0,1\}^{<w}[X]$, the following equivalence holds:

$$z = u \ \mathsf{XOR}\ \mathsf{ROTR}^r(v) \quad \Longleftrightarrow \quad \phi_2(u) + \phi_2(v) \cdot X^{w-r} - \phi_2(z) \in (X^w - 1) \qquad \text{in } \mathbb{F}_2[X], \tag{2.8}$$

The equivalence in (2.8) can be generalized to more complex expressions involving several XORs and rotations. It is particularly useful for arithmetizing some inner functions of SHA-256 and related hashes and ciphers.

Linking bit-polynomials to integers, and squaring modulo an arbitrary integer n We would now like to, given $u, v \in \{0,1\}^{<w}[X]$, express the constraint that the integer represented by u, i.e. $\psi_2(u)$, is the square of the integer represented by v, i.e. $\psi_2(v)$, modulo an arbitrary integer

n. We can do so by observing the following:

$$\psi_2(v) = \psi_2(u)^2 \bmod n \quad \iff \quad v - u^2 + n \cdot \mu \in (X-2) \qquad \text{in } \mathbb{Z}[X], \tag{2.9}$$

for some $\mu \in \mathbb{Z}$. To see why, assume there exists $\mu \in \mathbb{Z}$ such that $v(X) - u(X)^2 + n \cdot \mu \in (X-2)$ in $\mathbb{Z}[X]$. Then $v(X) - u(X)^2 + n \cdot \mu$ vanishes at $X = 2$, so $v(2) - u(2)^2 + n \cdot \mu = 0$ in $\mathbb{Z}$, which gives $\psi_2(v) = \psi_2(u)^2 \bmod n$. The forward direction is analogous.

A similar approach arithmetizes other algebraic modular operations. An important underlying principle is that membership to the ideal $(X-2)$ links bit-representations (modeled as elements from $\{0,1\}^{<w}[X]$) to the integer they represent, as long as elements are constrained to belong to $\{0,1\}^{<w}[X]$ and $[0\,..\,2^w - 1]$.

Remark 2.2.1 (Lookup or typing constraints)**.** In all examples above, the typing of elements is crucial. I.e. most of the ideal membership constraints described above lose their intended effect if one does not pair them with *lookup* or *typing* constraints, e.g. the constraint (2.9) must be accompanied of the lookup constraints $v, u \in \{0,1\}^{<w}[X]$ and $\mu \in \mathbb{Z}$. Because of this, our arithmetizations consist mostly of algebraic constraints involving witness elements (these may be zero equalities or ideal membership constraints), plus lookup or typing constraints.

Arithmetization of REL_f We now put together a full arithmetization for the relation REL_f. Recall that each application of the function f computes

$$u_{t+1} \;=\; (u_t^2 \bmod n) \;\; \mathsf{XOR} \;\; \mathsf{ROTR}^r(u_t).$$

To arithmetize the full $(n-1)$-fold iteration, we introduce witness vectors for each step and impose constraints that enforce the transition given by f. The entire arithmetization uses three witness elements per step, two typed in $\{0,1\}^{<w}[X]$ and one in $\mathbb{Z}$, and two constraints per transition.

<u>*Witness.*</u> For each step $t \in [n]$ we introduce three witness elements. We think of the witness as a $n \times 3$ matrix, or *trace* (even though our definition of UCS does not restrict to working with traces and AIR-like systems). We denote the inverse of ψ_2 with $\hat{\cdot}$, so that for all $u \in [0\,..\,2^w - 1]$ we have

$\hat{u} = \hat{u}(X) \in \{0,1\}^{<w}[X]$ and $\hat{u}(2) = u$. The following table describes each witness element, its intended meaning, and its domain .

Entry	Intended meaning	Lookup constraint
$\hat{u}_t$	bit-polynomial representation of u_t at step t	$\in \{0,1\}^{<w}[X] \subset \mathbb{Z}[X]$
$\hat{s}_t$	bit-polynomial representation of $(u_t)^2 \bmod n$	$\in \{0,1\}^{<w}[X] \subset \mathbb{Z}[X]$
μ_t	the quotient such that $(u_t)^2 = s_t + \mu_t\, n$	$\in \mathbb{Z}$

<u>*Transition constraints.*</u> For each step $t \in [n-1]$ we impose two constraints.

1. **Squaring modulo n.** $\hat{s}_t - (\hat{u}_t)^2 + n \cdot \mu_t \in (X-2)$ in $\mathbb{Z}[X]$. By eq. (2.9), this forces $s_t \equiv (u_t)^2 \pmod{n}$.

2. **XOR and rotation.** $\phi_2(\hat{u}_{t+1}) - \phi_2(\hat{s}_t) - X^{w-r} \cdot \phi_2(\hat{u}_t) \in (X^w - 1)$ in $\mathbb{F}_2[X]$. By (2.8), this enforces that $\hat{u}_{t+1} = \hat{s}_t \ \mathsf{XOR}\ \mathsf{ROTR}^r(\hat{u}_t)$.

We omit boundary constraints for brevity.

The reader may appreciate the proposed arithmetization as being both conceptually simple and small (in the sense of it requiring a small number of constraints given the relation sought to arithmetize). The question now is whether such arithmetization *is useful in practice*, i.e. whether it can be efficiently handled by proof systems. One of the main contributions of this work is to show that the answer is yes. We outline why and how in the next sections of the present technical overview.

Boundary constraints.

table 2.3 summarizes the UAIR$^+$ trace layout and non-boundary constraint structure for the iterated computation REL_f.

Further examples

The same ideal-membership approach extends to other settings; we highlight three (see **??** for full worked-out examples).

- **Lattice-based operations.** Constraints in, for example, cyclotomic rings $R_q = \mathbb{F}_q[X]/(\Phi(X))$ can be enforced through ideal-membership predicates over $\mathbb{F}_q[X]$ where the ideal is $(\Phi(X))$. The

Trace layout		*Constraints*		
#Cols	Type	#Mon.	Ideal	Ring
2	$\{0,1\}^{<w}[X]$	3	(X^w-1)	$\subseteq \mathbb{F}_2[X]$
1	$\mathbb{Z}$	3	$(X-2)$	$\subseteq \mathbb{Q}[X]$

Table 2.3: Summary of our arithmetization of REL_f. The trace has n rows (as many as iterations of f, plus one), and three columns, two typed in $\{0,1\}^{<w}[X]$ and another in $\mathbb{Z}$. The "#Mon." column counts the total number of monomials across all constraints **??** sharing the same ideal and ring. All monomials are of degree 1. Public-input, boundary constraints, and shifted row accessed (cf. **??**) are omitted for simplicity.

typing of the witnesses can either be in some subset of $\mathbb{Z}[X]$ or in $\mathbb{F}_q[X]$, depending on the application (see the discussion after definition 2.4.1 for more details).

- **Hamming weight.** For any $u \in \{0,1\}^{<w}[X]$, let $\mathsf{wt}(u)$ denote the number of nonzero coefficients of u. Then, given $c \in \mathbb{Z}$ we have that $\mathsf{wt}(u) = c \iff u - c \in (X-1)$ in $\mathbb{Z}[X]$. This is because evaluation at $X = 1$ sums coefficients (see **??**).

Universal Constraint Systems (UCS)

We now introduce the main relation we will analyse. We call it the *Universal Constraint System* (UCS) relation $\mathsf{REL}_{\mathsf{UCS}}$.

A UCS instance is specified by an *index* $\mathbb{i}$ that fixes a witness-vector length m, a number of rows n, a tuple of prime powers $\mathbf{q} = (q_1, \dots, q_{|\mathbf{q}|})$, a bit-size bound B, degree bounds $\mathbf{d} = (d_0, \dots, d_{|\mathbf{q}|})$, and a set of constraints $\mathcal{C}$ described below.

The witness consists of $|\mathbf{q}| + 1$ vectors $\mathbf{f}_0, \dots, \mathbf{f}_{|\mathbf{q}|}$, each of length m. The vectors are typed as

$$\mathbf{f}_0 \in \left(\mathbb{Q}^{<d_0,B}[X]\right)^m, \qquad \mathbf{f}_i \in \left(\mathbb{F}_{q_i}^{<d_i}[X]\right)^m \text{ for all } i \in [|\mathbf{q}|],$$

where, recall, $R^{<d,B}[X]$ denotes the set of polynomials of degree $< d$ with coefficients in R, of bit-length less than B (when R is a field, we drop the bit bound and allow coefficients in all R). The entries of the witness vectors are thus *polynomials themselves*. In particular, the multilinear extension (MLE) (cf. section 2.3.1) of $\mathbf{f}_i$ is a multilinear polynomial whose coefficients lie in $\mathbb{Q}[X]$ or $\mathbb{F}_{q_i}[X]$—that is, a polynomial whose coefficients are polynomials. Regarding why $\mathbf{f}_0$ is typed in

$\mathbb{Q}^{<d_0,B}[X]$ rather than $\mathbb{Z}^{<d_0,B}[X]$, see the technical observations after definition 2.4.1.

The public input is a vector $\mathbf{y} \in (\mathbb{Q}^{<d_0,B}[X])^{\ell}$.

Each constraint in $\mathcal{C}$ is a tuple $(Q, \mathfrak{I})$, where Q is a polynomial with coefficients either in $\mathbb{Q}$ or $\mathbb{F}_{q_i}$ for some $i \in [|\mathbf{q}|]$, and $\mathfrak{I}$ is an ideal of $\mathbb{Q}[X]$ or $\mathbb{F}_{q_i}[X]$, respectively. In the first case, $Q = Q(K, \mathbf{y}, \mathbf{f}_0)$ is a polynomial on $\mathbf{f}_0$ and $\mathbf{y}$ and a row variable K intended to range in $[n]$. We call $(Q, \mathfrak{I})$ a $\mathbb{Q}[X]$-*constraint*. In the later case, $Q = Q(K, \phi_{q_i}(\mathbf{y}), \phi_{q_i}(\mathbf{f}_0), \mathbf{f}_i)$ is a polynomial on $\mathbf{f}_i$, $\phi_{q_i}(\mathbf{f}_0)$, $\phi_{q_i}(\mathbf{y})$ and K, where, recall, ϕ_{q_i} denotes (coefficient-wise) reduction modulo q_i. We call this later $(Q, \mathfrak{I})$ a $\mathbb{F}_{q_i}$-*constraint*. See remark 2.2.3 for a short comment on having $\phi_{q_i}(\mathbf{f}_0)$, $\phi_{q_i}(\mathbf{y})$ be well-defined.

The relation $\mathsf{REL}_{\mathsf{UCS}}$ consists of all triples $(\mathbb{i}, \mathbb{x}; \mathbb{w})$ such that, for every constraint $(Q, \mathfrak{I}) \in \mathcal{C}$ and every row $k \in [n]$, the following holds:

$$
\begin{aligned}
&Q(k, \mathbf{y}, \mathbf{f}_0) \in \mathfrak{I} && \text{if } (Q, \mathfrak{I}) \text{ is a } \mathbb{Q}[X]\text{-constraint,} \\
&Q(k, \phi_{q_i}(\mathbf{y}), \phi_{q_i}(\mathbf{f}_0), \mathbf{f}_i) \in \mathfrak{I} && \text{if } (Q, \mathfrak{I}) \text{ is a } \mathbb{F}_{q_i}[X]\text{-constraint}, i \in [|\mathbf{q}|].
\end{aligned}
$$

Remark 2.2.2 (Shared witness principle)**.** The witness $\mathbf{f}_0$ acts as a shared witness across different modular arithmetic. This allowed in eq. (2.3) to have a single bit-polynomial column $\hat{u} \in \{0,1\}^{<w}[X] \subset \mathbb{Z}[X]$ simultaneously participate in a squaring of integers modulo some integer, and XOR operations expressed as additions modulo 2.

The formal definition of UCS is given in definition 2.4.1. When restricted to a single finite field and the zero ideal, UCS subsumes standard constraint systems—R1CS, AIR, CCS, Plonkish, and lookup constraints—as special cases (cf. sections 2.4.2 and 2.4.3).

After definition 2.4.1 we give a detailed discussion of the design choices behind the definition of UCS, and further technical observations about the definition.

Remark 2.2.3 (Well-defined projections of $\mathbf{f}_0$ and $\mathbf{y}$)**.** In the UCS definition, we also require that the reductions $\phi_{q_i}(\mathbf{f}_0)$, $\phi_{q_i}(\mathbf{y})$ are well-defined. Namely, we ask that $\mathbf{f}_0$ and $\mathbf{y}$ have entries in $\mathbb{Z}_{(q_i)}^{<d_0,B}[X]$, where, in short, $\mathbb{Z}_{(q_i)}$ is the subring of $\mathbb{Q}$ formed by rationals whose denominator is not divisible by q_i (see section 2.3.1).

In practice, this requirement is usually subsumed (i.e. implied) by the lookup or typing constraints on $\mathbf{f}_0$ and $\mathbf{y}$. For example, in our arithmetization of REL_f, the witness $\mathbf{f}_0$ contains bit-polynomial

entries, which are polynomials with integer coefficients, which belong to $\mathbb{Z}_{(q_i)}$ for all prime or prime power q_i.

2.2.2 Zinc+: a PIOP for UCS from PIOPs over finite fields

We now describe the Zinc+ compiler, which constructs a PIOP for our UCS relation $\mathsf{REL}_{\mathsf{UCS}}$ from PIOPs for standard algebraic relations over finite fields. The compiler generalizes the Zinc compiler from [**Zinc**], which handles algebraic relations over $\mathbb{Q}$.

For clarity, we present a simplified version that focuses on UCS instances that only have $\mathbb{Q}[X]$-constraints (i.e. instances where $\mathbf{q} = \emptyset$, so there are no $\mathbb{F}_{q_i}[X]$-constraints). The general case is discussed in remark 2.2.6 below and treated fully in section 2.5.

Recall that for a prime q, $\mathbb{Z}_{(q)}$ denotes the subring of $\mathbb{Q}$ formed by rationals whose denominator (in lowest form) is not divisible by q, cf. section 2.3.1. We let

$$\phi_q : \mathbb{Z}_{(q)}[X] \to \mathbb{F}_q[X]$$

be the natural projection that reduces coefficients modulo q. We also let

$$\begin{aligned} \psi_a : \mathbb{F}_q[X] &\to \mathbb{F}_q \\ f &\mapsto f(a) \end{aligned}$$

be the evaluation map that replaces X by a. Both ϕ_q and ψ_a are ring homomorphisms. We let

$$\psi_{q,a} = \psi_a \circ \phi_q : \mathbb{Z}_{(q)}[X] \to \mathbb{F}_q$$

be the composition of the two maps.

From UCS constraints over $\mathbb{Q}[X]$ to algebraic constraints over a finite field. Recall that a $\mathbb{Q}[X]$-constraint in $\mathsf{REL}_{\mathsf{UCS}}$ takes the form

$$Q(k, \mathbf{y}, \mathbf{f}_0) \in \mathfrak{I}, \qquad \text{for all rows } k \in [n], \tag{2.10}$$

where Q is a polynomial with coefficients in $\mathbb{Q}$ and involving k, the public input $\mathbf{y} \in \mathbb{Q}^{<d_0,B}[X]^\ell$ and the witness $\mathbf{f}_0 \in \mathbb{Q}^{<d_0,B}[X]^m$, and $\mathfrak{I} \subseteq \mathbb{Q}[X]$ is an ideal. Assume $n = 2^\mu$ and identify $[n]$ with $\{0,1\}^\mu$. We assume without loss of generality that $\mathfrak{I} \neq \mathbb{Q}[X]$, since otherwise the constraint is always satisfied.

Our Zinc+ compiler first reduces the constraint (2.10) to standard algebraic constraints over a finite field $\mathbb{F}_q$ via a three-step process. After that, it invokes any suitable PIOP working over $\mathbb{F}_q$ to certify said constraints. The three-step process is formed by (1) a randomized projection from $\mathbb{Z}_{(q)}[X] \subseteq \mathbb{Q}[X]$ onto $\mathbb{F}_q[X]$, where q is a randomly sampled prime, which reduces (2.10) to an ideal membership constraint over $\mathbb{F}_q[X]$ of the form

$$\phi_q(Q)(k, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \in \mathfrak{I}', \qquad \text{for all rows } k \in [n], \tag{2.11}$$

where $\mathfrak{I}' = \phi_q(\mathfrak{I} \cap \mathbb{Z}_{(q)})$; (2) a randomized *ideal check* that replaces the 2^μ ideal membership constraints in (2.11) with a strict equality over $\mathbb{F}_q[X]$; and (3) a randomized *projection* from $\mathbb{F}_q[X]$ onto the finite field $\mathbb{F}_q$, which turns the strict equality over $\mathbb{F}_q[X]$ from Step (2) into an equality over $\mathbb{F}_q$.

We note that ϕ_q is only well-defined on the subring $\mathbb{Z}_{(q)}[X]$ of $\mathbb{Q}[X]$. In our completeness and security proofs, we show that for random q the probability that the relevant elements do not belong to this subring is negligible. For brevity, we do not consider this issue further in the present overview. We refer to **??** for the technical details, or to the technical overview of the Zinc paper [**Zinc**], where this matter is discussed thoroughly.

We remark that both Zartan [**Zartan**] and Zinc [**Zinc**] already used a randomized projections onto finite fields. In particular, [**Zartan**] projects $\mathbb{Z}$ onto $\mathbb{F}_q$ for q a random prime, and [**Zinc**] projects $\mathbb{Z}_{(q)}$ onto $\mathbb{F}_q$. Overall, Zinc+ employs the projection given by $\psi_{q,a} = \psi_a \circ \phi_q$, which is applied in a two-stage manner: first ϕ_q is applied in Step (1), and then ψ_a is applied in Step (3).

We next describe the Zinc+ compiler in more detail.

Step 1: Prime projection from $\mathbb{Q}[X]$ onto $\mathbb{F}_q[X]$. The verifier samples a random $\Omega(\lambda)$-bit prime q. The constraint (2.10) is replaced by

$$\phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \in \mathfrak{I}', \qquad \text{in } \mathbb{F}_q[X], \quad \text{for all } \mathbf{b} \in \{0,1\}^\mu,$$

where $\mathfrak{I}' = \phi_q(\mathfrak{I} \cap \mathbb{Z}_{(q)}[X])$.

Proof sketch of soundness for Step 1. When $\mathfrak{I} = (0)$, if $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) = 0$ fails over $\mathbb{Q}[X]$ for some $\mathbf{b}$, the element $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) \in \mathbb{Q}[X]$ is a nonzero polynomial in $\mathbb{Q}[X]$, and it vanishes modulo q only when q divides the numerators of its coefficients in lowest form. Say that such a prime q is "bad". By a counting argument (cf. lemma 2.5.8 and [**Zinc**]), only polynomially many primes cause this, so the probability of sampling a bad prime is negligible provided the set of sampleable primes is large enough (see remark 2.2.4 for some comments on this).

Now assume $\mathfrak{I} \neq (0)$. We want to show that if $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) \notin \mathfrak{I}$ for some $\mathbf{b}$, then $\phi_q(Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)) \notin \mathfrak{I}'$ except with negligible probability over the choice of q. Every ideal of $\mathbb{Q}[X]$ (and of $\mathbb{F}_q[X]$) is principal. Let g denote the monic generator of $\mathfrak{I}$. Since $\mathfrak{I} \neq \mathbb{Q}[X]$, we have $\deg g \geq 1$. Because g is monic, $\phi_q(g)$ has the same degree as g whenever $g \in \mathbb{Z}_{(q)}[X]$, which fails only for the finitely many primes dividing a denominator of some coefficient of g; in particular $\mathfrak{I}' = (\phi_q(g))$ with $\deg \phi_q(g) = \deg g$ except with negligible probability.

If $\deg Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) < \deg g$, consider two cases. If $\phi_q(Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)) = 0$ in $\mathbb{F}_q[X]$, then $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)$ is a nonzero element of $\mathbb{Q}[X]$ that vanishes modulo q, and the counting argument from the $\mathfrak{I} = (0)$ case shows this happens with negligible probability. Otherwise $\phi_q(Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)) \neq 0$, and since its degree is less than $\deg \phi_q(g)$, it cannot belong to $(\phi_q(g)) = \mathfrak{I}'$, so soundness holds.

If $\deg Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) \geq \deg g$, we write $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) = g \cdot h + r$ via Euclidean division in $\mathbb{Q}[X]$, where $\deg r < \deg g$. Euclidean division is exact over $\mathbb{Q}[X]$ (since $\mathbb{Q}$ is a field), and the quotient h and remainder r are uniquely determined. Because $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) \notin (g)$, the remainder r is nonzero. Moreover, since g is monic, the Euclidean division algorithm introduces no new denominator primes: each step subtracts a monomial multiple of g from the current polynomial, and since g has leading coefficient 1, no denominator prime factors, other than the ones already present in Q and g, are

used. Consequently, the coefficients of both h and r lie in $\mathbb{Z}_{(q)}$ whenever those of $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)$ and g do (which holds except for negligibly many primes q). Once we have $r \neq 0$ with $\deg r < \deg g$, the low-degree argument above applies to r after projection: $\phi_q(Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)) = \phi_q(g) \cdot \phi_q(h) + \phi_q(r)$, and since $\deg \phi_q(r) < \deg \phi_q(g)$, the element $\phi_q(Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0))$ can belong to $(\phi_q(g)) = \mathfrak{I}'$ only if $\phi_q(r) = 0$, which happens with negligible probability by the counting argument for the zero-ideal case. See lemma 2.5.8 for the precise bound on the number of bad primes. □

Remark 2.2.4 (Correct typing of $\mathbf{y}$ and $\mathbf{f}_0$)**.** In the above argument, it is essential that the entries of $\mathbf{y}$ and $\mathbf{f}_0$ belong to $\mathbb{Q}^{<d_0,B}[X]$, i.e. that the entries are polynomials of degree bounded by d_0, and with coefficients of bit-size less than B. This is necessary to conclude that only polynomially many primes can divide the coefficients of $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)$. In our PIOP soundness model, we assume malicious PIOP provers send oracles to polynomials whose coefficients belong to $\mathbb{Q}^{<d_0,B}[X]$ (and, perhaps, other appropriately typed sets). We delegate the task of enforcing this correct typing to the Polynomial Commitment Scheme (PCS) or to the IOPP. See remark 2.4.2 for more details, and the technical overview of Zinc [**Zinc**] for a thorough discussion of this matter.

Step 2: Ideal check via random MLE evaluation. The projected constraint from Step 1 requires $\phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \in \mathfrak{I}'$ for all $\mathbf{b} \in \{0,1\}^\mu$ over $\mathbb{F}_q[X]$. Let $\bar{v}(K)$ denote the multilinear polynomial on μ variables such that $\bar{v}(\mathbf{b}) = \phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0))$ for all $\mathbf{b} \in \{0,1\}^\mu$. The verifier samples a random point $\mathbf{r} \leftarrow S^\mu$ for a large subset $S \subseteq \mathbb{F}_q$, and the prover sends

$$e = \bar{v}(\mathbf{r}) = \sum_{\mathbf{b} \in \{0,1\}^\mu} \phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}; \mathbf{r}) \quad \in \mathbb{F}_q[X].$$

The verifier checks that $e \in \mathfrak{I}'$. Then the original n-row constraint (2.11) is replaced by the constraint

$$\phi_q(\Sigma^{\mathbf{r},e})(\phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) = 0, \qquad \text{in } \mathbb{F}_q[X], \tag{2.12}$$

where $\Sigma^{\mathbf{r},e}(\mathbf{Y}, \mathbf{Z})$ is defined as

$$\phi_q(\Sigma^{\mathbf{r},e})(\mathbf{Y}, \mathbf{Z}) := \sum_{\mathbf{b} \in \{0,1\}^\mu} \phi_q(Q)(\mathbf{b}, \mathbf{Y}, \mathbf{Z}) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}; \mathbf{r}) - e \quad \in (\mathbb{F}_q[X])[\mathbf{Y}, \mathbf{Z}],$$

The prover and verifier will now focus on proving the strict equality (2.12) over $\mathbb{F}_q[X]$.

Proof sketch of soundness for Step 2. The following lemma justifies that if $e \in \mathfrak{I}'$, then with high probability over the choice of $\mathbf{r}$, the original ideal membership constraints hold as well.

Lemma 2.2.5 (Testing ideal membership via MLE evaluation; informal version of lemma 2.5.1)**.** *Let $\mathbb{K}$ be a field, let $\mathfrak{I} \subseteq \mathbb{K}[X]$ be an ideal such that $\mathfrak{I} \cap \mathbb{K} = (0)$, and let $n = 2^\mu$. Let $\mathbf{v} \in \mathbb{K}[X]^n$ be a vector and $\tilde{\mathbf{v}}$ its multilinear extension. If not all entries of $\mathbf{v}$ belong to $\mathfrak{I}$, then for any finite $S \subseteq \mathbb{K}$ and uniformly random $\mathbf{r} \leftarrow S^\mu$, $\Pr[\tilde{\mathbf{v}}(\mathbf{r}) \in \mathfrak{I}] \leq \frac{\mu}{|S|}$.*

The result follows by projecting onto the quotient ring $\mathbb{K}[X]/\mathfrak{I}$ and applying the Schwartz–Zippel lemma for exceptional sets (lemma 2.3.1); a full proof is given in lemma 2.5.1. Note that here we have $\mathfrak{I}' \cap \mathbb{F}_q = (0)$ because $\mathfrak{I} \neq \mathbb{Q}[X]$ and is generated by a monic polynomial. □

Thus ideal-membership predicates can be handled analogously to strict equalities in standard proof systems: just as one checks $\tilde{\mathbf{u}}(\mathbf{r}) = 0$ (i.e. membership in the zero ideal) to test whether all entries of a vector $\mathbf{u}$ are zero, one checks $\tilde{\mathbf{u}}(\mathbf{r}) \in \mathfrak{I}'$ to test whether all entries of $\mathbf{u}$ belong to $\mathfrak{I}$.

Step 3: Evaluation projection from $\mathbb{F}_q[X]$ to $\mathbb{F}_q$. The verifier samples a random element $a \leftarrow \mathbb{F}_q$ and both parties apply the evaluation map ψ_a. Keeping in mind that ψ_a is a ring homomorphism, the claim $\phi_q(\Sigma^{\mathbf{r},e})(\phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) = 0$ in $\mathbb{F}_q[X]$ is replaced by

$$\psi_{q,a}(\Sigma^{\mathbf{r},e})(\psi_{q,a}(\mathbf{y}),\, \psi_{q,a}(\mathbf{f}_0)) = 0 \qquad \text{in } \mathbb{F}_q, \tag{2.13}$$

where we used $\psi_a \circ \phi_q = \psi_{q,a}$.

Proof sketch of soundness for Step 3. The field element $\psi_{q,a}(\Sigma^{\mathbf{r},e})(\psi_{q,a}(\mathbf{y}),\, \psi_{q,a}(\mathbf{f}_0))$ is the result of evaluating the polynomial $u(X) = \phi_q(\Sigma^{\mathbf{r},e})(\mathbf{y}, \mathbf{f}_0) \in \mathbb{F}_q[X]$ at $X = a$. If u is nonzero, then the probability that (2.13) holds is at most $\deg u/|\mathbb{F}_q|$. This is negligible as long as d_0 and $\deg Q$ are much smaller than $|\mathbb{F}_q|$. □

Combining the three steps. The following diagram summarizes the three steps of the reduction from $\mathbb{Q}[X]$-constraints to algebraic constraints over $\mathbb{F}_q$:

$$\mathbb{Q}[X] \xrightarrow[\text{(Step 1)}]{\phi_q} \mathbb{F}_q[X] \xrightarrow[\text{(Step 2: ideal check)}]{\text{MLE eval at } \mathbf{r}} \mathbb{F}_q[X] \xrightarrow[\text{(Step 3)}]{\psi_a} \mathbb{F}_q.$$

Since (2.13) is a standard algebraic claim over a finite field, the prover and verifier can now invoke any PIOP for algebraic relations over $\mathbb{F}_q$ to prove it. In particular, the verifier can query the oracle $[[\psi_{q,a}(\widetilde{\mathbf{f}}_0)]]$ at any point $\mathbf{r} \in \mathbb{F}_q^\mu$ by querying $[[\widetilde{\mathbf{f}}_0]]$ at $\hat{\mathbf{r}}$ and then applying $\psi_{q,a}$ to the result, where $\hat{\mathbf{r}}$ is a preimage of $\mathbf{r}$ under the map induced by $\psi_{q,a}$.

The overall protocol is summarized informally in Figure 2.1; the formal version is given in Protocol 2.2 in section 2.5.

Remark 2.2.6 (Adaptation to $\mathbb{F}_{q_i}[X]$-constraints)**.** The full Zinc+ compiler (Protocol 2.2 in section 2.5) also handles $\mathbb{F}_{q_i}[X]$-constraints. The treatment is analogous: Step (1) is skipped; the ideal check (Step 2) works identically (since lemma 2.2.5 applies over any field), and the evaluation projection (Step 3) uses a map $\psi_{q_i,a_i} : \mathbb{F}_{q_i}[X] \to \mathbb{F}_{\tilde{q}_i}$ that evaluates at a random element a_i of a sufficiently large extension field $\mathbb{F}_{\tilde{q}_i}$ of $\mathbb{F}_{q_i}$. Choosing the extension degree ℓ_i large enough (so that $|\mathbb{F}_{\tilde{q}_i}| \geq \Omega(2^\lambda)$) ensures negligible soundness error for each $\mathbb{F}_{q_i}[X]$-part.

The $\mathbb{F}_{q_i}[X]$-constraints pose some technical hurdles of their own. For instance, when q_i is small (say $q_i = 2$), the extension $\mathbb{F}_{\tilde{q}_i}$ must be large, and one must analyse the event that $a_i \in \mathbb{F}_{\tilde{q}_i}$ has small extension degree over $\mathbb{F}_{q_i}$ (e.g. a_i happens to belong to the base field $\mathbb{F}_{q_i}$), which prevents the image of the projection ψ_{a_i} from covering all of $\mathbb{F}_{\tilde{q}_i}$ and can cause a soundness loss. Fortunately, this bad event can be shown to occur with negligible probability.

The $\mathbb{Q}[X]$-part and the $\mathbb{F}_{q_i}[X]$-parts are compiled independently: each part is projected to its own finite field ($\mathbb{F}_q$ for the $\mathbb{Q}[X]$-part, $\mathbb{F}_{\tilde{q}_i}$ for each $\mathbb{F}_{q_i}[X]$-part), and independent finite-field PIOPs are invoked for each. The witness vector $\mathbf{f}_0$, shared across all parts, is handled by answering the verifier's finite-field oracle queries via queries to the original $\mathbb{Q}[X]$-oracle $[[\widetilde{\mathbf{f}}_0]]$ composed with the appropriate projection. The knowledge error from each part is bounded separately, and the overall error is obtained by a union bound.

Figure 2.1: Zinc^+-PIOP (simplified, $\mathbb{Q}[X]$-part only): a PIOP for $\mathsf{REL}_{\mathsf{UCS}}$ from PIOPs over finite fields (informal).

Input: P and V receive $(\mathbb{i}, \mathbb{x}; \mathbb{w}) \in \mathsf{REL}_{\mathsf{UCS}}$ and $(\mathbb{i}, \mathbb{x})$, respectively, with $\mathbb{w} = (\mathbf{f}_0)$, $\mathbf{f}_0 \in (\mathbb{Q}^{<d,B}[X])^m$.

1: P sends the oracle $[[\tilde{\mathbf{f}}_0]]$ to the multilinear extension of $\mathbf{f}_0$.

2: **(Step 1: Prime projection.)** V samples a random $\Omega(\lambda)$-bit prime q. Both parties replace each constraint $Q(\mathbf{b}, \mathbf{y}, \mathbf{f}_0) \in \mathfrak{I}$ $(\forall \mathbf{b} \in \{0,1\}^\mu)$ (cf. (2.10)) in $\mathbb{Q}[X]$, by the constraint

$$\phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \in \mathfrak{I}' \qquad \forall \mathbf{b} \in \{0,1\}^\mu$$

in $\mathbb{F}_q[X]$ (cf. (2.11)). If some application of ϕ_q is not well defined, the protocol aborts.

3: **(Step 2: Randomized ideal check.)** V samples $\mathbf{r} \leftarrow (\mathbb{F}_q)^\mu$. For each constraint $(Q, \mathfrak{I})$, P computes and sends $e \in \mathbb{F}_q[X]$, which is supposedly the evaluation at $\mathbf{r}$ of the multilinear extension (MLE) of the vector $(\phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)))_{\mathbf{b} \in \{0,1\}^\mu}$ in $\mathbb{F}_q[X]$, i.e. supposedly

$$0 = \phi_q(\Sigma^{\mathbf{r},e})(\phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) = \sum_{\mathbf{b} \in \{0,1\}^\mu} \phi_q(Q)(\mathbf{b}, \phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}; \mathbf{r}) - e \qquad \text{in } \mathbb{F}_q[X].$$

V checks $e \in \mathfrak{I}'$ (over $\mathbb{F}_q[X]$). The new claim is (2.12), i.e. $\phi_q(\Sigma^{\mathbf{r},e})(\phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) = 0$ in $\mathbb{F}_q[X]$.

4: **(Step 3: Evaluation projection.)** V samples $a \leftarrow \mathbb{F}_q$ and both parties evaluate $\phi_q(\Sigma^{\mathbf{r},e})(\phi_q(\mathbf{y}), \phi_q(\mathbf{f}_0)) \in \mathbb{F}_q[X]$ at $X = a$, obtaining the finite-field claim (2.13), i.e.

$$\psi_{q,a}(\Sigma^{\mathbf{r},e})(\psi_{q,a}(\mathbf{y}),\, \psi_{q,a}(\mathbf{f}_0)) = 0 \qquad \text{in } \mathbb{F}_q. \tag{2.14}$$

5: **(Finite-field PIOP.)** P and V execute a PIOP over $\mathbb{F}_q$ to prove the above claim. Oracle queries to the witness are answered by querying $[[\tilde{\mathbf{f}}_0]]$ at an appropriate lifted point, and applying $\psi_{q,a} = \psi_a \circ \phi_q$ to the result. V accepts if and only if the finite-field PIOP accepts.

2.2.3 Efficiency and experimental results

Experimental results

In this section we report on a PoC partial implementation of our proof system. Concretely, we provide micro-benchmarks for: 1) Commit, open, and verify times for Zip+ compiled with Merkle tree commitments (i.e. our Polynomial Commitment Scheme, roughly speaking), and 2) Essentially all steps of the Zinc+ PIOP compiler (Figure 2.1). The last Step 4 of Figure 2.1 is only partially implemented and thus we report partial results for it.

Remark 2.2.7. When benchmarking Zinc+ against other schemes, it is important to compare on the basis of a concrete target computation (e.g. SHA-256 compression plus ECDSA verification) rather

than raw trace size alone. This is because **Zinc+ deliberately trades some performance (per fixed trace size) for the ability to express computations with significantly smaller traces than other schemes**. Consequently, a naïve comparison at equal trace sizes would understate Zinc+'s advantage. For this reason, we arithmetize SHA-256 and ECDSA and use the resulting trace sizes as the basis for all our benchmarks (see below for more details on what traces are).

We emphasize that while our experiments revolve around SHA-256 and ECDSA, we expect our framework to be useful and competitive for a wide variety of applications, particularly those listed in table 2.2.

UAIR+ constraints As done in section 2.2.1 with the relation REL_f, we arithmetize our computations as a specific type of UCS constraint, which we call *Universal AIR constraint with lookups (UAIR+)*. Specifically, witness vectors are organized into traces (i.e. matrices), and constraints apply consecutively on contiguous rows of the trace (sometimes the separation between constrained rows can be larger, cf. **??**). Additionally, trace columns admit types, which are enforced via lookup constraints, e.g. a column may be constrained to have all its entries in, say, $\{0,1\}^{<32}[X]$, or in $[0\,..\,2^{32}-1]$, etc. UAIR+ constraints generalize standard AIR constraints [**ethstark**]. We refer to **??** for a full description and discussion of the UAIR+ relation.

Remark 2.2.8 (Indicative benchmarks)**.** The benchmark in table 2.7 is intended as a rough estimate of the end-to-end performance achievable with our techniques, not as a direct comparison with fully deployed systems. Three caveats apply: first, our experiments do not cover the full proving and verification pipeline, although they include Zip+, which is expected to dominate the overall cost. Second, our implementation is an initial proof of concept and has not been optimized; third, due to an incomplete implementation, we make benchmark choices that are detrimental to the overall performance, as discussed in remark 2.2.9.

We expect the benchmarks to improve in completeness and performance as we further develop our scheme in the coming months.

Technical specifications All reported benchmarks are run on a MacBook Air M4 (16 GB RAM, 2025), multi-threaded. All experiments target 100 bits of security.

Our IPRS codes use rate 1/4, the base IPRS field is of prime size $p = 2^{16} + 1$ (a Fermat prime), the FFT radix is 8, and the FFT depth is 1 when #Vars. is 9, and 2 when #Vars. is 10 or 11. The coefficients of each polynomial are organized in a matrix of size $2 \times 2^{\#\text{Vars}-1}$. This leads to optimal proof size in this setting. We use the Unique Decoding Radius proximity parameter . Run at this rate with a target security of 100 bits, our instantiation of Zip+ requires 147 queries. If IPRS codes were shown to have Mutual Correlated Agreement up to the Johnson bound **??**, then the number of queries would be 100 and proof sizes would decrease substantially.

table 2.4 consolidates the main parameters used throughout our experiments.

Parameter	**Value**
Target security (λ)	100 bits
Base IPRS field (p)	$2^{16} + 1$ (Fermat prime)
IPRS rate	1/4
FFT radix	8
FFT depth	1 (#Vars. = 9), 2 (#Vars. = 10, 11)
Proximity parameter	Unique Decoding Radius (UDR)
Minimum relative distance (δ)	3/4
Number of queries	147 (100 if MCA up to Johnson bound)
Matrix layout	$2 \times 2^{\#\text{Vars}-1}$
Hash function (Merkle tree)	BLAKE3
Fiat–Shamir hash	BLAKE3
Prime modulus (q_0) bit-size	$\Omega(\lambda)$ ($\geq$ 128-bit prime)

Table 2.4: Consolidated parameter table for all experiments in this section. The proximity parameter and query count affect both proof size and verifier time. The FFT depth and base field determine the IPRS norm growth.

Benchmarks for the Zip+ IOPP

tables 2.5 and 2.6 provides Zip+ IOPP benchmarks for batches of MLE polynomials of sizes corresponding to our SHA-256 arithmetization (**??**), namely 7 polynomials with coefficients in $\{0, 1\}^{<32}[X]$ (in fact, two of them are in $\mathbb{F}_2[X]$, see remark 2.2.11) and 3 polynomials with coefficients in $[0 \,..\, 2^{32}-1]$. Each row corresponds to a different vector length, which in turn corresponds to a different number of SHA-256 compressions.

table 2.6 provides further benchmarks within parenthesis for our unoptimized arithmetization of

ECDSA verification (**??**). table 2.7 aggregates the benchmarks from tables 2.5 and 2.6 to provide an estimate of the Zip+ costs for proving 8 SHA-256 compressions followed by an ECDSA verification.

Our SHA-256 compression arithmetization has 10 columns typed in $\{0,1\}^{<32}[X]$, 3 in $[0\,..\,2^{32}-1]$, and 2 in $\mathbb{F}_2[X]$, the columns length for 2^c compressions is 2^{6+c+1}, cf. table 2.10. Our (non-optimized) ECDSA verification arithmetization has 24 columns typed in $[0\,..\,2^{32}-1]$, each of length 2^{10} cf. table 2.11.

Remark 2.2.9 (Detrimental benchmark choices)**.** We briefly mention three detrimental benchmark choices that we make due to our incomplete implementation: 1) the three columns typed in $\mathbb{F}_2^w[X]$ of the SHA-256 trace (with w much smaller than 32) are typed in $\{0,1\}^{<32}[X] \subseteq \mathbb{Z}[X]$ (we expect this to be substantially more expensive for Zip+); 2) when combining SHA-256 and ECDSA, we simply add together the costs of the two separate benchmarks, missing on the amortization benefits of our batching techniques —note that one of the key features of our framework is that it allows mixing constraints expressed in different algebraic structures, so we can combine the SHA-256 and ECDSA arithmetization together with almost no cost; 3) we treat each ECDSA $\mathbb{F}_p$-typed element as 8 copies of a 32-bit integer, with no specific gain. This removes the advantage of using modular arithmetic as opposed to integers.

Note that 1) and 3) can be done without altering the correctness of the arithmetization: one can always "lift" elements from $\mathbb{F}_2[X]$ or $\mathbb{F}_p$ to $\mathbb{Z}[X]$ and $[0\,..\,p-1]$, respectively, and obtain an equivalent arithmetization.

Batched Zip$^+$ on 7 polynomials over $\{0,1\}^{<32}[X]$				
#Vars.	Prover (ms)	Verifier (ms)	Proof (KB)	#Compr. (Proof, KB)
9	3.34	1.28	211	4 (158)
10	5.23	1.50	221	8 (168)
11	9.51	2.70	240	16 (186)

Table 2.5: Zip+ IOPP benchmarks (compiled with Merkle tree commitments) for batches of MLE polynomials of sizes corresponding to the dominant part of our SHA-256 arithmetization (cf. **??**). All rows provide costs of batching 7 MLEs of vectors with entries in $\{0,1\}^{<w}[X]$. The coefficients of each polynomial are organized in a $1 \times 2^{\#\text{Vars}}$ table, which leads to optimal proof size in this setting. The "#Compr." column indicates how many SHA-256 compressions (as per our arithmetization in **??**) the committed polynomials correspond to. We make the unfavorable (to us) choices in remark 2.2.9, namely, we treat two columns typed in $\mathbb{F}_2[X]$ as $\{0,1\}^{<32}[X]$-typed columns. In parentheses next to the number of compressions we indicate the proof size if we kept said columns in their correct $\mathbb{F}_2[X]$ type.

Batched Zip$^+$ on 3 (27) polynomials over $[0\,..\,2^{32}-1]$			
#Vars.	Prover (ms)	Verifier (ms)	Proof (KB)
10	1.10 (5.48)	0.63 (0.80)	15 (43)
11	1.46 (7.03)	0.68 (0.86)	20 (62)
12	1.68 (8.93)	0.87 (0.93)	30 (87)
13	2.73 (14.59)	1.35 (1.58)	41 (125)

Table 2.6: Zip+ IOPP benchmarks (compiled with Merkle tree commitments) for batches of 3 (respectively, 27) MLE polynomials with entries in $[0\,..\,2^{32}-1]$. Our SHA-256 compression arithmetization has 3 such polynomials, with 10 variables for 8 compressions, 11 for 9 compressions, and so on. Our initial, non-optimized, ECDSA arithmetization **??** has 24 such polynomials, each of 10 variables. We use a batch size of 27 in parentheses which corresponds to the aggregate number of polynomials typed in $[0\,..\,2^{32}-1]$ between both our SHA-256 and ECDSA arithmetizations.

Batched Zip$^+$ on trace of SHA-256 ($\times 8$) + ECDSA		
Prover (ms)	Verifier (ms)	Proof (KB)
10.71	2.30	211

Table 2.7: Zip+ IOPP benchmarks (compiled with Merkle tree commitments) for a trace layout matching 8 SHA-256 compressions plus one ECDSA verification, as per our arithmetizations. All times include the cost of Merkle tree hashing. Prover and verifier times are obtained by summing the results from table 2.5 and table 2.6 (cf. remark 2.2.9). Proof sizes correspond to batched Zip+ sizes (i.e. they are not just the sum of proof sizes in tables 2.5 and 2.6). See remark 2.2.7 for further comments on these benchmarks. For context, Binius64's *end-to-end* benchmark for the same task is given in table 2.9.

Benchmarks for the Zinc+ PIOP compiler

In table 2.8 we provide benchmarks for almost all the steps of the Zinc+ PIOP prover (Figure 2.1). For convenience, we include the corresponding Zip+ prover times as well. Our benchmarks use a UAIR+ whose trace matches that of our SHA-256 + ECDSA verification trace. Namely, it has 7 columns typed in $\{0,1\}^{<32}[X]$ and 27 columns typed $[0\,..\,2^{32}-1]$, and the trace has 2^{10} rows. The constraints are described after the table. They do not correspond to the actual constraints of our SHA-256 + ECDSA arithmetization, but we believe are of similar complexity. In remark 2.2.10 we discuss the main unimplemented parts of our Zinc+ PIOP and how they may affect the benchmarks in table 2.8.

The finite field PIOPs used by our Zinc+ PIOP compiler are simple: they 1) run batched sumcheck on the projected MLE evaluation claims (2.14) plus other sumcheck claims arising in the next step; 2) run a lookup argument to enforce the projected typing constraints in the UAIR+ (remark 2.2.10), 3) use a batched sumcheck approach for proving evaluation claims for the MLE of shifted columns (cf. **????**). Our final step in table 2.8 only includes numbers for 1) —see remark 2.2.10 for further discussion.

Zip+ IOPP prover	
Zip+ prover	10.71 ms
(Partial) Zinc+ PIOP prover	
Project trace from $\mathbb{Z}[X]$ to $\mathbb{F}_q[X]$	1.75 ms
Ideal elements computation	3.39 ms
Project trace from $\mathbb{F}_q[X]$ to $\mathbb{F}_q$	0.25 ms
Batched sumcheck over $\mathbb{F}_q$	1.72 ms
Subtotal	7.11 ms
Total	**17.82 ms**

Table 2.8: Prover times for the Zip+ IOPP and the main steps of the Zinc+ PIOP compiler Figure 2.1. We use a UAIR+ matching the column profile from table 2.7, namely it has 7 columns in $\{0,1\}^{<32}[X]$ and 27 columns in $[0\,..\,2^{32}-1]$. The trace has 2^{10} rows. This trace layout corresponds to that of our unoptimized arithmetization of SHA-256 + ECDSA (with 8 compressions). The constraints are described after the table.
The steps measured in the table are the following: (1) *Zip+ prover*: the commit and open times for the Zip+ IOPP prover, as per table 2.7; (2) *Project trace from $\mathbb{Z}[X]$ to $\mathbb{F}_q[X]$ – Step 1 of Figure 2.1*: prover reduces all elements in the trace modulo q, via the map $\phi_q : \mathbb{Z}[X] \to \mathbb{F}_q[X]$. This is not a negligible step because the reduction entails computing Montgomery forms of integers (for later efficient operation on $\mathbb{F}_q$); (3) *Ideal elements computation – Step 2 of Figure 2.1*: run the randomized ideal check via MLE evaluation at $\mathbf{r}$, reducing ideal-membership constraints to a strict equality over $\mathbb{F}_q[X]$; (4) *Project to $\mathbb{F}_q$ – Step 3 of Figure 2.1*: apply evaluation projection ψ_a, evaluating the $\mathbb{F}_q[X]$ polynomials at the challenge a to obtain constraints over $\mathbb{F}_q$; (5) *Sumcheck over $\mathbb{F}_q$ – Partial Step 4 of Figure 2.1*: execute the finite-field PIOP, combining all evaluated polynomials into a single opening claim for the PCS, see remark 2.2.10.

The UAIR used in the benchmarks from table 2.8 has 7 binary-polynomial columns $b_0, \dots, b_6$ each in $\{0,1\}^{<32}[X]^{2^{10}}$ and 27 integer columns $z_0, \dots, z_{26}$, each in $[0\,..\,2^{32}-1]^{2^{10}}$, with 14 constraints applied for every row $t \in [2^{10}]$:

$$
\begin{aligned}
b_i[t] - z_i[t] &\in (X-2), & i &\in \{0,1,2\}, && \text{(3 deg-1)}\\
b_{i+3}[t] - z_i[t+1] &\in (X-2), & i &\in \{0,1,2\}, && \text{(3 deg-1)}\\
z_{3+3i}[t] + z_{4+3i}[t] - z_{5+3i}[t] &= 0, & i &\in \{0,\dots,5\}, && \text{(6 deg-1)}\\
z_{21+3i}[t] \cdot z_{22+3i}[t] - z_{23+3i}[t] &= 0, & i &\in \{0,1\},. && \text{(2 deg-2)}
\end{aligned}
$$

Here $v_i[t]$ denotes the t-th entry of a vector v_i. The element $z_i[t+1]$ is defined as zero when $t = 2^{10}$.

Remark 2.2.10 (Unimplemented parts of Zinc+ PIOP)**.** The main unimplemented parts of our Zinc+ PIOP have to do with managing MLE evaluations of shifted columns and running lookup

arguments over the finite field $\mathbb{F}_q$. The former is the main reason why the UAIR+ constraints we use for benchmarking are not exactly those of our SHA-256 + ECDSA arithmetization. These missing parts are purely based on sumchecks over $\mathbb{F}_q$, most of which can be batched with other sumchecks. Other unimplemented parts of the Zinc+ PIOP have to do with boundary constraints and selector polynomials for some constraints, cf. **??**.

Regarding the lookup arguments, we remark that these are performed purely over $\mathbb{F}_q$. In particular, any decomposition vectors required during the argument (in order to handle lookups into large tables) are vectors of $\mathbb{F}_q$ elements. To save on implementation hurdles we have the prover send the decomposition vectors in the clear: for lookup tables $\{0,1\}^{<32}[X]$ and $[0\,..\,2^{32}-1]$, and batching lookup claims, this requires sending a vectors of length 2^{10} (in our example UAIR+) with entries in $\mathbb{F}_q$, which amounts to 12 KB.

We can give a rough estimate of the cost of the missing components. The lookup and shifted-column MLE sumchecks each operate on vectors of length 2^{10} over $\mathbb{F}_q$, i.e. they are of the same size as the batched sumcheck over $\mathbb{F}_q$ already measured in table 2.8 at 1.72 ms. The lookup argument (say LogUp) requires an additional smaller batched sumcheck invocation, which can be batched with the aforementioned batched sumcheck (see remark 2.2.10 for a discussion of how to handle chunk decomposition in lookups). The shifted-column MLE evaluation adds one more batched sumcheck (this cannot be batched with the previous one). Overall, becasue all these extra batched sumchecks are on smaller polynomials (in number of monomials) than the original one, we expect them to, at most, double our current sumcheck costs.

Partial comparisons with other schemes

For context, we next provide end-to-end benchmarks for the Binius64 proof system on SHA-256 hashing and SHA-256+ECDSA [**binius**]. We emphasize that the following comparison is between our partial (IOPP-only) benchmarks and Binius64's end-to-end numbers, and is therefore not apples-to-apples; nonetheless, the gap between the two provides preliminary evidence that a fully optimized end-to-end Zinc+ implementation may outperform state-of-the-art SNARKs for these computations. Binius64 is the most competitive scheme we are aware of for the specific computation

our benchmarks target, namely SHA-256 followed by ECDSA verification. For other applications (e.g. the Hamming weight + RLWE encryption example from **??**) our framework may show a larger advantage, though again this remains to be validated with end-to-end benchmarks.

On the other hand, using our work for, say, proving Poseidon2 hashing on a small field would result in worse performance for Zinc+ than state-of-the-art competitors[2]. Indeed, Zinc+'s value proposition is to allow for significantly smaller arithmetization sizes than other schemes in non-field-based computations.

Remark 2.2.11 (Comparison caveats)**.** The reported benchmarks are both favorable to us because we do not report end-to-end results (cf. remark 2.2.10), and unfavorable to us because of unoptimized implementation and detrimental benchmark choices. Regardless, we hope the reported experiments resolve what is, in our opinion, the most pressing question regarding the performance of our framework: namely whether the smaller arithmetization sizes compensate for the cost of committing to polynomials with coefficients in $\mathbb{Z}[X]$ (see section 2.2.3 for further discussion). The experiments provide preliminary support for a positive answer: for 8×SHA-256 + ECDSA, our Zip+-only prover (the IOPP compiled with Merkle tree commitments, not end-to-end) runs in 10.71 ms with a 211 KB proof, whereas the Binius64 end-to-end prover takes 136 ms with a 291 KB proof (table 2.9). While this is not an apples-to-apples comparison since our numbers cover only the IOPP/PCS component (including Merkle tree hashing)—the over 12× gap in prover time and 27% smaller proof size leave substantial room for the remaining PIOP-related costs. For reference, the (partial) PIOP costs reported in table 2.8 add up to 7.11 ms, which is about 66% of the Zip+ prover time.

table 2.9 provides end-to-end benchmark results for Binius64 on SHA-256 and SHA-256+ECDSA. To aid comparison: the Binius64 row for 8×SHA-256 + ECDSA (prover: 136 ms, proof: 291 KB) targets the same computation as our aggregated Zip+ benchmark in table 2.7 (prover: 10.71 ms, proof: 211 KB). Similarly, the SHA-256-only rows in table 2.9 correspond to the same number of compressions as the rows in table 2.5. Note that our numbers cover only the Zip+ layer (IOPP

[2]Technically, Zinc+ can be instantiated so that it degenerates into any existing multilinear-based scheme for finite fields. Hence, technically speaking, a perfectly instantiated and implemented Zinc+ is always roughly at least as performant as the best existing multilinear-based scheme over finite fields

compiled with Merkle tree commitments), whereas Binius64's are end-to-end.

Binius64: SHA-256			
#Compr.	Prover (ms)	Verifier (ms)	Proof (KB)
4	23.1	3.5	156.0
8	25.9	3.8	166.3
16	31.5	4.4	179.7
Binius64: 8×SHA-256 + ECDSA			
8	136.0	10.79	291

Table 2.9: Binius64 *end-to-end* benchmarks. Top: SHA-256 for 4 to 16 compression rounds. Bottom: 8 SHA-256 compressions plus one ECDSA signature verification. Times do not include the witness generation step.

Summary of SHA-256 and ECDSA arithmetizations.

table 2.10 and table 2.11 summarize the trace layout and constraint structure of our SHA-256 and ECDSA arithmetizations in UAIR$^+$ form, respectively. UAIR$^+$ is an AIR-like subclass of our general UCS relation $\mathsf{REL}_{\mathsf{UCS}}$. Its description is in **??**. The main idea of UAIR$^+$ is that witness vectors are organized into tables, called *traces*, and polynomial constraints apply to pairs of consecutive rows (sometimes one allows constraints accessing non-adjacent rows).

A full arithmetization for SHA-256 can be found in **??**, and **??** sketches the arithmetization of ECDSA.

Trace layout			*Constraints*		
#Cols	Lookup set	Domain	#Mon.	Ideal	Ring
10	$\{0,1\}^{<w}[X]$	$\subseteq \mathbb{Q}^{<w}[X]$	10	(X^w-1)	$\subseteq \mathbb{F}_2[X]$
5	$[0\,..\,2^w-1]$	$\subseteq \mathbb{Q}$	6	(0)	$\subseteq \mathbb{F}_2[X]$
4	$\mathbb{F}_2^{<w}[X]$	$\subseteq \mathbb{F}_2^{<w}[X]$	27	$(X-2)$	$\subseteq \mathbb{Q}[X]$

Table 2.10: SHA-256 (1 compression) UAIR$^+$ summary (64 rows, 19 witness columns, 53 monomials total), where $w = 32$. The "#Mon." column counts the total number of monomials across constraint polynomials sharing the same ideal and ring. All constraint polynomials are linear in the witness variables (the degree may increase when multiplied by selector polynomials; see **??**). Of the 5 rational-integer columns, 3 have entries in $[0\,..\,6]$; the remaining 2 have arbitrary entries in $[0\,..\,2^w-1]$. In our proxy benchmarks, we tighten the trace to have size $2^7 \times 8$ cells in $\{0,1\}^{<32}[X]$ and $2^7 \times 3$ in $[0\,..\,2^{32}-1]$, i.e. we make it have 2x more rows and 2x less columns. This favors proof size at the expense of slightly extra PIOP work over the random projected finite field.
Due to an incomplete implementation, the $\mathbb{F}_2^{<w}[X]$ columns are treated as being typed in $\{0,1\}^{<w}[X]$ (which is a detrimental choice for us).

Trace layout			*Constraints*		
#Cols	Lookup set	Domain	#Cstr.	Ideal	Ring
3	$\{0,1\}$	$\subseteq \mathbb{Q}$	9	(0)	$\subseteq \mathbb{F}_p$
9	$\mathbb{F}_p$	$\subseteq \mathbb{F}_p$	2	(0)	$\subseteq \mathbb{F}_n$
2	$\mathbb{F}_n$	$\subseteq \mathbb{F}_n$			

Table 2.11: ECDSA verification UAIR$^+$ summary (258 rows, 14 witness columns). The "#Cstr." column counts the number of non-boundary constraint polynomials per (ideal, ring) group. The maximum constraint degree is 6. Of the 3 rational-integer columns, all belong to $\{0,1\}$.
For our benchmarks, we make the non-optimal choice of representing all columns from $\mathbb{F}_p$ and $\mathbb{F}_n$ as 8 integer columns over $[0\,..\,2^w{-}1]$, where $w = 32$. In our proxy benchmarks, we ignore the 3 columns with entries in $\{0,1\}$.
As with the SHA-256 trace, and due to the same reasons (table 2.10) we tighten the ECDSA trace to have size $2^{10} \times 24$ rather than $2^8 \times 88$.
We believe this arithmetization can be greatly optimized, and we leave this to future work.

Efficiency and opportunities of our framework

The techniques used in Zinc+ in the context of SNARKs are relatively different from standard techniques that revolve around finite field arithmetic and polynomials with coefficients in said fields. We consider the problem of understanding how our framework compares to standard techniques an open problem. We provide an informal discussion of our observations around this question.

In terms of efficiency, we view Zinc+ as a framework that bootstraps the vast majority of SNARK costs to the single task of committing to multilinear polynomials with coefficients in $\mathbb{Q}[X]$, $\mathbb{Z}[X]$, $\mathbb{F}_q[X]$, etc.—in particular, for the applications we discussed, to polynomials with coefficients in $\{0,1\}^{<w}[X]$. Indeed, in many ways, working in $\{0,1\}^{<w}[X]$ creates w copies of a problem to be solved. For example, our commitments to vectors with entries in $\{0,1\}^{<w}[X]$ are effectively w parallel commitments to binary vectors (optimizations apply, see below). But on the other hand, the random projection technique onto finite fields compresses these w copies into a single one, by introducing a small soundness error. Putting together this compression saving and the smaller initial arithmetizations that our UCS constraints produce, we observe in our implementation that the costs once in the finite field are tiny compared to the commitment costs and the projection costs.

The fundamental question is then: *does the tradeoff of smaller arithmetizations, plus the compression savings of projection, outweigh the cost of committing to these polynomials (and also*

the projection costs)? We believe that our preliminary benchmarks answer the question positively.

We remark that committing to polynomials with coefficients in $\{0,1\}^{<w}[X]$, while seemingly a "w-wise" task as already mentioned, can be implemented so that it does not scale linearly in w. Indeed, we observe engineering opportunities here by, for example, representing elements of $\{0,1\}^{<w}[X]$ as `u32` types and using SIMD optimizations to operate on those. The fact that the Zinc+ framework concentrates all the computational burden on these commitments allows one to focus on optimizing a single type of operation.

Zip+, proof sizes, and future work Zip+ is a Ligero/Brakedown-like IOPP (in modern instantiation) for $\mathbb{Q}[X]$ and $\mathbb{F}_q[X]$-valued multilinear polynomials. Ligero/Brakedown on finite fields are known for having large proof sizes. And indeed our benchmarks show significant proof sizes relative to prover and verifier time (which on the other hand are very small). Besides being based on large-proof IOPPs, Zip+ experiments show a further proof size increase due to the fact that encodings of vectors in $\{0,1\}^{<w}[X]$ scale linearly on w in bit-size, being, from a purely size perspective, the same as w commitments to binary vectors. Note that these commitments are over $\mathbb{Z}[X]$ and not over $\mathbb{F}_2[X]$, hence there is no modular reduction that would help keep encoding sizes small.

Ongoing work of ours is focused on designing a WHIR, Ligerito, Blaze, FICS/FACS, etc. [**whir**; **Ligerito**; **Blaze**; **FICS-FACS**] –IOPPs known for their small proofs among others– variant for our framework.

2.3 Preliminaries

2.3.1 Basic notions

For $k \geq 1$, let $[k] := \{1, \dots, k\}$, and let $[0\,..\,k]$ be $\{0\} \cup [k]$. We write $x \leftarrow S$ for uniform sampling from a finite set S, and $x \leftarrow \mathcal{D}$ for sampling from a distribution $\mathcal{D}$.

We use $\mathbb{K}$ to denote (possibly infinite) fields and R to denote rings. We write $\mathbb{F}_q$ for the finite field with q elements (where q is a prime or a prime power), $\mathbb{Q}$ for the field of rational numbers, and

$\mathbb{Z}$ for the ring of integers. We use standard terminology and notation: an ideal I generated by a set $S \subseteq R$ is denoted (S), the quotient ring is denoted R/I. Sometimes we write $(S)_R$ to emphasize that the ideal is an ideal of the ring R. A ring is an integral domain if the product of any two nonzero elements is nonzero.

Polynomial rings and commutative algebra. For variables $\mathbf{Y} = (Y_1, \dots, Y_\mu)$, $R[\mathbf{Y}]$ denotes multivariate polynomials over R, and $R^{\mathsf{ml}}[\mathbf{Y}]$ the multilinear polynomials (individual degree ≤ 1 in each variable).

We often work with multivariate polynomials whose coefficients are themselves univariate polynomials. For a ring $\mathcal{R}$ (typically $\mathbb{Z}[X]$, $\mathbb{Q}[X]$, or $\mathbb{Z}_{(q)}[X]$) and variables $\mathbf{Y} = (Y_1, \dots, Y_\mu)$, we write $\mathcal{R}[\mathbf{Y}]$ for polynomials in $\mathbf{Y}$ with coefficients in $\mathcal{R}$. We view elements of $(\mathbb{Z}[X])[\mathbf{Y}]$ as polynomials in $\mathbf{Y}$ (not as elements of $\mathbb{Z}[X, \mathbf{Y}]$), denoting them by $f(\mathbf{Y})$. For polynomial rings, $\mathcal{R}^{<d,<B}[X]$ denotes polynomials of degree $< d$ with coefficients in $\mathcal{R}^{<B}$; we may write $\mathcal{R}^{<d}[X]$ when the bit bound is clear. Ring homomorphisms $f : \mathcal{R}_1 \to \mathcal{R}_2$ extend coefficient-wise to $\mathcal{R}_1[\mathbf{Y}] \to \mathcal{R}_2[\mathbf{Y}]$. More generally, all maps are extended entrywise to vectors and matrices.

For any field $\mathbb{F}_{p^e}$ containing subfield L we can map elements of L into $\mathbb{F}_{p^e}$ via the canonical embedding $\iota_{p^e} : L \hookrightarrow \mathbb{F}_{p^e}$. We extend this embedding to the ring $\mathbb{F}_{p^e}[X]$ its subrings $L[X]$ by applying ι_{p^e} coefficient-wise. If $(\mathsf{j}) = \mathcal{J} \subseteq \mathbb{F}_q[X]$ is an ideal, we note $(\iota_{q^\ell}(\mathsf{j})) = \mathcal{J}' \subseteq \mathbb{F}_{q^\ell}[X]$ is also.

The subset $\mathbb{Z}_{(p)} = \{a/b \in \mathbb{Q} : p \nmid b\}$ of $\mathbb{Q}$ is a ring called the *localization of* $\mathbb{Z}$ *at the prime* p. This ring admits a surjective homomorphism $\phi_p : \mathbb{Z}_{(p)} \to \mathbb{F}_p$, $a/b \mapsto a \cdot b^{-1}$, with kernel $\ker(\phi_p) = p\mathbb{Z}_{(p)}$.

For prime powers $q = p_i^{e_i}$, we set $\mathbb{Z}_{(q_i)} := \mathbb{Z}_{(p_i)}$, and extend ϕ by defining $\phi_{q_i} = \iota_{q_i} \circ \phi_{p_i}$. The *intersection localization ring* is

$$\mathbb{Z}_{(q_1 \cdots q_n)} \;:=\; \bigcap_{i \in [n]} \mathbb{Z}_{(p_i)} \;=\; \Big\{ a/b \in \mathbb{Q} \;\Big|\; \gcd\big(b, \textstyle\prod_i p_i\big) = 1 \Big\}. \tag{2.15}$$

Bitstring representation. We use standard representations: integers are represented in two's complement, so an integer $n \in \mathbb{Z}$ has bit-size $\lceil \log_2(|n|+1) \rceil + 1$; rationals $a/b \in \mathbb{Q}$ are represented as pairs (a, b) in *lowest terms* (i.e., $\gcd(a, b) = 1$ and $b > 0$), with bit-size the sum of the bit-sizes of a and b; polynomials are represented as tuples of coefficients (in increasing degree order), with bit-size

the sum of the coefficient bit-sizes. We write $\mathbb{Z}^{<B}$, $\mathbb{Q}^{<B}$, $\mathbb{Z}^{<d,<B}[X]$, $\mathbb{Q}^{<d,<B}[X]$, and $\mathbb{Z}_{(q)}^{<d,<B}[X]$ for elements with bit-size less than B (and degree $< d$ for polynomials). For a ring $\mathcal{R}$, we write $\mathcal{R}^{<B}$ for elements representable with fewer than B bits.

Multilinear extensions (MLE). A multilinear polynomial $f(\mathbf{Y}) \in R^{\mathsf{ml}}[\mathbf{Y}]$ is uniquely determined by its values on $\{0,1\}^\mu$. For any map $f : \{0,1\}^\mu \to R$, we write $\tilde{f}(\mathbf{Y})$ for its *multilinear extension* (MLE), the unique multilinear polynomial that agrees with f on the Boolean hypercube. For a vector $\mathbf{v} \in R^{2^\mu}$, we write $\tilde{\mathbf{v}}(\mathbf{Y})$ for the MLE of the map $\mathbf{y} \mapsto v_{\mathbf{y}}$ (interpreting $\mathbf{y} \in \{0,1\}^\mu$ as an integer in binary). We also use the standard *equality polynomial*

$$\widetilde{\mathsf{eq}}(\mathbf{y};\mathbf{Y}) := \prod_{i\in[\mu]} \left(y_i Y_i + (1-y_i)(1-Y_i)\right), \tag{2.16}$$

which satisfies $\widetilde{\mathsf{eq}}(\mathbf{y};\mathbf{z}) = 1$ if $\mathbf{y} = \mathbf{z}$ and $\widetilde{\mathsf{eq}}(\mathbf{y};\mathbf{z}) = 0$ otherwise for $\mathbf{y}, \mathbf{z} \in \{0,1\}^\mu$.

Schwartz–Zippel lemma for exceptional sets. We will need the following general form of Schwartz-Zippel lemma in order to address certain ideal-membership constraints in our proof system. A subset $S \subseteq \mathcal{R}$ is *exceptional* if $a - b$ has a multiplicative inverse for all distinct $a, b \in S$.

Lemma 2.3.1 (Schwartz–Zippel lemma for exceptional sets)**.** *Let $S \subseteq \mathcal{R}$ be a finite exceptional set and $f(\mathbf{Y}) \in \mathcal{R}[\mathbf{Y}]$ a nonzero multilinear polynomial in μ variables. Then* $\Pr_{\mathbf{r}\leftarrow S^\mu}[f(\mathbf{r}) = 0] \le \mu/|S|$.

2.3.2 Indexed relations and oracles.

An *indexed relation* REL is a set of index-input-witness triples $(\mathbb{i}, \mathbb{x}; \mathbb{w})$. The *language* associated with REL is $\mathsf{LANG}(\mathsf{REL}) := \{(\mathbb{i}, \mathbb{x}) \mid \exists\, \mathbb{w} \text{ such that } (\mathbb{i}, \mathbb{x}; \mathbb{w}) \in \mathsf{REL}\}$.

An *oracle* to a string or vector π of elements from a ring R is a black-box that allows querying any position j and obtaining π_j. We denote it by $[[\pi]]$. An *oracle to a polynomial* $f(\mathbf{X})$ over R is a black-box that allows querying any point $\mathbf{r}$ in the domain of f and obtaining $f(\mathbf{r})$. We denote it by $[[f]]$.

Remark 2.3.2 (Oracle typing assumption)**.** Soundness and knowledge soundness for PIOPs are required to hold only for malicious provers P^* that send *correctly typed* oracles—i.e., oracles that

actually correspond to polynomials with the prescribed number of variables, degrees, and coefficient domains. Informally speaking, enforcing that a (computationally bounded) prover sends such oracles is the responsibility of the polynomial commitment scheme (PCS) or other cryptographic mechanisms (such as IOPP) used to compile the PIOP into a concrete argument system.

2.3.3 Polynomial Interactive Oracle Reductions (PIORs)

Polynomial Interactive Oracle Reductions (PIORs) generalize PIOPs. A *public-coin polynomial IOR* from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$ is a pair $\Pi = (\mathsf{P}, \mathsf{V})$ specified as follows.

The prover P is an interactive algorithm that receives as input $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$. The verifier V is an interactive polynomial oracle algorithm begins with inputs $(\mathbb{i}, \mathbb{x})$, which may include both polynomial oracles and explicit values. They interact for k rounds. In round $i \in [k]$, the prover sends a polynomial oracle π_i and the verifier sends a uniformly random message ρ_i. After k rounds of interaction, the verifier makes evaluation queries to the polynomial oracles in its inputs and sent in prover messages $\pi_1, \dots, \pi_k$ as oracles. Then:

- V either rejects or outputs a new index-instance $(\mathbb{i}', \mathbb{x}')$, and
- P outputs a new witness $\mathbb{w}'$ such that $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \mathcal{R}'$.

Without loss of generality, we may view V's output as a deterministic function of its input and the interaction transcript. Note that oracles in $\mathbb{x}'$ are specified implicitly in terms of oracles given in $\mathbb{x}$ and $\pi_1, \dots, \pi_k$.

Definition 2.3.3 (Completeness)**.** We say that Π has *completeness* with error ϵ if for every $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$, letting the interaction proceed honestly and writing $(\mathbb{i}', \mathbb{x}') \leftarrow \mathsf{V}(\mathbb{i}, \mathbb{x}, \pi_1, \rho_1, \dots, \pi_k, \rho_k)$ and $(\pi_k, \mathbb{w}') \leftarrow \mathsf{P}(\mathbb{i}, \mathbb{x}, \mathbb{w}, \rho_1, \dots, \rho_{k-1})$, we have

$$\Pr_{\rho_1, \dots, \rho_k} \left[(\mathbb{i}', \mathbb{x}', \mathbb{w}') \notin \mathcal{R}' \right] \leq \epsilon(\mathbb{i}, \mathbb{x}).$$

We omit a standard soundness definition as we will prove the stronger notion of round-by-round knowledge soundness.

We will use the definition of Round-by-Round knowledge soundness from [**BCFW25**].

Definition 2.3.4 (Knowledge state function)**.** Let Π be an IOR from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$. A *knowledge state function* for Π is a (possibly inefficient) function KState, that takes as input a statement $(\mathbb{i}, \mathbb{x}, \mathbb{y})$, an interaction transcript tr, a knowledge witness w, and outputs a bit. It must satisfy:

- **Empty transcript.** If $\mathsf{tr} = \emptyset$, then $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}, \mathsf{w}) = 1$ if and only if $(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{w}) \in \mathcal{R}$.
- **Prover moves.** If tr is a transcript where the prover is about to move and $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}, \mathsf{w}) = 0$, then for every prover message π,

$$\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}\|\pi, \mathsf{w}) = 0.$$

- **Full transcript.** If $\mathsf{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}, \mathsf{w}) = 1$ if and only if $\mathsf{V}^{\mathbb{y}, \pi_1, \dots, \pi_k}(\mathbb{i}, \mathbb{x}, \rho_1, \dots, \rho_k)$ outputs $(\mathbb{x}', \mathbb{y}')$ such that $(\mathbb{i}, \mathbb{x}', \mathbb{y}', \mathsf{w}) \in \mathcal{R}'$.

Definition 2.3.5 (Round-by-round knowledge soundness)**.** An IOR Π from $\mathcal{R}$ to $\mathcal{R}'$ has *round-by-round knowledge soundness* with errors $(\kappa_1, \dots, \kappa_k)$ if there exists a knowledge state function KState for Π and a deterministic extractor $\mathcal{E}$ such that for every statement $(\mathbb{i}, \mathbb{x}, \mathbb{y})$, every round index $i \in [k]$, and every interaction transcript where the verifier is about to move it holds:

$$\Pr_{\rho_i}\left[\begin{array}{l} \exists \mathsf{w} = 0 \text{ s.t.} \\ \wedge\ \mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}, \mathcal{E}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}\|\rho_i, \mathsf{w})) = 1 \\ \wedge\ \mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathbb{y}, \mathsf{tr}\|\rho_i, \mathsf{w}) = 1 \end{array}\right] \leq \kappa_i(\mathbb{i}, \mathbb{x}, \mathbb{y}).$$

At times we will also use a prior, stronger notion of round-by-round knowledge soundness, which is known to imply the round-by-round knowledge soundness according to definition 2.3.5 with the same errors [**BCFW25**].

Definition 2.3.6 (Knowledge state function)**.** Let Π be an IOR from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$. A *knowledge state function* for Π is a (possibly inefficient) function KState, that takes as input a statement $(\mathbb{i}, \mathbb{x})$ and an interaction transcript tr, and outputs a bit. It must satisfy:

- **Empty transcript.** If $\mathsf{tr} = \emptyset$, then $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 0$.

- **Prover moves.** If tr is a transcript where the prover is about to move and $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 0$, then for every prover message π,

$$\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr} \| \pi) = 0.$$

- **Full transcript.** If $\mathsf{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 1$ if and only if $\mathsf{V}(\mathbb{i}, \mathbb{x}, \pi_1, \rho_1, \dots, \pi_k, \rho_k)$ outputs $(\mathbb{i}', \mathbb{x}')$ such that there exists $\mathbb{w}$' with $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \mathcal{R}'$.

Definition 2.3.7 (Round-by-round knowledge soundness)**.** An IOR Π from $\mathcal{R}$ to $\mathcal{R}'$ has *round-by-round knowledge soundness* with errors $(\kappa_1, \dots, \kappa_k)$ if there exists a knowledge state function KState for Π and a deterministic extractor $\mathcal{E}$ such that for every statement $(\mathbb{i}, \mathbb{x})$, every round index $i \in [k]$, and every interaction transcript where the verifier is about to move it holds:

$$\Pr_{\rho_i}\left[\begin{array}{l} \mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 0 \\ \wedge\ \mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr} \| \rho_i) = 1 \\ \wedge\ (\mathbb{i}, \mathbb{x}, \mathcal{E}(\mathbb{i}, \mathbb{x}, \mathsf{tr})) \notin \mathcal{R} \end{array}\right] \leq \kappa_i(\mathbb{i}, \mathbb{x}).$$

2.4 Universal Constraint Systems (UCS)

We start this section by presenting the main relation we are interested in proving, which we call the *universal constraint system* (UCS) relation. After that, we discuss some structural semantics of UCS, namely how one can write lookup and permutation constraints within a UCS.

2.4.1 General UCS definition

In this section we define the *universal constraint system* (UCS) relation $\mathsf{REL}_{\mathsf{UCS}}$. This relation combines algebraic constraints over $\mathbb{Q}[X]$ with algebraic constraints over finite-field polynomial rings $\mathbb{F}_{q_i}[X]$ (or, simply, finite fields $\mathbb{F}_{q_i}$).

Definition 2.4.1 (Universal constraint system (UCS))**.** The relation $\mathsf{REL}_{\mathsf{UCS}}$ is the set of all triples $(\mathbb{i}, \mathbb{x}; \mathbb{w})$ of the following form.

Index. An index is a tuple

$$\mathbb{i} = (m, \ell, n, \mathbf{q}, B, \mathbf{d}, \mathcal{C}),$$

where $m \geq 0$ is the *witness length*, $\ell \geq 0$ is the *input length*, $n \geq 0$ is the *number of rows*, $\mathbf{q} = (q_1, \dots, q_{|\mathbf{q}|})$ is a tuple of prime powers, $B \geq 0$ is a *bit-size bound*, $\mathbf{d} = (d_0, \dots, d_{|\mathbf{q}|})$ is a tuple of *degree bounds*, and $\mathcal{C}$ is a family of *constraints* (defined below). Let K be a formal variable (interpreted as ranging over rows in $[n]$), let $\mathbf{Y}$ be a tuple of ℓ formal variables (interpreted as placeholders for public input entries), and let $\mathbf{Z}_1, \mathbf{Z}_2$ be tuples of m formal variables (interpreted as placeholders for witness vectors). $\mathbf{Z}_1$ serves as placeholder for a witness with entries in $\mathbb{Q}^{<d,B}[X]$ and $\mathbf{Z}_2$ for witnesses with entries in $\mathbb{F}_{q_i}^{<d_i}[X]$. Additionally, in the finite-field constraints, $\mathbf{Z}_1$ serves as well as a placeholder for the projected integer witness $\phi_{q_i}(\mathbf{f}_0)$.

The constraint family $\mathcal{C}$ is a tuple

$$\mathcal{C} = \Big((Q_{0t}, \mathfrak{I}_{0t})_{t \in [|\mathcal{C}|]},\ (Q_{it}, \mathfrak{I}_{it})_{i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]} \Big),$$

where, for every $t \in [|\mathcal{C}|]$ and every $i \in [|\mathbf{q}|]$,

$$Q_{0t} \in (\mathbb{Q}^{<d_0,B}[X])[K, \mathbf{Y}, \mathbf{Z}_1], \qquad Q_{it} \in (\mathbb{F}_{q_i}[X])[K, \mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2],$$

and $\mathfrak{I}_{0t} = \langle \mathsf{j}_{0t} \rangle$ and $\mathfrak{I}_{it} = \langle \mathsf{j}_{it} \rangle$ are ideals given by generators

$$\mathsf{j}_{0t} \in \mathbb{Q}^{<d_0,B}[X], \qquad \mathsf{j}_{it} \in \mathbb{F}_{q_i}[X].$$

We call $(Q_{it}, \mathfrak{I}_{it})$ a $\mathbb{Q}[X]$ (or $\mathbb{F}_{q_i}[X]$) *constraint*.

Input and witness. The public input is $\mathbb{x} = \mathbf{y}$ with

$$\mathbf{y} \in (\mathbb{Z}_{(q_1 \cdots q_{|\mathbf{q}|})}^{<d_0,B}[X])^{\ell},$$

(recall that $\mathbb{Z}_{(q_1 \cdots q_{|\mathbf{q}|})}$ denotes the intersection localization ring from eq. (2.15)). The witness is $\mathbb{w} = (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|})$ with

$$\mathbf{f}_0 \in (\mathbb{Q}^{<d_0,B}[X])^m, \qquad \mathbf{f}_i \in (\mathbb{F}_{q_i}^{<d_i}[X])^m \ \text{ for all } \ i \in [|\mathbf{q}|]. \tag{2.17}$$

Constraints. We say that $(\mathbb{i}, \mathbb{x}; \mathbb{w}) \in \mathsf{REL}_{\mathsf{UCS}}$ if all of the following conditions hold.

(i) $\mathbb{Q}[X]$**-constraints.** For every $k \in [n]$ and every $t \in [|\mathcal{C}|]$,

$$Q_{0t}(k, \mathbf{y}, \mathbf{f}_0) \in \mathfrak{I}_{0t} \subseteq \mathbb{Q}[X].$$

(ii) $\mathbb{F}_{q_i}[X]$**-constraints.** For every $i \in [|\mathbf{q}|]$, every $k \in [n]$, and every $t \in [|\mathcal{C}|]$,

$$Q_{it}(k, \phi_{q_i}(\mathbf{y}), \phi_{q_i}(\mathbf{f}_0), \mathbf{f}_i) \in \mathfrak{I}_{it} \subseteq \mathbb{F}_{q_i}[X].$$

(iii) **Well-definedness of ϕ_{q_i}.** The homomorphisms ϕ_{q_i} from section 2.3.1 are defined only on the intersection localization ring $\mathbb{Z}_{(q_1 \cdots q_{|\mathbf{q}|})}$. Since the previous item applies ϕ_{q_i} to $\mathbf{f}_0$, we require the following well-definedness condition:

$$\mathbf{f}_0 \in (\mathbb{Z}^{<d_0,B}_{(q_1 \cdots q_{|\mathbf{q}|})}[X])^m. \tag{2.18}$$

When $|\mathbf{q}| = 0$, this condition is vacuous. We show in section 2.4.2 that the well-definedness condition can be expressed as a $\mathbb{Q}[X]$-constraint.

When an ideal $\mathfrak{I}_{it}$ is zero, the corresponding constraint $Q_{it}(\cdots) \in \mathfrak{I}_{it}$ reduces to the equality $Q_{it}(\cdots) = 0$. When $|\mathbf{q}| = 0$, the tuple $\mathbf{q}$ is empty and UCS reduces to a purely $\mathbb{Q}[X]$-constraint system with no $\mathbb{F}[X]$-constraints.

Remark 2.4.2 (Separation of PIOP and IOPP enforcement)**.** The satisfaction conditions in definition 2.4.1 (Items (i)–(iii)) will be enforced at the *PIOP level*, while the typing constraints for the witness (2.17) will be enforced at the *IOPP* (or *PCS*) *level*. More precisely, we design PIOPs that prove the $\mathbb{Q}[X]$-constraints, the $\mathbb{F}_{q_i}[X]$-constraints, and the well-definedness constraint, under the assumption that malicious provers send oracles of the correct type—i.e., oracles to multilinear polynomials with coefficients in $\mathbb{Q}^{<d_0,B}[X]$ or $\mathbb{F}_{q_i}^{<d_i}[X]$, respectively. Enforcing that the oracles are actually of this declared type will be the job of the IOPP (or PCS) compiler.

We remark that, in particular, the well-definedness condition (2.18) is not enforced at the IOPP level, but rather at the PIOP level (cf. section 2.4.2).

Remark 2.4.3 (The well-definedness condition)**.** We show in section 2.4.2 that the well-definedness condition (2.18) can be written as a $\mathbb{Q}[X]$-constraint. Hence, (2.18) does not require its own separate syntax, as long as one of the $\mathbb{Q}[X]$-constraints in the UCS instance enforces it.

In practice, the computations we are interested in proving already require constraining $\mathbf{f}_0$ in a way that guarantees the well-definedness condition automatically. For example, in all examples from **??**, the witness $\mathbf{f}_0$ is constrained to belong to some subset of $\mathbb{Z}^{<d}[X]^m$ (via a $\mathbb{Q}[X]$-constraint). This constraint implies the well-definedness condition for any choice of primes $q_1, \dots, q_{|\mathbf{q}|}$.

For ease of reference, we restate the definition of $\mathsf{REL}_{\mathsf{UCS}}$ below.

$$
\mathsf{REL}_{\mathsf{UCS}} = \left\{ \begin{pmatrix} \mathbb{i}, \\ \mathbb{x}; \\ \mathbb{w} \end{pmatrix} \;\middle|\; \begin{array}{l}
\textbf{Index-input-witness} \\
\quad \textbf{Index} \\
\qquad \mathbb{i} = (m, \ell, n, \mathbf{q}, B, \mathbf{d}, \mathcal{C}), \\
\qquad \mathbf{q} = (q_1, \dots, q_{|\mathbf{q}|}), \quad \mathbf{d} = (d_0, \dots, d_{|\mathbf{q}|}), \\
\qquad \mathcal{C} = ((Q_{0t}, \mathfrak{I}_{0t} = \langle \mathsf{j}_{0t} \rangle), (Q_{it}, \mathfrak{I}_{it} = \langle \mathsf{j}_{it} \rangle))_{i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]}, \\
\qquad \mathsf{j}_{0t} \in \mathbb{Q}^{<d_0, B}[X],\ \mathsf{j}_{it} \in \mathbb{F}_{q_i}[X], \\
\qquad Q_{0t} \in (\mathbb{Q}^{<d_0, B}[X])[K, \mathbf{Y}, \mathbf{Z}_1],\ Q_{it} \in (\mathbb{F}_{q_i}[X])[K, \mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2], \\
\qquad |\mathbf{Z}_1| = |\mathbf{Z}_2| = m \\
\quad \textbf{Input} \\
\qquad \mathbb{x} = (\mathbf{y}) \in (\mathbb{Z}^{<d_0, B}_{(q_1 \cdots q_{|\mathbf{q}|})}[X])^{\ell} \\
\quad \textbf{Witness} \\
\qquad \mathbb{w} = (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|}),\ \mathbf{f}_i \in (\mathbb{F}^{<d_i}_{q_i}[X])^m \quad \text{for all } i \in [|\mathbf{q}|] \\
\qquad \mathbf{f}_0 \in (\mathbb{Q}^{<d_0, B}[X])^m, \\
\\
\textbf{Constraints} \\
\quad \textbf{Algebraic constraints over } \mathbb{Q}[X] \textbf{ mod ideals} \\
\qquad Q_{0t}\,(k, \mathbf{y}, \mathbf{f}_0) \in \mathfrak{I}_{0t} \quad \text{for all } k \in [n],\ t \in [|\mathcal{C}|] \\
\\
\quad \textbf{Algebraic constraints over } \mathbb{F}_{q_i}[X] \\
\qquad Q_{it}\left(k, \phi_{q_i}(\mathbf{y}), \phi_{q_i}(\mathbf{f}_0), \mathbf{f}_i\right) \in \mathfrak{I}_{it}, \\
\qquad\qquad \text{for all } k \in [n],\ i \in [|\mathbf{q}|],\ t \in [|\mathcal{C}|]. \\
\quad \textbf{Well-definedness} \\
\qquad \mathbf{f}_0 \in (\mathbb{Z}^{<d_0, B}_{(q_1 \cdots q_{|\mathbf{q}|})}[X])^m.
\end{array} \right\}
$$

Design choices. We discuss some of the design choices behind the definition of UCS.

Why are witness and input entries typed in polynomial rings $R^{<w}[X]$ (e.g. $R = \mathbb{Z}, \mathbb{F}_q$) rather than in R (or some

One may instead try to reinterpret $R^{<w}[X]$ as w copies of R and treat each witness entry as an element of R^w. We see three advantages in preserving the ring structure of $R[X]$:

First, $R[X]$ makes ideal membership constraints natural and efficient to reason about. It is not clear how to translate ideal membership predicates into R^w without resorting to lookups, and even when lookups are available, the translation loses the algebraic structure of the ideals. Our PIOP framework leverages this algebraic structure by resolving ideal membership predicates with a single MLE randomized check, analogously to how standard proof systems check strict equalities.

Second, $R[X]$ enables multiple ideal membership predicates in the same relation. If there were only one ideal $\mathfrak{I}$, one could work over $R[X]/\mathfrak{I}$ directly. In the presence of membership predicates involving more than one ideal, staying in $R[X]/\mathfrak{I}$ would complicate the arithmetization of all but one of the predicates. Working in $R[X]$ glues together constraints that would otherwise be separated into different quotient rings. For example, in the previous example from section 2.2.1 we have ideal membership constraints simultaneously in $(X-2)$ and (X^w-1) (Example 1), and in $(X-1)$ and $(\Phi(X))$ (Example 2).

Third, our PIOP projects constraints over $\mathbb{Q}^{<d,B}[X]$ and $\mathbb{F}_q^{<w}[X]$ onto a random finite field by, respectively: 1) ($\mathbb{Q}$ case) replacing X by a random integer $a \in \mathbb{Z}$ and then reducing modulo a random prime; 2) ($\mathbb{F}_q$ case) replacing X by a random element a in $\mathbb{F}_q$ or a field extension of $\mathbb{F}_q$. When $R^{<w,B}[X]$ is large (for $R \in \{\mathbb{Q}, \mathbb{F}_q\}$) this projection provides equally large compression benefits, in that elements from $R^{<w,B}[X]$, which may be thousands of bits long (for example, in lattice-based settings), become 128–256 bits long.

Overall, it is possible to reformulate our definition of UCS as one operating over $\mathbb{Q}$ and $\mathbb{F}_q$ rather than $\mathbb{Q}^{<d_0,B}[X]$ and $\mathbb{F}_q^{<d_i}[X]$, but we think that the polynomial-ring formulation is more natural and better captures the algebraic structure of the computations we are interested in proving.

Why allow integers? Why not work entirely over $\mathbb{F}_q$ or $\mathbb{F}_q[X]$ for a sufficiently large prime q?

A plausible alternative to having constraints over $\mathbb{Z}$ or $\mathbb{Z}[X]$ is to work over $\mathbb{F}_q$ or $\mathbb{F}_q[X]$ for q larger than any value that the integer computation ever produces (let's call such a value the

computation norm), and avoid modular overflow via lookups or range checks. We see several advantages of staying over $\mathbb{Z}/\mathbb{Z}[X]$, though some use-cases may benefit from working on $\mathbb{F}_q$ or $\mathbb{F}_q[X]$.

First, by the same reasoning as the second point above, staying in $\mathbb{Z}$ or $\mathbb{Z}[X]$ allows projecting onto fields and rings $\mathbb{F}_q$ or $\mathbb{F}_q[X]$ for different values of q within the same relation. For example, our SHA-256 (+ECDSA verification) arithmetization showcases simultaneous constraints modulo 2^{32}, 2, and p for a 256-bit prime p.

Second, the third point above also applies: in applications such as verifiable FHE (e.g. the CKKS scheme) or RSA cryptography, one would require working with a very large prime q (i.e. the computation norm in such applications is large). The random-projection technique may provide significant compression benefits in these cases.

Third, when q is large, one faces the familiar overhead of field arithmetic in $\mathbb{F}_q$ or $\mathbb{F}_q[X]$ throughout the proof. For instance, Reed–Solomon encoding becomes expensive over large fields. Our Integer Pseudo Reed–Solomon (IPRS) codes operate over $\mathbb{Z}$ or $\mathbb{Q}$ and show promising practical performance (**??**). We think that specifically designed IPRS codes can be set up with good distance over $\mathbb{F}_q$ for a fixed large q, but we lack theoretical guarantees for this; we pose this as an open question in **??**.

When the computation norm q is small, say of 32 bits, then working over $\mathbb{F}_q[X]$ would be a better option. But in most applications it is not the case that 32 bits suffices to capture all integer values without overflowing the modulus. (Unless one pays arithmetization overheads by decomposing into limbs, etc. which is what we want to avoid in the first place.)

Technical observations. We make some technical observations about the definition of UCS.

Lookup constraints. In our examples section 2.2.1 and **??** we extensively rely on constraining witness elements in certain sets, e.g. $\{0,1\}^{<w}[X]$ and $[0\,..\,2^w-1]$. Our definition of UCS is expressive enough to subsume such constraints. See section 2.4.2 for an explanation of how to express lookup constraints in a UCS instance.

Rationals and Integers. In our examples, our witness vectors were either typed in $\mathbb{Z}^{<d}[X]$ or in $\mathbb{F}_q^{<d}[X]$. On the other hand, in our definition of UCS we used $\mathbb{Q}^{<d}[X]$ rather than $\mathbb{Z}^{<d}[X]$ as typing

choice. Besides the possible independent interest of typing witnesses in $\mathbb{Q}^{<d}[X]$, this choice is motivated by technical reasons related to the proof system.

The main idea is that our IOPP (or PCS) is only capable of extracting witnesses in $\mathbb{Q}^{<d}[X]$, and not in $\mathbb{Z}^{<d}[X]$. It is not clear to us how to build an IOPP or PCS that 1) guarantees typing $\mathbb{Z}^{<d}[X]$, and 2) does not incur large performance losses in doing so. Indeed, the only integer-based PCSs we are aware of [**Zartan**; **BlockDARK**] leverage hidden-order-group and expensive repeated checks (besides the one we mention below). Here we make the design choice of allowing rational coefficients in the witness, and delegate the task of enforcing integrality to the PIOP through a lookup PIOP. This subtlety is explained at length in the technical overview of the Zinc paper [**Zinc**]. Because it is not novel, we do not dwell on it further.

We note that combining an IOPP (or PCS) for $\mathbb{Q}^{<d_0,B}[X]$ with a lookup PIOP for $\mathbb{Z}^{<d}[X]$ provides an IOPP (or PCS) for $\mathbb{Z}^{<d_0,B}[X]$, and hence our work provides that.

Modular reductions of rationals Observe that the $\mathbb{F}_{q_i}[X]$-constraints involve the reduction of the vector $\mathbf{f}_0$ modulo q_i (via the map ϕ_q). However, the entries of $\mathbf{f}_0$ are polynomials with rational coefficients. The reduction of a rational a/b modulo a prime q is only well-defined when b is not zero modulo q, and in that case it is defined as $a \cdot b^{-1} \mod q$ where b^{-1} is the modular inverse of b section 4.2. We understand the triple $(\mathbb{i}, \mathbb{x}; \mathbb{w})$ to be invalid if some modular reduction is not well-defined. This is not a problem for our proof system, as the reduction can be performed by the prover and the verifier can check that the reduction was done correctly by checking that the difference between the reduced and non-reduced polynomials belongs to the ideal generated by q_i in $\mathbb{Q}[X]$.

2.4.2 Lookup and permutation constraints as UCS constraints

In this section and in section 2.4.3, we show how several common relations and constraints—namely lookups, permutations, R1CS, CCS, and AIR—can be written as UCS constraints. For clarity, we present these reductions in isolation; however, in practice one can combine multiple constraint types within a single UCS instance. See our UAIR$^+$ relation in **??** for an example.

We begin with lookup relations. The construction follows Section 4.4 of [**Zinc**], which proves

the result for any integral domain; we include it here for completeness but omit a detailed proof. We work over rings R of the form $\mathbb{Q}^{<d,B}[X]$ or $\mathbb{F}_q^{<d}[X]$ for some prime power q, bit-size bound B, and degree bound d.

We define the *lookup relation* over R as:

$$\mathsf{REL}_{\mathsf{Look}} = \left\{ (\mathbb{i}, \mathbb{x}; \mathbb{w}) \;\middle|\; \begin{array}{l} \mathbb{i} = (m, \tau, \mathbf{t}, R), \\ m, \tau \geq 0,\ \mathbf{t} \in R^{\tau}, \\ \mathbb{x} = \emptyset,\ \mathbb{w} = \mathbf{w} \in R^m, \\ \mathbf{w}_i \in \{\mathbf{t}_j \mid j \in [\tau]\} \text{ for all } i \in [m]. \end{array} \right\}.$$

Let $\mathbb{i}_{\mathsf{Look}} := (m, \tau, \mathbf{t}, R)$ denote an index for $\mathsf{REL}_{\mathsf{Look}}$. We now construct a UCS index $\mathbb{i}_{\mathsf{UCS}}$ such that $(\mathbb{i}_{\mathsf{Look}}, \emptyset; \mathbf{w}) \in \mathsf{REL}_{\mathsf{Look}}$ if and only if $(\mathbb{i}_{\mathsf{UCS}}, \emptyset; \mathbb{w}') \in \mathsf{REL}_{\mathsf{UCS}}$ for an explicit transformation $\mathbb{w}'$ of $\mathbf{w}$.

We set $n := m$ and $|\mathcal{C}| := 1$. Recall that the *Lagrange basis polynomial* for $i \in [N]$ is

$$\mathsf{Lag}_{[N],i}(T) := \prod_{j \neq i} \frac{T - j}{i - j}. \tag{2.19}$$

It is well-defined over $\mathbb{Q}$ or any $\mathbb{F}_q$ with $q > N$. Define

$$L(K, \mathbf{Z}) \;:=\; \sum_{k \in [m]} \mathbf{Z}_k \cdot \mathsf{Lag}_{[m],k}(K) \tag{2.20}$$

so that $L(k, \mathbf{z}) = \mathbf{z}_k$ for all $k \in [m]$. Define

$$Q_{\mathsf{Look}}(K, \mathbf{Z}) \;:=\; \prod_{j \in [\tau]} (L(K, \mathbf{Z}) - \mathbf{t}_j). \tag{2.21}$$

If $R = \mathbb{Q}^{<d,B}[X]$, define $\mathbb{i}_{\mathsf{UCS}} = (m, \ell{=}0, n, \mathbf{q}{=}\emptyset, B, \mathbf{d}{=}d, \mathcal{C})$ with $\mathcal{C} = (Q_{0,1}, \mathfrak{I}_{0,1}) := (Q_{\mathsf{Look}}, (0))$. Here the subscript $(0, 1)$ denotes prime-index $i = 0$ (the $\mathbb{Q}[X]$-part) and constraint-index $t = 1$ (the single constraint).

If $R = \mathbb{F}_q^{<d}[X]$, define $\mathbb{i}_{\mathsf{UCS}} = (m, \ell{=}0, n, \mathbf{q}{=}q, B, \mathbf{d}{=}(0, d), \mathcal{C})$ with $\mathcal{C} = ((Q_{0,1}, \mathfrak{I}_{0,1}), (Q_{1,1}, \mathfrak{I}_{1,1}))$ where $Q_{0,1} \equiv 0$, $\mathfrak{I}_{0,1} = (0) \subseteq \mathbb{Q}[X]$, $\mathfrak{I}_{1,1} = (0) \subseteq \mathbb{F}_q[X]$, and $Q_{1,1}(K, \mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2) := Q_{\mathsf{Look}}(K, \mathbf{Z}_2)$.

Lemma 2.4.4 (Lookup as UCS)**.** *Let* $\mathbb{i}_{\mathsf{Look}}$ *and* $\mathbb{i}_{\mathsf{UCS}}$ *be defined as above. For every* $\mathbf{w} \in R^m$*, letting* $\mathbb{x} = \emptyset$*, we have:*

- *If* $R = \mathbb{Q}^{<d,B}[X]$ *then* $(\mathbb{i}_{\mathsf{Look}}, \emptyset; \mathbf{w}) \in \mathsf{REL}_{\mathsf{Look}}$ *if and only if* $(\mathbb{i}_{\mathsf{UCS}}, \emptyset; \mathbf{w}) \in \mathsf{REL}_{\mathsf{UCS}}$.
- *If* $R = \mathbb{F}_q^{<d}[X]$ *then, writing* $\mathbf{0}$ *for the all-zero* $\mathbb{Q}$*-witness component of length* m*,* $(\mathbb{i}_{\mathsf{Look}}, \emptyset; \mathbf{w}) \in \mathsf{REL}_{\mathsf{Look}}$ *if and only if* $(\mathbb{i}_{\mathsf{UCS}}, \emptyset; (\mathbf{0}, \mathbf{w})) \in \mathsf{REL}_{\mathsf{UCS}}$.

The proof is straightforward and we omit it. The key observation is that R is an integral domain (both $\mathbb{Q}^{<d,B}[X]$ and $\mathbb{F}_q^{<d}[X]$ are polynomial rings over fields, hence have no zero divisors). Thus $Q_{\mathsf{Look}}(i, \mathbf{w}) = \prod_j (\mathbf{w}_i - \mathbf{t}_j) = 0$ if and only if $\mathbf{w}_i \in \{\mathbf{t}_j\}$.

Remark 2.4.5 (Degree blowup and practical lookup arguments)**.** The polynomial Q_{Look} in (2.21) has degree τ in the selector value $L(K, \mathbf{Z})$. The purpose of the above construction is to exhibit lookups as UCS instances, so that generic UCS compilers and soundness statements apply. In protocols, we will not prove (2.21) via a high-degree polynomial identity; instead, we will use lookup arguments that exploit the lookup semantics directly, following the approach of [**Zinc**].

Remark 2.4.6 (Lookup constraints on polynomial expressions of witness entries)**.** The lookup relation $\mathsf{REL}_{\mathsf{Look}}$ defined above requires each entry of a witness vector $\mathbf{w}$ to belong to a table $\mathbf{t}$. In applications, one sometimes needs to constrain *polynomial expressions* of witness entries to belong to a lookup table. Concretely, given a witness vector $\mathbf{w} \in R^m$ and a polynomial $P \in R[\mathbf{Z}]$, one may require that

$$P(\mathbf{w}_i) \;\in\; \{\mathbf{t}_j \mid j \in [\tau]\} \quad \text{for all } i \in [m].$$

The extension to this setting is straightforward: one replaces the selector polynomial $L(K, \mathbf{Z})$ in (2.20) with

$$L'(K, \mathbf{Z}) \;:=\; P\Big(\sum_{i \in [m]} \mathbf{Z}_i \cdot \mathsf{Lag}_{[m],i}(K)\Big),$$

and defines Q'_{Look} analogously. This generalization is needed in our definition of the relation UAIR$^+$ (**??**) and in some of our applications in **??**, notably in the SHA-256 arithmetization (**??**); see **??** for further discussion.

Remark 2.4.7 (Well-definedness as a lookup constraint)**.** Recall that definition 2.4.1 includes a well-definedness condition (2.18) requiring $\mathbf{f}_0 \in (\mathbb{Z}^{<d_0,B}_{(q_1\cdots q_{|\mathbf{q}|})}[X])^m$. This is simply a lookup constraint: the table is $\mathbf{t} := \mathbb{Z}^{<d_0,B}_{(q_1\cdots q_{|\mathbf{q}|})}[X]$, and each entry of $\mathbf{f}_0$ must belong to this finite set. Thus, the well-definedness condition can be enforced via a $\mathbb{Q}[X]$-constraint in UCS and does not require separate syntax in the definition of UCS.

Definition 2.4.8 (Well-formed UCS index)**.** A UCS index $\mathbb{i}$ is *well-formed* if it includes a $\mathbb{Q}[X]$-constraint that enforces the well-definedness condition (2.18).

In many applications, other lookup constraints placed on $\mathbf{f}_0$ already subsume the well-definedness condition (see **??**), so it does not need to be explicitly included. Note that while $\mathbf{t}$ may be very large (or even infinite in the limit), in practice the table size is bounded by the constraint parameters.

Remark 2.4.9 (Permutation constraints as UCS instances)**.** So-called *copy constraints* of the form $\mathbf{u}_i = \mathbf{v}_{\sigma(i)}$, where the permutation $\sigma : [n] \to [n]$ is fixed by the index, can also be encoded as UCS instances. Such constraints capture copy relations that arise from wiring in arithmetizations such as Plonk [**GWC19**]. We define the *permutation relation* over a ring R as

$$\mathsf{REL}_{\mathsf{perm}} = \left\{ (\mathbb{i}, \mathbb{x}; \mathbb{w}) \;\middle|\; \begin{array}{l} \mathbb{i} = (n, \sigma, R), \\ \mathbb{x} = \emptyset, \quad \mathbb{w} = (\mathbf{u}, \mathbf{v}) \in R^n \times R^n, \\ \mathbf{u}_i = \mathbf{v}_{\sigma(i)} \text{ for all } i \in [n]. \end{array} \right\}.$$

To write this relation as a UCS instance, we view the witness as a length-$2n$ vector $\mathbf{w} = (\mathbf{u}||\mathbf{v})$ and define the selector polynomial

$$Q_{\mathsf{Perm}}(K, \mathbf{Z}) := \sum_{i\in[n]} (\mathbf{Z}_i - \mathbf{Z}_{n+\sigma(i)}) \cdot \mathsf{Lag}_{[n],i}(K).$$

Then evaluating at $K = i \in [n]$ yields $Q_{\mathsf{Perm}}(i, \mathbf{w}) = \mathbf{u}_i - \mathbf{v}_{\sigma(i)}$. Thus, $Q_{\mathsf{Perm}}(i, \mathbf{w}) = 0$ for all $i \in [n]$ if and only if $(\mathbf{u}, \mathbf{v}) \in \mathsf{REL}_{\mathsf{perm}}$. The UCS construction then proceeds exactly as in lemma 2.4.4: for $R = \mathbb{Q}^{<d,B}[X]$, one sets $\mathcal{C} = (Q_{\mathsf{Perm}}, (0))$; for $R = \mathbb{F}_q^{<d}[X]$, one places the constraint in the finite-field component with $Q_{1,1}(K, \mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2) := Q_{\mathsf{Perm}}(K, \mathbf{Z}_2)$.

2.4.3 R1CS, CCS, and AIR as UCS instances

In this section we explain how several standard relations—R1CS, CCS, AIR (over fields, rationals, or polynomial rings)—can be written as $\mathsf{REL}_{\mathsf{UCS}}$ relations. Since the rings $\mathbb{Q}^{<d,B}[X]$ and $\mathbb{F}_q^{<d}[X]$ are integral domains, the constructions and proofs follow directly from [**Zinc**], which establishes the result for arbitrary integral domains; we summarize the key ideas below.

All transformations below work in a *strictly algebraic regime*: for every constraint we set $\mathfrak{I}_{it} = (0)$, so that a constraint $Q_{it}(\cdots) \in \mathfrak{I}_{it}$ reduces to the polynomial identity $Q_{it}(\cdots) = 0$.

R1CS. Recall that the R1CS relation over a ring R requires $(M_A \cdot \mathbf{z}) \circ (M_B \cdot \mathbf{z}) - (M_C \cdot \mathbf{z}) = \mathbf{0}$, where $M_A, M_B, M_C \in R^{n \times (\ell+m)}$ are constraint matrices and $\mathbf{z} = (\mathbf{y}||\mathbf{w})$ concatenates the public input and witness. The transformation to UCS proceeds as follows: first, use bivariate Lagrange interpolation to encode each matrix $M \in \{M_A, M_B, M_C\}$ as a polynomial $\hat{M}(K, Z)$; then define a linear selector $L_M(K; \mathbf{Y}, \mathbf{Z}_1)$ that evaluates to the k-th entry of $M \cdot \mathbf{z}$ when $K = k$; finally, set the UCS constraint polynomial to

$$Q_{\mathsf{R1CS}}(K, \mathbf{Y}, \mathbf{Z}_1) \;:=\; L_{M_A} \cdot L_{M_B} - L_{M_C}.$$

For R1CS over $\mathbb{Q}^{<d,B}[X]$, the UCS index has $\mathbf{q} = \emptyset$ and a single $\mathbb{Q}[X]$-constraint $(Q_{\mathsf{R1CS}}, (0))$. For R1CS over $\mathbb{F}_q^{<d}[X]$, one instead uses an $\mathbb{F}_q[X]$-constraint with a trivial $\mathbb{Q}[X]$-part. The equivalence $(\mathbb{i}_{\mathsf{R1CS}}, \mathbb{x}; \mathbb{w}) \in \mathsf{REL}_{\mathsf{R1CS}} \Leftrightarrow (\mathbb{i}_{\mathsf{UCS}}, \mathbb{x}; \mathbb{w}') \in \mathsf{REL}_{\mathsf{UCS}}$ follows from the defining property of Lagrange interpolation; see [**Zinc**] for the full proof.

CCS and AIR. CCS relations [**ccs**] admit an analogous transformation as R1CS relations. Further, since AIR relations [**ethstark**] can be written as CCS relations [**ccs**], they too can be instantiated as UCS instances. For an AIR-like instantiation tailored to the UCS framework, see **??**.

2.5 Zinc+ Compiler: from finite field PIOPs to PIOPs for UCS

In this section we present the Zinc+ compiler, which can be seen as a Polynomial Interactive Oracle Reduction (PIOR) from the UCS relation (definition 2.4.1) to a family of polynomial satisfiability relations over finite fields. In particular, given PIOPs for specific subclasses of finite-field relations (e.g., AIR-like constraints, R1CS, or CCS), one obtains a PIOP for the corresponding subclass of $\mathsf{REL}_{\mathsf{UCS}}$ via the PIOR. Before getting to the Zinc+ PIOR in section 2.5.4 we introduce a slight generalization of polynomial oracles called (possibly) incomplete polynomial oracles in section 2.5.1, introduce our techniques for batching ideal membership constraints and projecting the constraints onto finite fields in section 2.5.2, and introduce the strictly algebraic UCS relation we reduce to in section 2.5.3.

2.5.1 Incomplete polynomial oracles

Recall at the end of a PIOR that the verifier outputs a new instance $\mathbb{x}' \leftarrow \mathsf{V}(\mathbb{i}, \mathbb{x}, \pi_1, \rho_1, \dots, \pi_k, \rho_k)$, and any oracles therein are specified implicitly in terms of oracles given in $\mathbb{x}$ and $\pi_1, \dots, \pi_k$. For example, the new instance could contain an oracle $[[f']] = [[f]] + \rho_1 \cdot [[\pi_1]]$, for some $[[f]]$ included in the old instance. Assuming ρ_1 and the coefficients of f and π_1 all reside over the same ring, this new polynomial oracle is complete, in that it can be evaluated at any set of points in that ring.

In Zinc+ the verifier outputs a new instance with polynomial oracles of the form $\psi_\rho([[f(\psi_\rho^{-1}(\cdot))]])$, where we can always compute $v' = [[f(\psi_\rho^{-1}(\cdot))]]$, but for malicious provers v' may lie outside the domain ψ_ρ is defined on. We model this via the (possibly) *incomplete polynomial oracle* $\{\{f'\}\} = \psi_\rho([[f(\psi_\rho^{-1}(\cdot))]])$, which either outputs a correct evaluation or $\perp$ if an undefined operation is encountered during the evaluation, and which we syntactically distinguish from complete polynomial oracles using braces rather than brackets.

We can modify any PIOP to work with possibly incomplete oracles as follows. If the oracle outputs $\perp$, the verifier automatically rejects, and otherwise the protocol proceeds normally. If we can show that during the execution of the modified PIOP incomplete oracles that are not complete output $\perp$ whp, then whenever the verifier accepts we can assume whp that the possibly incomplete

oracle is in fact complete. For the possibly incomplete oracle $\{\{f'\}\}$ used in Zinc+, we show that when queried on a random point it outputs $\perp$ whp. This means that the very weak requirement that the PIOP queries $\{\{f'\}\}$ on a random point suffices for the security of our reduction, and in practice this does not rule out any public-coin PIOPs of interest.

2.5.2 Batching ideal membership checks and projecting to finite fields

In this section, we provide lemmas for reducing ideal membership constraints in $\mathsf{REL}_{\mathsf{UCS}}$ to strict equalities in $\mathbb{Q}[X]$ and $\mathbb{F}_{q_i}[X]$, and subsequently to zerocheck-type polynomial relations over fields $\mathbb{F}_{q_0}$ and $\mathbb{F}_{\tilde{q}_i}$.

Ideal batching. We first give our ideal batching lemma, which captures the fact that w.h.p.a vector $\mathbf{v}$ belongs to an ideal $\mathfrak{I}$ if the evaluation of the multilinear extension of $\mathbf{v}$ at a random point belongs to $\mathfrak{I}$. This is a consequence of the Schwartz-Zippel lemma over rings (lemma 2.3.1), which follows by looking at membership in $\mathfrak{I}$ as equality to zero over an appropriate quotient by $\mathfrak{I}$.

Lemma 2.5.1 (Testing ideal membership in $\mathbb{K}[X]$ via one random MLE evaluation)**.** *Let $\mathbb{K}$ be a field, and let $\mathfrak{I} \subseteq \mathbb{K}[X]$ be an ideal such that $\mathfrak{I} \cap \mathbb{K} = (0)$. Let $\mu \geq 1$, let $\mathbf{v} : \{0,1\}^\mu \to \mathbb{K}[X]$ be a vector indexed by $\{0,1\}^\mu$, and let $\tilde{\mathbf{v}}(\mathbf{Y}) \in (\mathbb{K}[X])[\mathbf{Y}]$ be its multilinear extension (as in (2.16)). The following hold:*

1. *If $\mathbf{v}(\mathbf{x}) \in \mathfrak{I}$ for all $\mathbf{x} \in \{0,1\}^\mu$, then $\tilde{\mathbf{v}}(\mathbf{r}) \in \mathfrak{I}$ for every $\mathbf{r} \in \mathbb{K}^\mu$.*

2. *If there exists $\mathbf{x}_0 \in \{0,1\}^\mu$ such that $\mathbf{v}(\mathbf{x}_0) \notin \mathfrak{I}$, then, for every nonempty finite set $S \subseteq \mathbb{K}$,*

$$\Pr_{\mathbf{r} \leftarrow S^\mu}\left[\tilde{\mathbf{v}}(\mathbf{r}) \in \mathfrak{I}\right] \;\leq\; \frac{\mu}{|S|}.$$

The proof is deferred to **??**.

Extend map. Let $\mathbb{P}$ be the set of all prime powers. Throughout the rest of the section, we fix the extend map

$$\mathsf{Extend} : \mathbb{P} \to \mathbb{P}$$

that will parameterize our protocols and definitions. This map is defined so that, given a prime power q, $\mathsf{Extend}(q) = q^\ell$ is a sufficiently large power of q (where precise requirements in terms of the security parameter are given in the round-by-round knowledge proof of Zinc+). We will generally write $\tilde{q}$ to mean the output of $\mathsf{Extend}(q)$.

Projecting from rings to fields. We now show how to sample a random projection from rings $\mathbb{Q}[X]$ and $\mathbb{F}_q[X]$ to sufficiently large finite fields.

Definition 2.5.2 (Evaluation projections)**.** Let $q = p^e$ be a prime power, $\tilde{q} = q^\ell$. Let $a \in \mathsf{Prim}(\mathbb{F}_{\tilde{q}})$ such that $\langle a \rangle = \mathbb{F}_{\tilde{q}}$. We define the surjective ring homomorphism

$$\psi_{q,a} : \mathbb{F}_q[X] \to \mathbb{F}_{\tilde{q}}, \qquad f(X) \mapsto \iota_{\tilde{q}}(f)(a),$$

where the canonical embedding is first applied coefficient-wise to f to lift it to the extension ring $\mathbb{F}_{\tilde{q}}$ and then the resulting polynomial is evaluated at a. We also define the right-inverse

$$\psi_{q,a}^{-1} : \mathbb{F}_{\tilde{q}} \to \mathbb{F}_q^{<\ell}[X]$$

which sends $v \in \mathbb{F}_{\tilde{q}}$ to the unique $g \in \mathbb{F}_q^{<\ell}[X]$ such that $\psi_{q,a}(g) = v$. To compute this explicitly, one can fix an irreducible degree ℓ polynomial b and represent elements of $\mathbb{F}_{\tilde{q}}$ as polynomials in $\mathbb{F}_q[X]/(b(X)) \cong \mathbb{F}_{\tilde{q}}$, where the coefficients in this representation are exactly the coefficients of g. We stress that g is unique and choice of b only affects arithmetic efficiency. While $\psi_{q,a} \circ \psi_{q,a}^{-1}$ is the identity function $\mathsf{id}_{\mathbb{F}_{\tilde{q}}}$ on $\mathbb{F}_{\tilde{q}}$, we have that $\psi_{q,a}^{-1} \circ \psi_{q,a}$ is only the identity function on the restricted domain $\mathbb{F}_q^{<\ell}[X] \subset \mathbb{F}_q[X]$.

Next, let q_0 be a prime, $a \in \mathbb{F}_{q_0}$. We extend $\psi_{q_0,a}$ to a surjective ring homomorphism

$$\psi_{q_0,a} : \mathbb{Z}_{(q_0)}[X] \to \mathbb{F}_{q_0}, \qquad f(X) \mapsto \phi_{q_0}(f)(a),$$

where first we apply the ring homomorphism ϕ_{q_0} coefficient-wise to f to project it to $\mathbb{F}_q$ and then

evaluate at a. We also define the right-inverse

$$\psi^{-1}_{q_0,a} : \mathbb{F}_{q_0} \to \mathbb{Z}_{(q_0)}[X]$$

which simply returns $v \in \mathbb{F}_{q_0}$ as a constant integral polynomial in $\mathbb{Z}_{(q_0)}[X]$.

Lemma 2.5.3. *Let $\psi : \mathcal{R} \to \mathcal{R}'$ be a surjective ring homomorphism. Let $n = 2^\mu$ and let $\mathbf{v} \in \mathcal{R}^n$ be a vector and $\tilde{\mathbf{v}}$ its multilinear extension. Let $\widetilde{\psi(\mathbf{v})}$ be the multilinear extension of $\psi(\mathbf{v}) \in \mathcal{R}'^n$. Then, for any* inverse map *$\psi^{-1} : \mathcal{R}' \to \mathcal{R}$, i.e. any map such that $\psi \circ \psi^{-1}$ is the identity map on $\mathcal{R}'$, we have*

$$\psi(\tilde{\mathbf{v}}(\psi^{-1}(\mathbf{u}))) = \widetilde{\psi(\mathbf{v})}(\mathbf{u}) \qquad \textit{for all } \mathbf{u} \in \mathcal{R}'^\mu.$$

Note that an inverse map ψ^{-1} exists since ψ is surjective.

The proof is deferred to **??**.

2.5.3 Strictly algebraic UCS

We introduce the strictly algebraic UCS relation, defined over a single prime-power field, which we will reduce UCS to. More precisely, Zinc+ reduces UCS to the joint relation $\wedge_{i\in[0,|\mathbf{q}|]}\mathsf{REL}_{\mathsf{UCS-ALG}}(\tilde{q}_i)$, where the new index-instance is of the form $\mathbb{i}' = (\mathbb{i}'_0, \dots, \mathbb{i}'_{|\mathbf{q}|})$, $\mathbb{x}' = (\mathbb{x}'_0, \dots, \mathbb{x}'_{|\mathbf{q}|})$ and

$$(\mathbb{i}', \mathbb{x}') \in \mathcal{L}\left(\wedge_{i\in[0,|\mathbf{q}|]}\mathsf{REL}_{\mathsf{UCS-ALG}}(\tilde{q}_i)\right) \iff (\mathbb{i}'_i, \mathbb{x}'_i) \in \mathcal{L}(\mathsf{REL}_{\mathsf{UCS-ALG}}(\tilde{q}_i)) \ (\forall i \in [0, |\mathbf{q}|]).$$

By combining the Zinc+ PIOR with PIOPs for each $\mathsf{REL}_{\mathsf{UCS-ALG}}(\tilde{q}_i)$ (or a single field-agnostic PIOP that can work for any prime-power), we obtain a PIOP for UCS.

Definition 2.5.4 (The relation $\mathsf{REL}_{\mathsf{UCS-ALG}}(q)$)**.** Let q be a prime power. The *strictly algebraic UCS relation* $\mathsf{REL}_{\mathsf{UCS-ALG}}(q) \subseteq \mathsf{REL}_{\mathsf{UCS}}$ is the subset of $\mathsf{REL}_{\mathsf{UCS}}$ formed by all triples $(\mathbb{i}, \mathbb{x}; \mathbb{w}) \in \mathsf{REL}_{\mathsf{UCS}}$ where all constraints are set over the same finite field and every ideal appearing in the index is the

zero ideal.

$$\mathsf{REL}_{\mathsf{UCS-ALG}}(q) = \left\{ \begin{pmatrix} \mathbb{i}, \\ \mathbb{x}, \\ \mathbb{w} \end{pmatrix} \middle| \begin{array}{l} \textbf{Index} \\ \quad \mathbb{i} = (m = 2^\nu, \ell, \mathcal{C}),\ \ \mathcal{C} = [Q_t]_{t \in [|\mathcal{C}|]},\ \ Q_t \in \mathbb{F}_q[\mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2], \\ \quad |\mathbf{Y}| = \ell,\ \ |\mathbf{Z}_1| = |\mathbf{Z}_2| = m \\ \textbf{Input} \\ \quad \mathbb{x} = (\mathbf{y}, [[\tilde{f}_1]], [[\tilde{f}_2]]),\ \mathbf{y} \in \mathbb{F}_q^\ell,\ \tilde{f}_1, \tilde{f}_2 \in \mathbb{F}_q[\mathbf{W}],\ |\mathbf{W}| = \nu \\ \textbf{Witness} \\ \quad \mathbb{w} = (\mathbf{f}_1, \mathbf{f}_2) \in \mathbb{F}_q^m \times \mathbb{F}_q^m, \\ \textbf{Algebraic constraints over } \mathbb{F}_q \\ \quad Q_t\left(\mathbf{y}, [\tilde{f}_1(\mathbf{w})]_{\mathbf{w} \in \{0,1\}^\nu}, [\tilde{f}_2(\mathbf{w})]_{\mathbf{w} \in \{0,1\}^\nu}\right) = 0 \text{ for } t \in [|\mathcal{C}|] \end{array} \right\}.$$

Remark 2.5.5 (Mapping from UCS witnesses to $\mathsf{REL}_{\mathsf{UCS-ALG}}$ witnesses)**.** The two witness vectors $\mathbf{f}_1, \mathbf{f}_2 \in \mathbb{F}_q^m$ in $\mathsf{REL}_{\mathsf{UCS-ALG}}(q)$ arise from the Zinc+ PIOR (Protocol 2.2) as follows. For the i-th target field $\mathbb{F}_{\tilde{q}_i}$, the PIOR sets $\mathbf{f}_1 = \psi_{q_i,a_i}(\mathbf{f}_0)$, which is the shared integer witness projected to $\mathbb{F}_{\tilde{q}_i}$, and $\mathbf{f}_2 = \psi_{q_i,a_i}(\mathbf{f}_i)$, which is the dedicated $\mathbb{F}_{q_i}[X]$-witness projected to $\mathbb{F}_{\tilde{q}_i}$. For the $\mathbb{Q}[X]$-part ($i = 0$), the constraints involve only $\mathbf{f}_1$ (i.e., only the projected shared witness), and $\mathbf{f}_2$ is unused.

2.5.4 Zinc+ Compiler

In this section, we present the Zinc+ PIOR, which together with a field-agnostic PIOP for $\mathsf{REL}_{\mathsf{UCS-ALG}}$ gives us a PIOP for $\mathsf{REL}_{\mathsf{UCS}}$. We then prove its completeness and round-by-round knowledge soundness. Below we sometimes write $\tilde{q}_0$ to mean q_0 for the sake of compactness.

The well-defined condition in $\mathsf{REL}_{\mathsf{UCS}}$ as a $\mathbb{Q}[X]$-constraint Recall that $\mathsf{REL}_{\mathsf{UCS}}$ specifies a well-defined constraint, which states that one of the witness vectors, namely $\mathbf{f}_0$, lies in the intersection of certain localization rings (definition 2.4.1, specifically eq. (2.18)). As demonstrated in remark 2.4.7, this condition can be written in the form of an algebraic constraint over $\mathbb{Q}[X]$. When such a constraint is included in the index, we call it a *well-formed* index, and only specify Zinc+ to

work with well-formed $\mathsf{REL}_{\mathsf{UCS}}$ instances.

Figure 2.2: Protocol $\Pi^{\mathsf{UCS}} = (\mathsf{P}_{\mathsf{UCS}}, \mathsf{V}_{\mathsf{UCS}})$: Zinc+ PIOR from $\mathsf{REL}_{\mathsf{UCS}}$ to $\wedge_{i\in[0,|\mathbf{q}|]}\mathsf{REL}_{\mathsf{UCS-ALG}}(\tilde{q}_i)$

Input: A well-formed index (definition 2.4.8) and input $(\mathbb{i}, \mathbb{x})$ for $\mathsf{REL}_{\mathsf{UCS}}$ (definition 2.4.1). $\mathsf{P}_{\mathsf{UCS}}$ additionally receives a witness $\mathbb{w}$ such that $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathsf{REL}_{\mathsf{UCS}}$. Write

$$\mathbb{i} = (m = 2^\nu, \ell, n = 2^\mu, \mathbf{q}, B, \mathbf{d}, \mathcal{C} = [(Q_{it}, \langle j_{it}\rangle)]_{i\in[0,|\mathbf{q}|], t\in[|\mathcal{C}|]}),\ \ \mathbb{x} = (\mathbf{y}) \in (\mathbb{Z}_{(\mathbf{q})}^{<d_0,B}[X])^\ell,$$
$$\mathbb{w} = (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|}),\ \mathbf{f}_0 \in (\mathbb{Z}_{(\mathbf{q})}^{<d_0,B}[X])^m,\ \mathbf{f}_i \in (\mathbb{F}_{q_i}^{<d_i}[X])^m \text{ for } i \in [|\mathbf{q}|].$$

1: $\mathsf{P}_{\mathsf{UCS}}$ sends polynomial oracles $[[\tilde{f}_0]], \dots, [[\tilde{f}_{|\mathbf{q}|}]]$, where $\tilde{f}_0 \in (\mathbb{Q}^{<d_0,B}[X])[\mathbf{W}]$, $\tilde{f}_1, \dots, \tilde{f}_{|\mathbf{q}|} \in (\mathbb{F}_{q_i}^{<d_i}[X])[\mathbf{W}]$, and $|\mathbf{W}| = \nu$.
2: $\mathsf{V}_{\mathsf{UCS}}$ sends a random prime $q_0 \leftarrow \mathcal{P}$ and field elements $\mathbf{r}_0 \leftarrow \mathbb{F}_{q_0}^\mu$, $\mathbf{r}_i \leftarrow \mathbb{F}_{\tilde{q}_i}^\mu$ for $i \in [|\mathbf{q}|]$.
3: If $Q_{0t} \notin (\mathbb{Z}_{(q_0)}[X])[K, \mathbf{Y}, \mathbf{Z}_1]$ or $\mathsf{j}_{0t}, \mathbf{y}, \mathbf{f}_0 \notin \mathbb{Z}_{(q_0)}[X]$ for any $t \in [|\mathcal{C}|]$, the protocol aborts.
4: $\mathsf{P}_{\mathsf{UCS}}$ sends for $i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]$:

$$\begin{aligned} e_{0t} &= \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \phi_{q_0}(Q_{0t}(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) && \in \mathbb{F}_{q_0}[X] \\ e_{it} &= \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \iota_{\tilde{q}_i}(Q_{it})(\mathbf{b}, \phi_{\tilde{q}_i}(\mathbf{y}), \phi_{\tilde{q}_i}(\mathbf{f}_0), \iota_{\tilde{q}_i}(\mathbf{f}_i)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) && \in \mathbb{F}_{\tilde{q}_i}[X] \end{aligned}$$

5: $\mathsf{V}_{\mathsf{UCS}}$ rejects if $e_{0t} \notin \langle \phi_{q_0}(\mathsf{j}_{0t})\rangle$ or $e_{it} \notin \langle \iota_{\tilde{q}_i}(\mathsf{j}_{it})\rangle$ for any $i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]$.
6: $\mathsf{V}_{\mathsf{UCS}}$ sends $\mathbf{a} = (a_0, a_1, \dots, a_{|\mathbf{q}|})$ by sampling $a_i \leftarrow \mathbb{F}_{\tilde{q}_i}$ for $i \in [0, |\mathbf{q}|]$, aborting if any $a_i \notin \mathsf{Prim}(\mathbb{F}_{\tilde{q}_i})$.
7: **for** $i \in [0, |\mathbf{q}|]$ **do**
8: Let

$$\begin{aligned} &Q'_{0t} = \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \psi_{q_0,a_0}(Q_{0t})(\mathbf{b}, \mathbf{Y}, \mathbf{Z}_1) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) - e_{0t}(a_0) \in \mathbb{F}_{q_0}[\mathbf{Y}, \mathbf{Z}_1] \\ &Q'_{it} = \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \psi_{q_i,a_i}(Q_{it})(\mathbf{b}, \mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) - e_{it}(a_i) \in \mathbb{F}_{\tilde{q}_i}[\mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2] \\ &\mathbf{y}'_0 = \psi_{q_0,a_0}(\mathbf{y}) \in \mathbb{F}_{q_0}^\ell,\ \mathbf{y}'_i = \psi_{q_i,a_i}(\mathbf{y}) \in \mathbb{F}_{\tilde{q}_i}^\ell \\ &\{\{\tilde{f}'_{i0}\}\} = \psi_{q_i,a_i}([[\tilde{f}_0(\psi^{-1}_{q_i,a_i}(\mathbf{W}))]]),\ [[\tilde{f}'_{i1}]] = \psi_{q_i,a_i}([[\tilde{f}_i(\psi^{-1}_{q_i,a_i}(\mathbf{W}))]]) \in \mathbb{F}_{\tilde{q}_i}[\mathbf{W}] \\ &\mathbf{f}'_{i0} = \psi_{q_i,a_i}(\mathbf{f}_0),\ \mathbf{f}'_{i1} = \psi_{q_i,a_i}(\mathbf{f}_i) \in (\mathbb{F}_{\tilde{q}_i})^m \end{aligned}$$

9: $\mathsf{V}_{\mathsf{UCS}}$ outputs for $i \in [|\mathbf{q}|]$:

$$\begin{aligned} \mathbb{i}'_0 &= \Big(m, \ell, q_0, \mathcal{C}'_0 = [Q'_{0t}]_{t\in[|\mathcal{C}|]}\Big) && \mathbb{x}'_0 = \Big(\mathbf{y}_0,\ \{\{\tilde{f}'_{00}\}\}\Big) \\ \mathbb{i}'_i &= \Big(m, \ell, \tilde{q}_i, \mathcal{C}'_i = [Q'_{it}]_{t\in[|\mathcal{C}|]}\Big) && \mathbb{x}'_i = \Big(\mathbf{y}'_i,\ \{\{\tilde{f}'_{i0}\}\},\ [[\tilde{f}'_{i1}]]\Big) \end{aligned}$$

10: $\mathsf{P}_{\mathsf{UCS}}$ outputs $\mathbb{w}'_i = (\mathbf{f}'_{i0}, \mathbf{f}'_{i1})$.
11: **end for**

Completeness and round-by-round knowledge soundness. Let $B_{\mathcal{P}}$ be a lower bound on the

bit length of primes in $\mathcal{P}$. Recall that the ℓ_0-norm of a polynomial indicates the number of nonzero rational coefficients.

As constraint polynomials are often very sparse, our completeness bound is written in terms of a completeness parameter ξ. After setting an upper bound on the density of the constraint polynomials, one may pick $\mathcal{P}$ that meets their desired completeness guarantees for all circuits that may be defined by such constraint polynomials. That said, it may still turn out $\mathcal{P}$ needs to be even larger to guarantee λ bits of security.

Theorem 2.5.6 (Completeness of the Zinc+ PIOR)**.** *Let*

$$|\mathcal{P}|(B_{\mathcal{P}} - 1) \geq 2^{\xi}(B-1)\left(d_0 \cdot (\ell + m) + \sum_{t \in [|\mathcal{C}|]} \|\mathsf{j}_{0t}\|_0 + \|Q_{0t}\|_0\right),$$

and let $\mathsf{Extend}(q) = q^k$ *pick* k *such that* $q^k \geq 2^{2\xi} \cdot (8(\xi+1))^2$*. Then* Π_{UCS} *(Figure 2.2) is complete (definition 2.3.3) with error* $\epsilon < (|\mathbf{q}| + 2)/2^{\xi}$*.*

Proof. First we show that unless the protocol halts in step 3 or 6 that $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \wedge_{i \in [0,|\mathbf{q}|]} \mathsf{REL}_{\mathsf{UCS-ALG}}(\tilde{q}_i)$. Then we bound the probability of this happening. We begin with $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathsf{REL}_{\mathsf{UCS}}$, where

$$\mathbb{i} = (m, \ell, n = 2^{\mu}, \mathbf{q}, B, \mathbf{d}, \mathcal{C} = [(Q_{it}, \langle j_{it} \rangle)]_{i \in [0,|\mathbf{q}|], t \in [|\mathcal{C}|]}), \;\; \mathbb{x} = (\mathbf{y}) \in (\mathbb{Z}_{(\mathbf{q})}^{<d_0,B}[X])^{\ell}$$
$$\mathbb{w} = (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|}), \;\; \mathbf{f}_0 \in \mathbb{Q}^{<d_0,B}[X]^m, \;\; \mathbf{f}_i \in \mathbb{F}_{q_i}^{<d_i}[X]^m \text{ for } i \in [|\mathbf{q}|].$$

In step 1, the prover sends the oracles for $\tilde{f}_0 \in (\mathbb{Q}^{<d_0,B}[X])[\mathbf{W}]$ and $\tilde{f}_1, \dots, \tilde{f}_{|\mathbf{q}|} \in (\mathbb{F}_{q_i}^{<d_i}[X])[\mathbf{W}]$, which are the MLEs of the corresponding witness vectors. In step 4, the prover sends

$$\begin{aligned} e_{0t} &= \textstyle\sum_{\mathbf{b} \in \{0,1\}^{\mu}} \phi_{q_0}(Q_{0t})(\mathbf{b}, \phi_{q_0}(\mathbf{y}), \phi_{q_0}(\mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) && \in \mathbb{F}_{q_0}[X] \\ e_{it} &= \textstyle\sum_{\mathbf{b} \in \{0,1\}^{\mu}} \iota_{\tilde{q}_i}(Q_{it})(\mathbf{b}, \phi_{\tilde{q}_i}(\mathbf{y}), \phi_{\tilde{q}_i}(\mathbf{f}_0), \iota_{\tilde{q}_i}(\mathbf{f}_i)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) && \in \mathbb{F}_{\tilde{q}_i}[X] \end{aligned}$$

for $i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]$. The verifier defines

$$\begin{aligned} Q'_{0t} &= \textstyle\sum_{\mathbf{b} \in \{0,1\}^{\mu}} \psi_{q_0,a_0}(Q_{0t})(\mathbf{b}, \mathbf{Y}, \mathbf{Z}_1) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) - e_{0t}(a_0) \in \mathbb{F}_{q_0}[\mathbf{Y}, \mathbf{Z}_1] \\ Q'_{it} &= \textstyle\sum_{\mathbf{b} \in \{0,1\}^{\mu}} \psi_{q_i,a_i}(Q_{it})(\mathbf{b}, \mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) - e_{it}(a_i) \in \mathbb{F}_{\tilde{q}_i}[\mathbf{Y}, \mathbf{Z}_1, \mathbf{Z}_2] \end{aligned}$$

Then plugging in $\mathbf{y}'_i = \psi_{q_i,a_i}(\mathbf{y})$, $\mathbf{f}'_{i0} = \psi_{q_i,a_i}(\mathbf{f}_0)$, $\mathbf{f}'_{i1} =$ we obtain

$$\begin{aligned} Q'_{0t} = \; & \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \left(\psi_{q_0,a_0}(Q_{0t})(\mathbf{b}, \psi_{q_0,a_0}(\mathbf{y}), \psi_{q_0,a_0}(\mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0)\right) \\ & - \left(\textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \phi_{q_0}(Q_{0t})(\mathbf{b}, \phi_{q_0}(\mathbf{y}), \phi_{q_0}(\mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0)\right)(a_0) \\ = \; & \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) \cdot [\psi_{q_0,a_0}(Q_{0t})(\mathbf{b}, \psi_{q_0,a_0}(\mathbf{y}), \psi_{q_0,a_0}(\mathbf{f}_0)) \\ & \qquad - \phi_{q_0}(Q_{0t})(\mathbf{b}, \phi_{q_0}(\mathbf{y}), \phi_{q_0}(\mathbf{f}_0))(a_0)] = 0, \end{aligned}$$

and similarly

$$\begin{aligned} Q'_{it} = \; & \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \left(\psi_{q_i,a_i}(Q_{it})(\mathbf{b}, \psi_{q_i,a_i}(\mathbf{y}), \psi_{q_i,a_i}(\mathbf{f}_0), \psi_{q_i,a_i}(\mathbf{f}_i)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i)\right) \\ & - \left(\textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \iota_{\tilde{q}_i}(Q_{it})(\mathbf{b}, \phi_{\tilde{q}_i}(\mathbf{y}), \phi_{\tilde{q}_i}(\mathbf{f}_0), \iota_{\tilde{q}_i}(\mathbf{f}_i)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i)\right)(a_i) \\ = \; & \textstyle\sum_{\mathbf{b}\in\{0,1\}^\mu} \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) \cdot [\psi_{q_i,a_i}(Q_{it})(\mathbf{b}, \psi_{q_i,a_i}(\mathbf{y}), \psi_{q_i,a_i}(\mathbf{f}_i), \psi_{q_i,a_i}(\mathbf{f}_i)) \\ & \qquad - \iota_{\tilde{q}_i}(Q_{it})(\mathbf{b}, \phi_{\tilde{q}_i}(\mathbf{y}), \phi_{\tilde{q}_i}(\mathbf{f}_0), \iota_{\tilde{q}_i}(\mathbf{f}_i))(a_i)] = 0. \end{aligned}$$

Finally, we conclude our claim of conditional correctness section with the observation that since each $a_i \in \mathsf{Prim}(\mathbb{F}_{\tilde{q}_i})$ by hypothesis, it holds by lemma 2.5.3 that $\{\{\tilde{f}'_{i0}\}\}$ and $[[\tilde{f}'_{i1}]]$ are (complete) polynomial oracles for the MLEs of the corresponding witness vectors.

Now it's left to bound the probability the protocol terminates early. The total number of rational coefficients we apply ϕ_{q_0} to is at most $N_{\text{rat}} = d_0 \cdot (\ell + m + |\mathcal{C}| + \sum_{t\in[|\mathcal{C}|]} \|Q_{0t}\|_0)$, where $\|Q\|_0$ denotes the size of the support of Q as a polynomial in $K, \mathbf{Y}$, and $\mathbf{Z}_1$. Then using lemma 2.5.8 and the union bound we can conclude the probability q_0 divides the denominator of any of these rational coefficients is less than

$$\frac{N_{\text{rat}}(B-1)}{|\mathcal{P}|(B_{\mathcal{P}}-1)}.$$

So given our hypothesis on $|\mathcal{P}|(B_{\mathcal{P}}-1)$ this happens with probability at most $2^{-\xi}$. Similarly, given our hypothesis on Extend we have that the event E_i that $a_i \notin \mathsf{Prim}(F_{\tilde{q}})$ occurs with probability at most $2^{-\xi}$ (by lemma 2.5.10). Taking the union bound over all E_i and also the event q_0 divides some denominator gives our completeness bound. □

Theorem 2.5.7 (Round-by-round knowledge soundness of the Zinc+ PIOR)**.** *Let the following*

hold for all $i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]$*:*

$$\begin{array}{ll} |\mathcal{P}|(B_{\mathcal{P}}-1) & \geq 2^{\lambda+4} d_0^2 B, \\ B_{\mathcal{P}} & \geq \lambda + \log(m \cdot \ell \cdot d_0 \cdot \mathsf{deg}(Q_{0t})), \\ |\mathsf{Prim}(\mathbb{F}_{\tilde{q}_i})| & \geq 2^{\lambda+1} m \cdot \ell \cdot d_i \cdot \mathsf{deg}(Q_{it}). \end{array}$$

Then Π_{UCS} *(Figure 2.2) has round-by-round knowledge soundness (definition 2.3.7) with errors* $\kappa_1 = \kappa_2 = 2^{-\lambda}$.

Proof. We make the simplifying assumption that $(\mathbb{i}', \mathbb{x}') \notin \mathcal{R}'$ unless $\{\{\tilde{f}'_{00}\}\}$ is complete, which effectively forces $\mathbf{f}_0 \in \mathbb{Z}_{(q)}[X]$. We believe this is justified based on the fact this can be enforced via asking for a single random evaluation of $\{\{\tilde{f}'_{00}\}\}$, which will abort with probability greater than $1 - \nu/2^{B_{\mathcal{P}}}$ unless complete.

First we define the extractor and knowledge state function, then we upper bound the round-by-round knowledge soundness errors. Throughout we use

$$\mathbb{i} = (m = 2^\nu, \ell, n = 2^\mu, \mathbf{q}, B, \mathbf{d}, \mathcal{C} = [(Q_{it}, \langle j_{it} \rangle)]_{i \in [0,|\mathbf{q}|], t \in [|\mathcal{C}|]}), \quad \mathbb{x} = (\mathbf{y}) \in (\mathbb{Z}_{(\mathbf{q})}^{<d_0,B}[X])^\ell,$$

and assume $\mathbb{i}$ is well-formed.

Extractor. For any transcript tr where the verifier is about to move, the extractor $\mathcal{E}$ queries each polynomial oracle $[[\tilde{f}_i]]$ at each point in the $\log m$-dimensional hypercube, and outputs the resulting vectors $(\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|})$, where $\mathbf{f}_0 \in (\mathbb{Q}^{<d_0,B}[X])^m$ and $\mathbf{f}_i \in (\mathbb{F}_{q_i}^{<d_i}[X])^m$ for $i \in [|\mathbf{q}|]$.

Knowledge state function. We define the knowledge state function's behavior in each round:

1. **Ring projection and ideal batching randomness:** At this stage the transcript tr has the form $\mathrm{tr} = ([[\tilde{f}_0]] \| \cdots \| [[\tilde{f}_{|\mathbf{q}|}]])$. The verifier samples a random prime $q_0 \leftarrow \mathcal{P}$ and field elements $\mathbf{r}_i \leftarrow \mathbb{F}_{\tilde{q}_i}^\mu$ for all $i \in [0, |\mathbf{q}|]$. We set $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr} \| q_0 \| \mathbf{r}) = 1$ if and only if running the extractor returns $(\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|})$ such that $\mathbf{f}_0 \in (\mathbb{Z}_{(q_0)}[X])^m$ and for $i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]$:

$$\begin{array}{lll} \sum_{\mathbf{b} \in \{0,1\}^\mu} \phi_{q_0}(Q_{0t}(\mathbf{b}, \mathbf{y}, \mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) & \in \langle \phi_{q_0}(\mathrm{j}_{0t}) \rangle & \subseteq \mathbb{F}_{q_0}[X], \\ \sum_{\mathbf{b} \in \{0,1\}^\mu} \iota_{\tilde{q}_i}(Q_{it})(\mathbf{b}, \phi_{\tilde{q}_i}(\mathbf{y}), \phi_{\tilde{q}_i}(\mathbf{f}_0), \phi_{\tilde{q}_i}(\mathbf{f}_i)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) & \in \langle \iota_{\tilde{q}_i}(\mathrm{j}_{it}) \rangle & \subseteq \mathbb{F}_{\tilde{q}_i}[X]. \end{array} \tag{2.22}$$

   Recall that since the index is well-formed this also implies $\mathbf{f}_0 \in (\mathbb{Z}_{(q_1,\dots,q_{|\mathbf{q}|})}[X])^m$.

2. **Field projection randomness:** At this stage the transcript tr has the form $\mathrm{tr} = ([[\tilde{f}_0]] \| \cdots \| [[\tilde{f}_{|\mathbf{q}|}]]$,

$q_0 \| \mathbf{r}, [e_{it}]_{i \in [0,|\mathbf{q}|], t \in [|\mathcal{C}|]})$. The verifier samples random field elements $a_i \leftarrow \mathbb{F}_{\tilde{q}_i}$, for $i \in [0, |\mathbf{q}|]$. We set $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr} \| q_0 \| \mathbf{r}) = 1$ if and only if running the extractor returns $(\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_{|\mathbf{q}|})$ such that $\mathbf{f}_0 \in (\mathbb{Z}_{(q_0)}[X])^m$ and for $i \in [|\mathbf{q}|], t \in [|\mathcal{C}|]$:

$$\begin{array}{ll} \left(\sum_{\mathbf{b} \in \{0,1\}^\mu} \phi_{q_0}(Q_{0t})(\mathbf{b}, \phi_{q_0}(\mathbf{y}), \phi_{q_0}(\mathbf{f}_0)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_0) - e_{0t}\right)(a_0) = 0 & \text{over } \mathbb{F}_{q_0} \\ \left(\sum_{\mathbf{b} \in \{0,1\}^\mu} \iota_{\tilde{q}_i}(Q_{it})(\mathbf{b}, \phi_{\tilde{q}_i}(\mathbf{y}), \phi_{\tilde{q}_i}(\mathbf{f}_0), \phi_{\tilde{q}_i}(\mathbf{f}_i)) \cdot \widetilde{\mathsf{eq}}(\mathbf{b}, \mathbf{r}_i) - e_{it}\right)(a_i) = 0 & \text{over } \mathbb{F}_{\tilde{q}_i} \end{array} \tag{2.23}$$

3. **Full transcript:** It remains to show for a full transcript $\mathsf{tr} = ([[\tilde{f}_0]] \| \cdots \| [[\tilde{f}_{|\mathbf{q}|}]], q_0 \| \mathbf{r}, \mathbf{e}, \mathbf{a})$ that $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 1$ if and only if $\mathsf{V}(\mathbb{i}, \mathbb{x}, \mathsf{tr})$ outputs $(\mathbb{i}', \mathbb{x}')$ such that there exists $\mathbb{w}$' with $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \mathcal{R}'$.

   First assume $\mathsf{V}(\mathbb{i}, \mathbb{x}, \mathsf{tr})$ outputs $(\mathbb{i}', \mathbb{x}')$ such that there exists $\mathbb{w}$' with $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \mathcal{R}'$. Since $(\mathbb{i}', \mathbb{x}') \notin \mathcal{L}(\mathcal{R}')$ unless $\{\{\tilde{f}'_{00}\}\}$ is complete by assumption, we have that $\mathbf{f}_0 \in \mathbb{Z}_{(q_0)}$. The $\mathbb{w}$' is by definition of UCS and by lemma 2.5.3 comprised of the vectors obtained by applying ψ to the $\mathbf{f}_i$ as in an honest run of the protocol. Then eq. (2.23) is exactly checking if that unique $\mathbb{w}$' satisfies $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \mathcal{R}'$, outputting $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 1$ if so.

   Next assume $\mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 1$. In this case it is straightforward to see we can explicitly produce a $\mathbb{w}$' such that $(\mathbb{i}', \mathbb{x}', \mathbb{w}') \in \mathcal{R}'$ using the extracted vectors.

RBR knowledge soundness. We upper bound the round-by-round knowledge soundness errors based on the knowledge state function and extractor described above.

1. **Ring projection and ideal batching randomness:** We show that

$$\Pr_{q_0, \mathbf{r}} \left[\begin{array}{l} \mathsf{KState}(\mathbb{i}, \mathbb{x}, ([[f_i]])_{i=0}^{|\mathbf{q}|} \| (q_0, \mathbf{r})) = 1 \\ \wedge\ (\mathbb{i}, \mathbb{x}, \mathcal{E}(\mathbb{i}, \mathbb{x}, ([[f_i]])_{i=0}^{|\mathbf{q}|})) \notin \mathcal{R} \end{array} \right] \leq 2^{-\lambda}.$$

   By lemma 2.5.9 we have that

$$\Pr_{q_0 \leftarrow \mathcal{P}} \left[\phi_{q_0}(Q_{0t}(k, \mathbf{y}, \mathbf{f}_0)) \in \langle \phi_{q_0}(\mathsf{j}_{0t}) \rangle \ \wedge\ Q_{0t}(k, \mathbf{y}, \mathbf{f}_0) \notin \langle \mathsf{j}_{0t} \rangle \right] < \frac{8 d_0^2 B}{|\mathcal{P}|(B_{\mathcal{P}} - 1)} < 2^{-\lambda - 1}.$$

   We note applying the canonical embedding cannot result in an unsatisfied constraint over $\mathbb{F}_{q_i}$ being satisfied in $\mathbb{F}_{\tilde{q}_i}$.

Else suppose for some (k, i, t) a constraint doesn't hold over $\mathbb{F}_{q_0}$ or $\mathbb{F}_{\tilde{q}_i}$. In this case lemma 2.5.1 tells us that eq. (2.22) will hold with probability at most $\mu/|\mathbb{F}_{\tilde{q}_i}| \leq 2^{-\lambda-1}$. A union bound over these two cases gives $\kappa_1 = 2^{-\lambda}$.

We now turn to the second source of randomness, namely the field-projection points $\mathbf{a}$, and bound the probability that the knowledge state transitions from 0 to 1 in this round.

2. **Field projection randomness:** We show that for all transcripts of the form $\mathsf{tr} = ([[f_i]])_{i=0}^{|\mathbf{q}|} \| (q_0, \mathbf{r}) \| [e_{it}]_{it}$ that

$$\Pr_{\mathbf{a}} \left[\begin{array}{l} \mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 0 \\ \wedge\ \mathsf{KState}(\mathbb{i}, \mathbb{x}, \mathsf{tr} \| \mathbf{a}) = 1 \\ \wedge\ (\mathbb{i}, \mathbb{x}, \mathcal{E}(\mathbb{i}, \mathbb{x}, \mathsf{tr})) \notin \mathcal{R} \end{array} \right] \ \leq\ 2^{-\lambda}.$$

To do this we must consider the case that for some $i \in [0, |\mathbf{q}|], t \in [|\mathcal{C}|]$ lemma 2.5.1 doesn't hold, but when projected to $\mathbb{F}_{\tilde{q}_i}$ via ψ_{a_i} it does. This happens with probability at most $2m\ell d_i \mathsf{deg}(Q_{it})/|\mathsf{Prim}(\mathbb{F}_{\tilde{q}_i})| \leq 2^{-\lambda} = \kappa_2$.

□

Supplementary lemmas. Here we include some lemmas we draw on in our proofs of completeness and RBR knowledge.

Lemma 2.5.8. *Let $B > B_{\mathcal{P}}$. For any $a \in \mathbb{Z}^{<B}\ \{0\}$ of bit length less than B, the number of primes of bit length at least $B_{\mathcal{P}}$ dividing it is less than $(B-1)/(B_{\mathcal{P}}-1)$.*

Proof. Say t primes $p_1, \ldots, p_t$ divide a. By assumption $|a| < 2^{B-1}$ and that $p_i \geq 2^{B_{\mathcal{P}}-1}$ for all $i \in [t]$. Then we have $2^{t(B_{\mathcal{P}}-1)} \leq |a| < 2^{B-1}$, which implies $t < (B-1)/(B_{\mathcal{P}}-1)$. □

Lemma 2.5.9. *Let $f, \mathsf{j} \in \mathbb{Q}^{<d,<B}[X]$, $d \geq 1$, $B > B_{\mathcal{P}} \geq 2$. Then*

$$\Pr_{p \leftarrow \mathcal{P}} \left[\phi_p(f) \in \langle \phi_p(\mathsf{j}) \rangle \ \wedge\ f \notin \langle \mathsf{j} \rangle \ \middle|\ f, \mathsf{j} \in \mathbb{Z}_{(p)} \right] < \frac{8d^2 B}{|\mathcal{P}|(B_{\mathcal{P}}-1)}.$$

Proof. Let $(q, r) \in \mathbb{Q}[X] \times \mathbb{Q}[X]$ be the unique quotient–remainder pair given by Euclidean division in $\mathbb{Q}[X]$:

$$f = \mathsf{j}q + r, \qquad \deg(r) < \deg(\mathsf{j}).$$

Then $f \in \langle \mathsf{j} \rangle$ if and only if $r = 0$.

Write r in lowest terms as $r = n_r/d_r$, where $n_r \in \mathbb{Z}[X]$ and $d_r \in \mathbb{Z}^+$ satisfy $\gcd(d_r, c) = 1$, where $c = \mathsf{cont}(n_r)$ is the content of n_r. Similarly write a, the leading coefficient of j, in lowest terms $a = n_a/d_a$. Define $N := c \cdot n_a$. We show that for every prime p,

$$\Big(\phi_p(f) \in \langle \phi_p(\mathsf{j}) \rangle \ \wedge \ f \notin \langle \mathsf{j} \rangle \ \wedge \ f, \mathsf{j} \in \mathbb{Z}_{(p)}\Big) \Longrightarrow p \mid N.$$

Fix p and assume $\phi_p(f) \in \langle \phi_p(\mathsf{j}) \rangle$, $f \notin \langle \mathsf{j} \rangle$, and $f, \mathsf{j} \in \mathbb{Z}_{(p)}$. Define $N := c \cdot n_a$. We show $p \mid N$. First assume $p \nmid n_a$. Then the leading coefficient of j is a unit in $\mathbb{Z}_{(p)}$, so Euclidean division of f by j can be performed in $\mathbb{Z}_{(p)}[X]$. Let $(q_p, r_p) \in \mathbb{Z}_{(p)}[X] \times \mathbb{Z}_{(p)}[X]$ be the resulting quotient–remainder pair, so

$$f = \mathsf{j} q_p + r_p, \qquad \deg(r_p) < \deg(\mathsf{j}).$$

By uniqueness of Euclidean division in $\mathbb{Q}[X]$, we have $r_p = r$, and in particular $r \in \mathbb{Z}_{(p)}[X]$. Applying ϕ_p gives

$$\phi_p(f) = \phi_p(\mathsf{j}) \cdot \phi_p(q_p) + \phi_p(r).$$

Since $p \nmid n_a$, we have $\deg(\phi_p(\mathsf{j})) = \deg(\mathsf{j})$, hence $\deg(\phi_p(r)) < \deg(\phi_p(\mathsf{j}))$. Together with $\phi_p(f) \in \langle \phi_p(\mathsf{j}) \rangle$, this implies $\phi_p(r) = 0$. Because $r \in \mathbb{Z}_{(p)}[X]$, we have $\phi_p(r) = 0$ if and only if $p \mid \mathsf{cont}(n_r) = c$. Hence $p \mid c$, and therefore $p \mid N$.

Next assume $p \nmid c$. If also $p \nmid n_a$, then the same argument shows $r \in \mathbb{Z}_{(p)}[X]$ and $\phi_p(r) = 0 \Longleftrightarrow p \mid c$, so $p \nmid c$ implies $\phi_p(r) \neq 0$ while $\deg(\phi_p(r)) < \deg(\phi_p(\mathsf{j}))$. A nonzero polynomial of degree strictly less than $\deg(\phi_p(\mathsf{j}))$ cannot lie in $\langle \phi_p(\mathsf{j}) \rangle$, contradicting $\phi_p(f) \in \langle \phi_p(\mathsf{j}) \rangle$. Therefore $p \mid n_a$, and hence $p \mid N$.

Let $\mathcal{P}_{\text{bad}} = \{p \in \mathcal{P} : p \mid N\}$. The above implication yields

$$\Pr_{p \leftarrow \mathcal{P}} \Big[\phi_p(f) \in \langle \phi_p(\mathsf{j}) \rangle \ \wedge \ f \notin \langle \mathsf{j} \rangle \ \Big| \ f, \mathsf{j} \in \mathbb{Z}_{(p)}\Big] \leq \frac{|\mathcal{P}_{\text{bad}}|}{|\mathcal{P}|}.$$

Let $B_N = \lceil \log(|N| + 1) \rceil$. If $B_N \leq B_{\mathcal{P}}$ then $|\mathcal{P}_{\text{bad}}| = 0$. Otherwise, by lemma 2.5.8,

$$|\mathcal{P}_{\text{bad}}| < \frac{B_N - 1}{B_{\mathcal{P}} - 1},$$

so

$$\Pr_{p \leftarrow \mathcal{P}}[\cdots] < \frac{B_N - 1}{|\mathcal{P}|(B_{\mathcal{P}} - 1)}. \tag{2.24}$$

We now bound B_N. Since the coefficients of f and j have bitlength $< B$, we may write each coefficient in lowest terms with numerator and denominator of magnitude $< 2^B$. Let $\Delta \in \mathbb{Z}^+$ be the product of all coefficient denominators appearing in f and j. Then $\Delta < 2^{2dB}$, and the polynomials

$$F := \Delta f \in \mathbb{Z}[X], \qquad J := \Delta \mathsf{j} \in \mathbb{Z}[X]$$

satisfy $\|F\|_\infty, \|J\|_\infty < 2^B \Delta < 2^{(2d+1)B}$.

Let $m := \deg(F)$ and $n := \deg(J)$, so $m, n < d$. Let b be the leading coefficient of J. By pseudo-division, there exist $Q', R' \in \mathbb{Z}[X]$ with $\deg(R') < n$ such that

$$b^{m-n+1} F = JQ' + R'. \tag{2.25}$$

Dividing (2.25) by Δ and comparing with $f = \mathsf{j}q + r$ shows that

$$r = \frac{R'}{b^{m-n+1}\Delta}.$$

In particular, there exists $\alpha \in \mathbb{Z}^+$ such that $R' = \alpha n_r \in \mathbb{Z}[X]$ and $b^{m-n+1}\Delta = \alpha d_r \in \mathbb{Z}\ \{0\}$, hence

$$c = \mathsf{cont}(n_r) \leq \|n_r\|_\infty \leq \|R'\|_\infty.$$

Each coefficient of R' can be expressed as a determinant of a matrix of dimension at most $m+n < 2d$ whose entries are coefficients of F and J. By Hadamard's inequality,

$$\|R'\|_\infty \leq (2d)^d \cdot \left(\max(\|F\|_\infty, \|J\|_\infty)\right)^{2d} < (2d)^d \cdot \left(2^{(2d+1)B}\right)^{2d} = (2d)^d \cdot 2^{(4d^2+2d)B}.$$

Therefore $c < (2d)^d \cdot 2^{(4d^2+2d)B}$. Since $|n_a| < 2^B$, we have

$$|N| < (2d)^d \cdot 2^{(4d^2+2d+1)B}.$$

It follows that

$$B_N = \lceil \log(|N|+1) \rceil \leq (4d^2+2d+1)B + d\log(2d) + 1 < 8d^2 B.$$

Substituting this bound into eq. (2.24) completes the proof. □

Lemma 2.5.10. *Let q be a prime power, $k \geq 2$ an integer. Let* $\mathsf{Extend}(q) = q^k$ *pick k such that $q^k \geq 2^{2\xi} \cdot (8(\xi+1))^2$. Define* $\mathsf{Prim}(\mathbb{F}_{q^k}) = \{\alpha \in \mathbb{F}_{q^k} \mid \mathbb{F}_q(\alpha) = \mathbb{F}_{q^k}\}$. *Then*

$$\frac{|\mathsf{Prim}(\mathbb{F}_{q^k})|}{q^k} \geq 1 - 2^{-\xi}.$$

Proof. An element $\alpha \in \mathbb{F}_{q^k}$ fails to generate $\mathbb{F}_{q^k}$ over $\mathbb{F}_q$ if and only if α lies in a proper intermediate subfield $\mathbb{F}_{q^d}$ for some divisor $d \mid k$ with $d < k$. Thus

$$\mathbb{F}_{q^k} \ \mathsf{Prim}(\mathbb{F}_{q^k}) \subseteq \bigcup_{\substack{d \mid k \\ d<k}} \mathbb{F}_{q^d}.$$

Taking cardinalities and applying the union bound yields

$$|\mathbb{F}_{q^k} \ \mathsf{Prim}(\mathbb{F}_{q^k})| \leq \sum_{\substack{d \mid k \\ d<k}} |\mathbb{F}_{q^d}| = \sum_{\substack{d \mid k \\ d<k}} q^d.$$

Dividing by q^k gives

$$1 - \frac{|\mathsf{Prim}(\mathbb{F}_{q^k})|}{q^k} \leq \sum_{\substack{d \mid k \\ d<k}} q^{d-k}.$$

Let ℓ denote the smallest prime divisor of k. Then every proper divisor d of k satisfies $d \leq k/\ell$, so $q^{d-k} \leq q^{k/\ell - k} = q^{-(k-k/\ell)}$. Since k has at most $k-1$ proper divisors,

$$1 - \frac{|\mathsf{Prim}(\mathbb{F}_{q^k})|}{q^k} \leq (k-1)\, q^{-(k-k/\ell)}.$$

Using $\ell \geq 2$, we have $k - k/\ell \geq k/2$, hence

$$(k-1)\, q^{-(k-k/\ell)} \leq (k-1)\, q^{-k/2}.$$

Write $N := q^k$. Then $q^{-k/2} = N^{-1/2}$, so the previous equation becomes

$$1 - \frac{|\mathsf{Prim}(\mathbb{F}_{q^k})|}{q^k} \leq (k-1)\, N^{-1/2}.$$

Moreover, since $q \geq 2$ and $k \geq 2$, we have $N = q^k \geq 2^k$, and thus $k - 1 \leq \log_2 N$. Therefore

$$1 - \frac{|\mathsf{Prim}(\mathbb{F}_{q^k})|}{q^k} \leq \frac{\log_2 N}{\sqrt{N}}.$$

By the hypothesis $N = q^k \geq 2^{2\xi} \cdot (8(\xi+1))^2$, we have $\sqrt{N} \geq 2^{\xi} \cdot 8(\xi+1)$. Also,

$$\log_2 N \leq \log_2\Big(2^{2\xi} \cdot (8(\xi+1))^2\Big) = 2\xi + 2\log_2(8(\xi+1)) \leq 8(\xi+1),$$

where the last inequality uses $\xi \geq 1$ and $\log_2(8(\xi+1)) \leq 3 + (\xi+1)$. Combining the above bounds gives

$$\frac{\log_2 N}{\sqrt{N}} \leq \frac{8(\xi+1)}{2^{\xi} \cdot 8(\xi+1)} = 2^{-\xi}.$$

Hence $1 - \frac{|\mathsf{Prim}(\mathbb{F}_{q^k})|}{q^k} \leq 2^{-\xi}$, which is equivalent to the claimed bound. $\square$

Chapter 3

Microlotus: Faster Proving Over Small Fields and Cyclotomic Rings

Nearly 30 years after the introduction of the first lattice-based cryptosystem [**Ajtai96**], lattice-based cryptography is now in widespread deployment. A major reason for this is the threat of large-scale quantum computers and the harvest now, decrypt later problem. Following NIST standardization of the Kyber key exchange mechanism and Falcon signature, we have seen them integrated in applications used by billions daily such as TLS, SSH, Signal and iMessage, Zoom, Wireguard and IKEv2, and MLS. We have also seen increasing real-world deployment of fully homomorphic encryption, all of which is lattice-based.

As lattice-based cryptography and succinct arguments have both realized increased use, naturally the interest in proving statements about lattice-based cryptography has grown as well. One important use case is building efficient versions of more complex primitives by verifying simpler ones using succinct non-interactive arguments of knowledge (SNARKs). In the lattice setting this approach has been particularly successful for constructing various types of signatures–aggregate, blind, group, multi, threshold, and ring–from standard signatures [**PQCRYPTO:BCOS20**; **AC:LyuNgu22**; **C:AABKT24**]. SNARKs have also been used to construct building blocks needed for other schemes such as...

Folklore suggests that lattice-based arguments are the natural candidate for verifying lattice-

based cryptography. This work studies using hash-based arguments instead. One of our contributions is the first construction of a hash-based PC for proving evaluations directly over the cyclotomic rings used by all modern lattice-based schemes. This approach does not actually suggest a contradiction, as the crux of the folklore argument was that lattice-based arguments natively proved statements over cyclotomic rings. We've simply shown how to construct linear codes over cyclotomic rings with the necessary proximity gaps required for building succinct PCs.

One compelling counterargument to the folklore is that arguments that work directly over cyclotomic rings $Z_q[X]/(\Phi(X))$ pay a linear proof size cost in the ring dimension n, where concretely we often have $n = 512$ or larger. On the other hand, prior work simulating ring arithmetic in $\mathbb{F}_q$ has incurred a quadratic prover complexity depdendence on n [**PQCRYPTO:BCOS20**]. An innovative new result of Huang, Mao, and Zhang [**EPRINT:HuaMaoZha25**] shows it's possible to avoid both costs via a new *ring-switching* argument, which recasts the statement over a Galois ring for which they present a new hash-based PC. Our work provides another way past this dichotomy by introducing an optimal $O(n \log n)$ prover complexity interactive oracle proof (IOP) for simulating ring multiplication over fields, which works with any field-agnostic multilinear PC. This avoids the constraint blowup of the ring-switching approach...

One obstacle we encounter when simulating ring arithmetic over $\mathbb{F}_q$ is that the small moduli used in many lattice-based constructions do not confer sufficient soundness when used with IOPs and hash-based PCs over the field. Numerous recent works have introduced methods for working over small fields [**ITCS:BGKS20**; **EPRINT:StarkWare21**; **EPRINT:HabLevPap24**; **EC:DiaPos25**; **EPRINT:DiaPos24**; **EPRINT:BagDomTha24**], all of which use some type of embedding of the witness in a sufficiently large extension field. Diamond and Posen present a particularly elegant way work over binary extension fields with essentially zero embedding overhead [**EPRINT:BagDomTha24**]. We extend their aproach to work over fields of odd characteristic.

[Implementation details go here. – Psi]

3.0.1 Contributions

This work then makes the following contributions:

1. We show how any linear $[n, k, d]$-code over a field $\mathbb{F}_q$ gives a linear $[n, k, d]$-code for all cyclotomic rings of the form $\mathbb{Z}_q[X]/(\Phi_m(X))$. We then further prove that linear codes over all finite rings R (and even certain infinite rings) satisfy a $(d/3, E)$-proximity gap in the unique decoding radius regime, where E is equal to the GCD of the order of its residue fields (for cyclotomic rings E is the order of its largest exceptional set minus one). This result enables us to construct an IOP of proximity to the ring, which we in turn use to build a hash-based multilinear PC over cyclotomic rings.
2. We present a new sumcheck-based IOP for f-circulant matrix multiplication. The $f = -1$ case corresponds to multiplication in power-of-two cyclotomic rings. If q is chosen to support a NTT over the ring as is generally the case, we . We also present a slightly less efficient IOP for multiplication in odd cyclotomics using the Chinese remainder transform.
3. We generalize the packing technique of Diamond and Posen [**EPRINT:BagDomTha24**] to fields of odd characteristic.
4. [Implementation details go here. – Psi]

3.0.2 Simulating ring arithmetic in native time

In **??** we introduce several new protocols for simulating ring arithmetic over fields, giving asymptotic improvements over prior work. These algorithms are specifically tailored for the popular power-of-two degree cyclotomic rings with prime moduli ubiquitous in lattice-based cryptography constructions, and provide security even for small moduli via field extension techniques. Algorithms include Chinese remainder transform (CRT)[1], multiplication, inner product (e.g., R-SIS), matrix-vector multiplication (e.g., M-SIS), and

In the following let $R = \mathbb{Z}[X]/\Phi_m(X)$ be the mth cyclotomic ring, which unless otherwise stated

[1]By CRT we mean the linear map from the power basis to the $\mathbb{Z}_q$-basis of R_q where ring multiplication simplifies to computing the Hadamard product of the basis vectors. This is sometimes referred to as NTT, and CRT is used to describe a different per-coefficient basis for composite moduli, but we choose to stick with original naming conventions [**EC:LyuPeiReg13**].

will be of the form $m = 2^{n+1}$. Let q a prime such that $q \equiv 1 \mod m$ (i.e., such that Φ_m splits fully over $\mathbb{Z}_q$), and let $n = \phi(m)$ the ring degree.

CRT transformation and a first multiplication attempt

For $a, b, c \in R$ we wish to reduce the relation $a \cdot b \equiv c \mod q$ to a sumcheck-friendly form over $\mathbb{F}_q$, where we are given (a, b, c) in the integral power basis $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_q^n$. In order to prove this in $O(n \log n)$ time over $\mathbb{F}_q$ we could check

$$\mathbf{c} = \mathsf{CRT}_{m,q}^{-1}(\mathsf{CRT}_{m,q} \cdot \mathbf{a} \circ \mathsf{CRT}_{m,q} \cdot \mathbf{b}) \tag{3.1}$$

where $\mathsf{CRT}_{m,q}$ is the $n \times n$ Chinese remainder transform (CRT) matrix over $\mathbb{Z}_q^*$ definition 3.0.3. This check can be performed in $O(n \log n)$ time natively. Liu, Xie, and Zhang [**CCS:LiuXieZha21**] give a sumcheck argument for the discrete Fourier transform $a_F = \mathsf{DFT}_{m,q} \cdot a$ with the same complexity, where $\mathsf{DFT}_{m,q}$ coincides with $\mathsf{CRT}_{m,q}$ for prime m. In **??** we present a generalization of this result, which works for prime powers and supports the inverse CRT as well. Equipped with this, we could proceed as follows.

Let $\tilde{F}, \widetilde{F^{-1}}, \tilde{a}, \tilde{b}, \tilde{c}$ be the multilinear extensions of $\mathsf{CRT}_{m,q}, \mathsf{CRT}_{m,q}^{-1}, \mathbf{a}, \mathbf{b}, \mathbf{c}$. In order to avoid the quadratic cost of having to compute $\tilde{F}(i, j)$ over $\{0,1\}^{2v}$, the prover first sends oracles $\widetilde{a_F}, \widetilde{b_F}$ corresponding to $\mathsf{CRT}_{m,q} \cdot a$, $\mathsf{CRT}_{m,q} \cdot b$. For a verifier sampled $\boldsymbol{\alpha} \leftarrow U(\mathbb{K}^v)$, where $\mathbb{K}/\mathbb{F}_q$ is an extension field determined by λ, the prover convinces the verifier the oracle is correct by using a sumcheck to show

$$\widetilde{a_F}(\boldsymbol{\alpha}) = \sum_{\mathbf{z} \in \{0,1\}^n} \tilde{F}(\boldsymbol{\alpha}, \mathbf{z}) \cdot \tilde{a}(\mathbf{z})$$

and similarly for $\widetilde{b_F}$. Then for a random $\boldsymbol{\beta} \leftarrow U(\mathbb{K}^n)$, the prover convinces a verifier that

$$\tilde{c}(\boldsymbol{\beta}) = \sum_{\mathbf{z} \in \{0,1\}^n} \widetilde{F^{-1}}(\boldsymbol{\beta}, \mathbf{k}) \cdot \widetilde{a_F}(\mathbf{z}) \cdot \widetilde{b_F}(\mathbf{z})$$

holds using another sumcheck.

While this approach achieves native asymptotic prover time, we show below how a constant factor

speedup can be achieved by viewing the ring multiplication instead as a circulant matrix-vector field multiplication. Regardless, computing the CRT transformation is often necessary outside the context of multiplication in lattice-based cryptography, so our $O(n \log n)$ prover algorithm remains useful.

Cyclotomic ring multiplication as circulant matrix-vector multiplication

For power-of-two degree cyclotomic rings, multiplication in the integral power basis can be represented as a particular structured matrix-vector multiplication

$$\mathbf{c} = Z_{-1}(\mathbf{a}) \cdot \mathbf{b} = \begin{bmatrix} a_0 & -a_{n-1} & \cdots & -a_1 \\ a_1 & a_0 & \ddots & \vdots \\ \vdots & \vdots & \ddots & -a_{n-1} \\ a_{n-1} & a_{n-2} & \cdots & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ \vdots \\ \vdots \\ b_{n-1} \end{bmatrix}$$

where $Z_{-1}(\mathbf{a})$ denotes the f-circulant matrix given by $f = -1$ and $\mathbf{a}$ (see definition 3.0.4). Below we give an efficient argument for f-circulant matrix-vector multiplication, and therefore power-of-two cyclotomic ring multiplication.

3.0.3 Multilinear polynomials

Lemma 3.0.1. *Lagrange interpolation of multilinear polynomials Let $f : \{0,1\}^v \to \mathbb{F}$ be any function. Then the following multilinear polynomial $\hat{f}$ extends f:*

$$\hat{f}(X_1, \dots, X_v) = \sum_{w \in \{0,1\}^v} f(w) \cdot \chi_w(X_1, \dots, X_v)$$

where, for any $w = (w_1, \dots, w_v)$

$$\chi_w(X_1, \dots, X_v) := \prod_{i=1}^{v} (X_i w_i + (1 - X_i)(1 - w_i))$$

The set $\{\chi_w : w \in \{0,1\}^v\}$ is referred to as the set of multilinear Lagrange basis polynomials with interpolating set $\{0,1\}^v$.

We will also sometimes write $\widetilde{\mathsf{eq}}(w, X)$ instead of $\chi_w(X)$ to denote the same polynomial.

Lemma 3.0.2. *Let $p : \mathbb{F}^\ell \to \mathbb{F}$ be a multilinear polynomial. Then for any $0 \leq i \leq \ell$, any $(\alpha_1, \dots, \alpha_i) \in \mathbb{F}^i$, and any $x' \in \{0,1\}^{\ell-i}$,*

$$p(\alpha_1, \dots, \alpha_i, x') = \sum_{y \in \{0,1\}^i} \widetilde{\mathsf{eq}}((\alpha_1, \dots, \alpha_i), y) \cdot p(y, x')$$

Setting $i = 1$, Equation (4) simplifies to

$$p(\alpha_1, x') = (1 - \alpha_1) \cdot p(0, x') + \alpha_1 \cdot p(1, x')$$

Setting $i = \ell$, we get the multilinear extension formula

$$p(\alpha_1, \dots, \alpha_\ell) = \sum_{y \in \{0,1\}^\ell} \widetilde{\mathsf{eq}}((\alpha_1, \dots, \alpha_\ell), y) \cdot p(y)$$

3.0.4 Rings

In this work we will assume all rings R to be unital and commutative. Though we will not cover ring theory in any depth, we briefly remind the reader of a few key concepts we make use of in this paper:

- *Maximal ideal*: an ideal $\mathfrak{m}$ that is maximal (with respect to set inclusion) amongst all proper ideals. In other words, no ideal $\mathfrak{n}$ is contained between $\mathfrak{m}$ and R.
- *Local ring*: a ring with a unique maximal ideal. Here the sum of any two non-units is a unit, for any $r \in R$ either r or $1 - r$ is a unit, and if any finite sum is a unit, then it has a term that is a unit.
- *Semilocal ring*: a ring with finitely many maximal ideals.
- *Residue field*: for any maximal ideal $\mathfrak{m}$, the residue field is the quotient ring $R/\mathfrak{m}$, which is a field.
- *Reduced ring*: a ring with no non-zero nilpotent elements. Equivalently, if $r^2 = 0$, then we must have $r = 0 \in R$.

Cyclotomic rings

Let $\zeta = \zeta_m$ be a primitive mth root of unity (i.e., $\zeta^m = 1$, but $\zeta^i \neq 1$ for $i \in [m-1]$). The mth cyclotomic polynomial Φ_m, for any positive integer m is the minimal polynomial of ζ over the integers:

$$\Phi_m(X) = \prod_{\substack{k \in [m], \\ \gcd(k,m)=1}} (X - \zeta^k)$$

where we observe its conjugates are the other primitive mth roots of unity. There is a natural isomorphism between $\mathbb{Z}[\zeta]$ and $R = \mathbb{Z}[X]/\Phi_m(X)$ given by $\zeta \mapsto X$. Let $n = \phi(m)$. Then $\mathbf{p} = [\zeta]_{i=0}^{n-1}$ defines a $\mathbb{Z}$-basis for R, called the *power basis*. We note that any $\mathbb{Z}$-basis for R is a $\mathbb{Z}_q$-basis for $R_q = \mathbb{Z}_q[X]/\Phi_m(X)$.

Another important $\mathbb{Z}_q$-basis for R_q is the Chinese remainder transform basis, or *CRT basis*, where ring multiplication simplifies to computing the Hadamard product of the Z_q^n vectors. The Chinese remainder transform can be seen as a specialization of the discrete Fourier transform to cyclotomic rings, and is also called the number theoretic transform (NTT). By virtue of being the rings of integers of algebraic number fields, cyclotomic rings are Dedekind, which means ideals can be factored into a produce of prime ideals. A particular case of interest is $q \equiv 1 \mod m$, which corresponds to the "fully splitting" case. Notice that $\mathbb{Z}_q$ contains a primitive root of unity ω_m. Then the principal ideal $\langle q \rangle$ in R can be written as the product of the prime ideals $\mathfrak{p}_i = \langle q \rangle + \langle \zeta_m - \omega_m^i \rangle$ for $i \in \mathbb{Z}_m^*$, i.e., it splits completely into n ideals of norm q each. Each quotient ring $R/\mathfrak{p}_i$ is isomorphic to the field $\mathbb{Z}_q$ via the map $\zeta_m \mapsto \omega_m^i$. Then by the Chinese remainder theorem

$$R/\mathfrak{n} \cong \prod_i R/\mathfrak{p}_i \cong (\mathbb{Z}_q)^n$$

The CRT basis $\mathbf{c}$ is then defined such that $c_i \equiv 1 \bmod \mathfrak{p}_i$ and $c_i \equiv 0 \bmod \mathfrak{p}_j$ for all $j \neq i$.

The Chinese remainder transform and its inverse convert between the power and CRT bases. The linear transform, given as a matrix $\mathsf{CRT}_{m,q}$ below, has the identity $\mathbf{p}^\top = \mathbf{c}^\top \cdot \mathsf{CRT}_{m,q}$.

Definition 3.0.3 ([**EC:LyuPeiReg13**])**.** Let m be a prime power and let R denote any commutative ring containing some element ω_m of multiplicative order m, i.e., a primitive mth root of unity.

- The *discrete Fourier transform* DFT_m over R is the $\mathbb{Z}_m \times \mathbb{Z}_m$ matrix whose (i,j)th entry is ω_m^{ij}.
- The *Chinese remainder transform* CRT_m over R is the (square) submatrix of DFT_m obtained by restricting to the rows indexed by $\mathbb{Z}_m^*$ and the columns indexed by $[\varphi(m)]$.

Applying DFT_m corresponds to evaluating evaluating a polynomial of degree less than m over $R[Y]$ (represented by the vector of coefficients in the natural order) at all the mth roots of unity. Similarly, CRT_m corresponds with evaluating a polynomial of degree less than $\phi(m)$ at the primitive mth roots of unity ω_m^i for $i \in \mathbb{Z}_m^*$.

3.0.5 Circulant matrices

Multiplication in the power basis of a power-of-two cyclotomic ring can be expressed as a negacyclic convolution or skew-circulant matrix-vector multiplication. We consider the following generalization of circulant matrices:

Definition 3.0.4 (f-circulant matrix)**.** Given a vector a, the f-circulant matrix $Z_f(a)$ is defined as

$$Z_f(a) = \begin{bmatrix} a_0 & f \cdot a_{n-1} & \cdots & f \cdot a_1 \\ a_1 & a_0 & \ddots & \vdots \\ \vdots & \vdots & \ddots & f \cdot a_{n-1} \\ a_{n-1} & a_{n-2} & \cdots & a_0 \end{bmatrix}$$

For $f = 1$ this corresponds to circulant matrices, and $f = -1$ corresponds to skew-circulant matrices.

Since $Z_1(a) = F_n^{-1} \cdot \mathsf{diag}(F_n a) \cdot F_n$, where F_n is the DFT matrix, circulant matrix-vector multiplication can be computed in $O(n \log n)$ time using 3 FFTs. A method given in [**Rosowski20**] runs in $O(n \log n)$ without FFTs, but still requires primitive nth roots of unity. In general,

$$Z_f(a) = \mathbf{diag}(\eta) \cdot F_n^{-1} \cdot \mathbf{diag}(F \cdot \mathbf{diag}(\eta) \cdot a) \cdot F \cdot \mathbf{diag}(\eta)$$

for $\eta = [1, f^{\frac{1}{n}}, \ldots, f^{\frac{n-1}{n}}]$ [**ClinePlemmonsWorm74**]. In other words we compute can compute ring multiplication in the power basis as $Z_{-1}(a) \cdot b = \eta \circ F_n^{-1}(F_n(\eta \circ a) \circ F_n(\eta \circ b))$.

For n a power-of-two, this gives us another way to write the CRT matrix: $C_{2n} = F_n \cdot \mathbf{diag}(\eta)$. This can be confirmed by observing the (i,j)th entry of such a matrix is $\omega_{2n}^{2ij} \cdot \omega_{2n}^{j}$; for first term we

use the fact a primitive nth root ω_n can be written as ω_{2n}^2 for some primitive $2n$th root ω_{2n}, and the second follows from the related fact the nth roots of -1 are the primitive $2n$th roots of unity. Simplifying, this gives $\omega_{2n}^{2ij+j} = \omega_{2n}^{(2i+1)\cdot j} = \omega_{2n}^{\mathbb{Z}_{2n}^*(i)\cdot j}$ as expected.

3.0.6 Sumcheck

Variations of the sumcheck protocol [**JACM:LFKN92**] have become central to construction of many of today's state-of-the-art IOPs. Here we consider sumcheck instances over a field $\mathbb{F}$ to be claims of the form (σ, g, v, d), where g is a v-variate polynomial of degree at most d in each variable, and σ is its claimed sum over the v-dimension hypercube:

$$\sigma = \sum_{x_1,\dots,x_v \in \{0,1\}^v} g(x_1, \dots, x_v)$$

The sumcheck protocol gives a reduction of this claim to checking the evaluation of g at a random point $g(\alpha_1, \dots, \alpha_v) = \sigma_v$. For self-containedness, we have included in appendix .1 an optimized sumcheck protocol (Figure 3) and a linear time and space prover algorithm for it (Figure 4) from [**BagadDaoDombThaler25**].

3.1 PIOPs over fields for cyclotomic ring arithmetic

We present a (multilinear) PIOP for proving matrix-vector products $As \equiv t$ in power-of-two cyclotomic rings $R_p = \mathbb{Z}_p[X]/(X^{2^\nu} + 1)$ with prime moduli p, where all messages and polynomial coefficients are in $\mathbb{K}$ for a sufficiently large extension field $\mathbb{K}/\mathbb{F}_p$. This is given in section 3.1.1. Subsequent sections present multiple variants and optimizations of this protocol, as we now describe.

Scope and restrictions. Our protocol requires three structural assumptions on the ring R_p: the modulus p is prime, the cyclotomic polynomial is $\Phi_{2n}(X) = X^n + 1$ with $n = 2^\nu$ a power of two, and p splits Φ_{2n} completely (i.e., $p \equiv 1 \pmod{2n}$, so that $X^n + 1$ factors into n distinct linear factors over $\mathbb{F}_p$). Each assumption serves a distinct purpose:

- *Prime modulus.* The sumcheck protocol, the polynomial commitment scheme, and the

extension-field arithmetic all require that we work over a field. A composite modulus q is not a field, so one cannot run the sumcheck or PCS directly over $\mathbb{Z}_q$. The standard workaround is to factor $q = \prod p_i$ and verify the relation modulo each prime factor independently; our protocol handles each such factor.

- *Fully splitting.* The CRT-domain approach requires that $X^n + 1$ admits n distinct roots $\omega, \omega^3, \dots, \omega^{2n-1}$ in $\mathbb{F}_p$, where ω is a primitive $2n$-th root of unity. This full splitting is what makes the ring product become a pointwise product after the CRT transform, yielding the key identity $t_C(i,k) = \sum_j A_C(i,j,k) \cdot s_C(j,k)$ that Sumcheck 1 verifies. When Φ_{2n} does not split completely over $\mathbb{F}_p$ (i.e., $p \not\equiv 1 \pmod{2n}$), the roots of $X^n + 1$ live in a proper extension of $\mathbb{F}_p$, and the CRT gives evaluation over that extension rather than over $\mathbb{F}_p$ itself; the protocol structure could in principle be adapted to this setting at the cost of increasing the base field size and the CRT matrix complexity, but we do not pursue this here.

- *Power-of-two degree.* This is primarily an efficiency assumption rather than a correctness requirement. The power-of-two structure $n = 2^\nu$ gives the CRT matrix $C_{k,\ell} = \omega^{(2k+1)\ell}$ a recursive dyadic periodicity: each factor $f_j(k) = (1 - \gamma_j) + \gamma_j \omega^{(2k+1)2^j}$ has period $2^{\nu-j}$ in k, which is what enables the $O(n)$-time verifier evaluation of $\tilde{C}$ in lemma 3.1.1. For a general cyclotomic $\Phi_m(X)$ of degree $\varphi(m)$, the roots do not exhibit this clean doubling structure, so the factored product-table construction would not apply and the verifier's $\tilde{C}$ evaluation would cost $O(\varphi(m)^2)$ in the worst case. The power-of-two case also aligns with the DP24 packing reduction (section 3.1.6), which uses a tower of quadratic extensions $\mathbb{F}_p \subset \mathbb{F}_{p^2} \subset \cdots \subset \mathbb{K}$ whose depth is $\log_2[\mathbb{K} : \mathbb{F}_p]$.

These restrictions are broadly compatible with deployed lattice-based cryptography: NIST standards (ML-KEM, ML-DSA) and most lattice-based signature schemes use power-of-two cyclotomic rings with primes chosen so that $p \equiv 1 \pmod{2n}$, and composite moduli decompose into such primes.

Why matrix-vector products. The relation $t = As$ over R_q is the common algebraic core of lattice-based cryptography: MLWE/RLWE encryption, Dilithium/Falcon/Hawk signatures, FHE

operation proofs, Ajtai hash commitments, and lattice-based VRFs all reduce — individually or in batches — to one or more instances of this relation. Matrix-matrix products $T = AS$ reduce to matrix-vector products via column batching: the verifier samples $\rho \leftarrow\$ \mathbb{K}^{M'}$ and the single relation $T\rho = A(S\rho)$ is checked, while multiple independent instances $As_i = t_i$ batch to a single inner product as described in section 3.1.6. We therefore present the protocol in full generality for $A \in R_q^{N\times M}$, then give optimized treatments for the inner product case $N = 1$ (which covers RSIS/SWIFFT, Falcon verification, and all batched multi-instance settings) and ring multiplication $N = M = 1$.

Beyond lattice cryptography, the matrix-vector relation over cyclotomic rings is a natural building block for proving more general NP statements that involve ring arithmetic. Any computation expressed as an intermediate representation over R_q — such as an arithmetic circuit whose gates are ring additions and multiplications, or a sequence of NTT-based operations — decomposes into a series of bilinear relations over R_q, each of which is an instance of matrix-vector multiplication (with $N = M = 1$ for scalar multiplication, $N = 1$ for inner products, and general N, M for linear maps). Because our protocol is a sumcheck-based PIOP, it composes naturally with other sumcheck protocols: claims produced by one sumcheck invocation can feed directly into another, enabling modular stacking without additional commitment overhead. The CRT-domain structure we exploit is also not specific to matrix-vector products; any relation that becomes pointwise after applying the CRT transform admits the same two-sumcheck architecture (a pointwise-check sumcheck followed by a CRT consistency sumcheck). While we do not pursue a fully general ring-arithmetic compiler in this work, the techniques developed here — CRT-domain verification, holographic indexing, packing, and oracle reduction — provide the essential ingredients for doing so.

For $n = 2^\nu$ and a $N \times M$ matrix A, our protocol has prover complexity $O(NMn)$[check this – Psi], verifier complexity $O(n + \log NM)$, and proof size $O(\log Mn)$. The additive $O(n)$ term in verifier complexity arises from a single evaluation of the CRT matrix's multilinear extension (lemma 3.1.1) and cannot be eliminated without fundamentally changing the protocol structure. An alternative "coefficient-domain" approach based on ring switching and a convolution sumcheck can in principle replace this $O(n)$ with $O(\log^2 n)$, but it produces cross-polynomial product constraints that are

incompatible with the foldable-code PCS and with DP24 packing (section 3.1.6), and we therefore do not pursue it. In practice, n is often small (e.g., $n = 64$ for SWIFFT) and the constant factor in the $O(n)$ evaluation is small, so it does not dominate concrete verifier performance. That said, in **??** we introduce a $O(\log^2 n + \log NM)$ verifier variant, with the tradeoff of it having $O(\log M + \log^2 n)$ proof size.

We write the main PIOP construction in section 3.1.1 in the "commit and prove" style, where the verifier begins with three oracles to polynomial representations of matrix A and vectors s and t (that we assume were earlier sent by the prover). When A is constant (e.g., proving knowledge of an MSIS opening) and/or t is public, we find ourselves in the holographic setting where an indexer can output variations of these oracles that allow for a more efficient argument. This is explained in section 3.1.2.

3.1.1 A PIOP for cyclotomic matrix-vector multiplication

Let $\omega \in \mathbb{F}_p$ be a primitive $2n$-th root of unity and let $\tilde{C} \in \mathbb{F}_p^{\mathsf{ml}}[\mathbf{K}, \mathbf{L}]$ denote the multilinear extension of the CRT matrix $C = \mathsf{CRT}_{n,p}$,

$$C_{k,\ell} = \omega^{(2k+1)\ell}, \qquad k, \ell \in \{0,1\}^\nu,$$

which maps coefficient vectors to evaluations at the roots of $X^n + 1$. Define virtual polynomials

$$\begin{aligned} A_C(i,j,k) &:= \textstyle\sum_{\ell \in \{0,1\}^\nu} \tilde{C}(k,\ell) \cdot \tilde{A}(i,j,\ell), \\ s_C(j,k) &:= \textstyle\sum_{\ell \in \{0,1\}^\nu} \tilde{C}(k,\ell) \cdot \tilde{s}(j,\ell), \\ t_C(i,k) &:= \textstyle\sum_{\ell \in \{0,1\}^\nu} \tilde{C}(k,\ell) \cdot \tilde{t}(i,\ell). \end{aligned}$$

When $t \equiv As \mod p$, the CRT transform gives the pointwise relation $t_C(i,k) = \sum_{j \in \{0,1\}^\mu} A_C(i,j,k) \cdot s_C(j,k)$ for all $(i,k) \in \{0,1\}^{\tau+\nu}$.

Figure 3.1: Protocol $\Pi_{\mathsf{RingMul}} = (\mathsf{P}, \mathsf{V})$: a PIOP for checking matrix-vector multiplication in a cyclotomic ring $R_p = \mathbb{Z}_p[X]/(X^{2^\nu} + 1)$ over $\mathbb{F}_p$

Input: The verifier receives oracles $[[\tilde{A}]] \in \mathbb{F}_p^{\mathsf{ml}}[\mathbf{I}, \mathbf{J}, \mathbf{K}]$, $[[\tilde{s}]] \in \mathbb{F}_p^{\mathsf{ml}}[\mathbf{J}, \mathbf{K}]$, and $[[\tilde{t}]] \in \mathbb{F}_p^{\mathsf{ml}}[\mathbf{I}, \mathbf{K}]$. The prover receives matrix $A \in R_p^{N \times M}$ and vectors $s \in R_p^M$, $t \in R_p^N$ such that

$$\tilde{A}(i, j, k) = \mathsf{cf}_k(A_{i,j}), \qquad \tilde{s}(j, k) = \mathsf{cf}_k(s_j), \qquad \tilde{t}(i, k) = \mathsf{cf}_k(t_i)$$

holds for $i \in \{0,1\}^\tau, j \in \{0,1\}^\mu, k \in \{0,1\}^\nu$, where $\tau = \log N$, $\mu = \log M$, $\nu = \log n$, and $n = 2^\nu$ is the degree of the cyclotomic polynomial.

1: The verifier samples $\alpha \leftarrow\$ \mathbb{K}^\tau$, $\alpha' \leftarrow\$ \mathbb{K}^\nu$.

2: The prover sends e_t claimed to equal $t_C(\alpha, \alpha')$.

3: P and V engage in a sumcheck for the claim

$$e_t = \sum_{j \in \{0,1\}^\mu} A_C(\alpha, j, \alpha') \cdot s_C(j, \alpha')$$

with challenges $\beta \leftarrow\$ \mathbb{K}^\mu$, reducing to the claim $\sigma' = A_C(\alpha, \beta, \alpha') \cdot s_C(\beta, \alpha')$.

4: The prover sends e_A claimed to equal $A_C(\alpha, \beta, \alpha')$. The verifier computes $e_s \leftarrow \sigma'/e_A$ and checks $e_A \neq 0$.[2]

5: The verifier samples batching challenge $\xi \leftarrow\$ \mathbb{K}$.

6: P and V engage in a sumcheck for the claim

$$e_t + \xi \cdot e_A + \xi^2 \cdot e_s = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(\alpha', \ell) \cdot \Big[\tilde{t}(\alpha, \ell) + \xi \cdot \tilde{A}(\alpha, \beta, \ell) + \xi^2 \cdot \tilde{s}(\beta, \ell)\Big]$$

with folding challenges $\gamma \leftarrow\$ \mathbb{K}^\nu$, reducing to the claim

$$\sigma'' = \tilde{C}(\alpha', \gamma) \cdot \Big[\tilde{t}(\alpha, \gamma) + \xi \cdot \tilde{A}(\alpha, \beta, \gamma) + \xi^2 \cdot \tilde{s}(\beta, \gamma)\Big].$$

7: The verifier evaluates $\tilde{C}(\alpha', \gamma)$ in $O(n)$ field operations (lemma 3.1.1), queries $e'_t \leftarrow [[\tilde{t}(\alpha, \gamma)]]$, $e'_A \leftarrow [[\tilde{A}(\alpha, \beta, \gamma)]]$, $e'_s \leftarrow [[\tilde{s}(\beta, \gamma)]]$, and checks

$$\sigma'' = \tilde{C}(\alpha', \gamma) \cdot (e'_t + \xi \cdot e'_A + \xi^2 \cdot e'_s).$$

Lemma 3.1.1. *Let $\omega \in \mathbb{F}_p$ be a primitive $2n$-th root of unity with $n = 2^\nu$, and let $\tilde{C} \in \mathbb{F}_p^{\mathsf{ml}}[\mathbf{K}, \mathbf{L}]$ denote the multilinear extension of $C = \mathsf{CRT}_{n,p}$, given by $C_{k,\ell} = \omega^{(2k+1)\ell}$ for $k, \ell \in \{0,1\}^\nu$. Then for any $\alpha', \gamma \in \mathbb{K}^\nu$, the evaluation $\tilde{C}(\alpha', \gamma)$ can be computed in $O(n)$ field operations.*

Proof. We first derive a factored representation by separating the sum over ℓ. Since $\omega^{(2k+1)\ell} =$

$\prod_{j=0}^{\nu-1} \omega^{(2k+1)\ell_j 2^j}$ and $\widetilde{\mathsf{eq}}(\gamma, \ell) = \prod_j((1-\gamma_j)(1-\ell_j) + \gamma_j \ell_j)$, the inner sum over ℓ factors across coordinates:

$$\begin{aligned}\sum_{\ell \in \{0,1\}^\nu} \omega^{(2k+1)\ell}\, \widetilde{\mathsf{eq}}(\gamma, \ell) &= \prod_{j=0}^{\nu-1} \sum_{\ell_j \in \{0,1\}} \omega^{(2k+1)\ell_j 2^j}\, ((1-\gamma_j)(1-\ell_j) + \gamma_j \ell_j) \\ &= \prod_{j=0}^{\nu-1} \Big[(1-\gamma_j) + \gamma_j\, \omega^{(2k+1)2^j}\Big]. \end{aligned} \tag{3.2}$$

Define the univariate polynomial

$$g(X) \;:=\; \prod_{j=0}^{\nu-1} \Big[(1-\gamma_j) + \gamma_j\, X^{2^j}\Big],$$

which has degree $\sum_{j=0}^{\nu-1} 2^j = n-1$. By (3.2), $g(\omega^{2k+1}) = \prod_j[(1-\gamma_j) + \gamma_j\, \omega^{(2k+1)2^j}]$, so

$$\tilde{C}(\alpha', \gamma) \;=\; \sum_{k \in \{0,1\}^\nu} \widetilde{\mathsf{eq}}(\alpha', k) \cdot g(\omega^{2k+1}). \tag{3.3}$$

To evaluate the right-hand side in $O(n)$ time, we exploit the periodicity of each factor of g. Write $g(\omega^{2k+1}) = \prod_{j=0}^{\nu-1} f_j(k)$ where

$$f_j(k) \;:=\; (1-\gamma_j) + \gamma_j\, \omega^{(2k+1)2^j}.$$

Since ω has order $2n = 2^{\nu+1}$, the element $\omega^{2^{j+1}}$ has order $2^{\nu-j}$, and therefore $\omega^{(2k+1)2^j} = \omega^{2^j} \cdot (\omega^{2^{j+1}})^k$ is periodic in k with period $2^{\nu-j}$. In particular:

- $f_{\nu-1}$ has period 2 in k ($\omega^{2^\nu} = -1$, so $f_{\nu-1}(k)$ depends only on k mod 2),
- $f_{\nu-2}$ has period 4, ..., f_0 has period n.

We compute the product table $[g(\omega^{2k+1})]_{k=0}^{n-1}$ iteratively. Initialize with the partial product $P_{\nu-1}(k) := f_{\nu-1}(k)$ for $k \in \{0,1\}$, at a cost of $O(1)$. For $s = \nu-2, \nu-3, \ldots, 0$: extend P_{s+1} from its period $2^{\nu-s-1}$ to period $2^{\nu-s}$ by replication, and compute $P_s(k) := P_{s+1}(k) \cdot f_s(k)$ for

$k = 0, \dots, 2^{\nu-s} - 1$. The cost at stage s is $O(2^{\nu-s})$ multiplications, so the total is

$$\sum_{s=0}^{\nu-1} 2^{\nu-s} \;=\; 2 + 4 + \cdots + 2^{\nu} \;=\; 2(2^{\nu} - 1) \;=\; O(n).$$

After the final stage, $P_0(k) = g(\omega^{2k+1})$ for all $k \in \{0, \dots, n-1\}$. The table $[\widetilde{\mathsf{eq}}(\alpha', k)]_{k \in \{0,1\}^{\nu}}$ is computed in $O(n)$ by the standard recursive formula. The evaluation (3.3) is then an inner product of two length-n vectors, costing $O(n)$. □

3.1.2 A holographic variant

When the matrix A is public (e.g., given by the structure of a hash function such as SWIFFT, or by a lattice scheme's public key), the oracle $[[\tilde{A}]]$ need not be sent by the prover: it can be provided by an indexer in a preprocessing phase. We now describe how to exploit this by committing directly to the CRT-domain representation A_C, which eliminates the CRT consistency check for A from Sumcheck 2 entirely.

Indexed CRT oracle. Recall that $A_C(i, j, k) = \sum_{\ell \in \{0,1\}^{\nu}} \tilde{C}(k, \ell) \cdot \tilde{A}(i, j, \ell)$. Since A is public, the indexer can compute A_C explicitly and provide the verifier with an oracle $[[A_C]] \in \mathbb{K}^{\mathsf{ml}}[\mathbf{I}, \mathbf{J}, \mathbf{K}]$ in a preprocessing phase. The crucial observation is that the verifier can now query $A_C(\alpha, \beta, \alpha')$ directly at Step 4 of Figure 3.1, obtaining e_A without relying on the prover. This removes $\tilde{A}$ from the batched sum in Sumcheck 2, which becomes

$$e_t + \xi \cdot e_s = \sum_{\ell \in \{0,1\}^{\nu}} \tilde{C}(\alpha', \ell) \cdot \Big[\tilde{t}(\alpha, \ell) + \xi \cdot \tilde{s}(\beta, \ell)\Big],$$

with folding challenges $\gamma \leftarrow\$ \mathbb{K}^{\nu}$ reducing to the final claim

$$\sigma'' = \tilde{C}(\alpha', \gamma) \cdot (e'_t + \xi \cdot e'_s),$$

verified by querying $e'_t \leftarrow [[\tilde{t}(\alpha, \gamma)]]$ and $e'_s \leftarrow [[\tilde{s}(\beta, \gamma)]]$. The oracle $[[\tilde{A}]]$ no longer appears.

Additional simplification when t is public. In many lattice-based settings, t is also public — for instance, in Module-SIS where $t = 0$, or in Module-LWE where t is the public ciphertext. In this case, an indexer can similarly provide $[[t_C]]$, and the verifier obtains e_t by querying $t_C(\alpha, \alpha')$ directly. Sumcheck 2 then reduces to a CRT consistency check on $\tilde{s}$ alone:

$$e_s = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(\alpha', \ell) \cdot \tilde{s}(\beta, \ell).$$

This is the minimal form of the protocol: a single prover-committed oracle $[[\tilde{s}]]$, two sumchecks, and one PCS evaluation.

Prover and indexer preprocessing. The indexer computes A_C by applying the CRT transform (an n-point NTT at the odd powers of ω) to each of the NM entries of A, for a total cost of $O(NMn \log n)$ field operations, or $O(NMn)$ using a batched radix-2 NTT. If t is also indexed, the same applies at cost $O(Nn \log n)$. On the prover side, when A is reused across multiple protocol invocations with the same public key (e.g., batch signature verification), the prover can cache the table of evaluations $\{A_C(i, j, k)\}_{i,j,k \in \{0,1\}^{\tau+\mu+\nu}}$ and reuse it across invocations, amortizing the NTT cost. Concretely, the prover's online cost for computing Sumcheck 1 round polynomials is dominated by the $O(NMn)$ summation, which does not change; the savings come from avoiding redundant CRT transforms of A.

3.1.3 Optimized inner product variant

When $N = 1$, the matrix–vector product $As = t$ reduces to a single inner product $\langle a, s \rangle = t$ in R_p, where $a \in R_p^M$ and $t \in R_p$ is a single ring element. This is the relevant setting for SWIFFT hash verification and for proving knowledge of a Module-SIS preimage with a single row. We describe how Figure 3.1 simplifies in this case.

Structural simplification. Setting $N = 1$ gives $\tau = \log N = 0$, so the variable block $\mathbf{I}$ is empty. The polynomial $\tilde{A}(j, \ell)$ has $\mu + \nu$ variables, $\tilde{s}(j, \ell)$ has $\mu + \nu$ variables, and $\tilde{t}(\ell)$ has ν variables. The

CRT-domain virtual polynomials become

$$A_C(j,k) = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(k,\ell) \cdot \tilde{A}(j,\ell), \qquad s_C(j,k) = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(k,\ell) \cdot \tilde{s}(j,\ell), \qquad t_C(k) = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(k,\ell) \cdot \tilde{t}(\ell).$$

Sumcheck 1 runs over $\mathbf{J}$ only (μ rounds), proving

$$e_t = \sum_{j \in \{0,1\}^\mu} A_C(j,\alpha') \cdot s_C(j,\alpha')$$

for verifier challenges $\alpha' \leftarrow\$ \mathbb{K}^\nu$, and produces the reduced claim $\sigma' = A_C(\beta,\alpha') \cdot s_C(\beta,\alpha')$ with $\beta \leftarrow\$ \mathbb{K}^\mu$.

Holographic A, public t. In the common case where A is public and t is public (or $t = 0$ as in SIS), the protocol reaches its simplest form. The indexer provides $[[A_C]]$, and the verifier computes $e_t = t_C(\alpha')$ in $O(n)$ time via the factored evaluation of lemma 3.1.1 applied to the known coefficients of t. Concretely, the verifier evaluates

$$e_t = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(\alpha',\ell) \cdot \tilde{t}(\ell) = \sum_{k \in \{0,1\}^\nu} \widetilde{\mathsf{eq}}(\alpha',k) \cdot t_C(k)$$

by first computing the CRT of t (an n-point NTT, or looked up from the indexed preprocessing) and then taking the $\widetilde{\mathsf{eq}}$-weighted inner product with α'.

The protocol now involves a single prover oracle $[[\tilde{s}]] \in \mathbb{K}^{\mathsf{ml}}[\mathbf{J},\mathbf{L}]$ of dimension $\mu + \nu$:

1. The verifier samples $\alpha' \leftarrow\$ \mathbb{K}^\nu$ and computes $e_t = t_C(\alpha')$.
2. Sumcheck 1 (μ rounds): proves $e_t = \sum_{j \in \{0,1\}^\mu} A_C(j,\alpha') \cdot s_C(j,\alpha')$, yielding $\sigma' = A_C(\beta,\alpha') \cdot s_C(\beta,\alpha')$.
3. The verifier queries $e_A \leftarrow [[A_C(\beta,\alpha')]]$ and computes $e_s \leftarrow \sigma'/e_A$.
4. Sumcheck 2 (ν rounds): proves $e_s = \sum_{\ell \in \{0,1\}^\nu} \tilde{C}(\alpha',\ell) \cdot \tilde{s}(\beta,\ell)$, yielding $\sigma'' = \tilde{C}(\alpha',\gamma) \cdot \tilde{s}(\beta,\gamma)$.
5. The verifier queries $e'_s \leftarrow [[\tilde{s}(\beta,\gamma)]]$ and checks $\sigma'' = \tilde{C}(\alpha',\gamma) \cdot e'_s$.

The total proof consists of $\mu + \nu$ sumcheck rounds and a single PCS evaluation of $\tilde{s}$ at the point $(\beta, \gamma) \in \mathbb{K}^{\mu+\nu}$. This clean structure — one oracle, one evaluation claim — composes particularly well with DP24 packing (section 3.1.6), since there is no need for commitment merging or IOR reductions before invoking the PCS.

3.1.4 Reducing the number of commitments

When the prover commits to multiple oracles, each oracle incurs a Merkle tree and associated PCS opening costs. We describe a standard selector-polynomial technique that merges the oracles for $\tilde{s}$ and $\tilde{t}$ into a single oracle h', and analyze when this merge is beneficial in conjunction with the IOR and packing optimizations of later sections.

The selector polynomial. Define $h' \in \mathbb{K}^{\mathsf{ml}}[B, \mathbf{X}, \mathbf{L}]$ on $1 + \max(\tau, \mu) + \nu$ variables by

$$h'(b, x, \ell) \;:=\; (1-b) \cdot \tilde{s}(x_{1:\mu}, \ell) \;+\; b \cdot \tilde{t}(x_{1:\tau}, \ell),$$

where $x_{1:r}$ denotes the first r coordinates of x. When $\tau < \mu$, the polynomial $\tilde{t}(x_{1:\tau}, \ell)$ does not depend on coordinates $x_{\tau+1}, \dots, x_\mu$, so h' is well-defined and multilinear. By construction, $h'(0, x, \ell) = \tilde{s}(x_{1:\mu}, \ell)$ and $h'(1, x, \ell) = \tilde{t}(x_{1:\tau}, \ell)$, so a single committed oracle $[[h']]$ replaces both $[[\tilde{s}]]$ and $[[\tilde{t}]]$.

After Sumcheck 2 (with holographic A), the verifier needs the evaluations $\tilde{s}(\beta, \gamma)$ and $\tilde{t}(\alpha, \gamma)$, which correspond to

$$\begin{aligned} h'(0,\ \beta,\ \gamma) &= \tilde{s}(\beta, \gamma), \\ h'(1,\ (\alpha, \beta_{\tau+1:\mu}),\ \gamma) &= \tilde{t}(\alpha, \gamma), \end{aligned}$$

where we pad $\alpha \in \mathbb{K}^\tau$ with the coordinates $\beta_{\tau+1}, \dots, \beta_\mu$ (or any fixed values) to obtain a point of length μ. This gives two evaluation claims on h' at points that differ in the first coordinate ($b = 0$ vs $b = 1$) and possibly in coordinates 1 through τ ($\beta_{1:\tau}$ vs α), but agree on coordinates $\tau + 1$ through $\mu + \nu$ (both use $\beta_{\tau+1:\mu}$ and γ). These two claims are reduced to a single evaluation via the IOR of section 3.1.5.

When to use h'. The merge is not always beneficial. Without h', the prover commits to two oracles: $\tilde{s}$ of dimension $\mu+\nu$ and $\tilde{t}$ of dimension $\tau+\nu$. With h', a single oracle of dimension $1+\mu+\nu$ (assuming $\mu \geq \tau$) replaces both, but the IOR adds a small number of extra field elements to the transcript (see section 3.1.5). The following guidelines cover the main cases:

- $N = 1$, *t public (e.g., SIS).* Only $\tilde{s}$ is committed. Do not use h'.
- $N = 1$, *t private.* Here $\tilde{t}$ has only ν variables while $\tilde{s}$ has $\mu+\nu$. The merge produces an oracle of dimension $1+\mu+\nu$, saving one Merkle tree at the cost of one extra PCS round. This is beneficial when $\mu \geq 3$, which holds in all practical MSIS parameter sets. When $\mu \leq 2$, the overhead of the extra variable and IOR outweighs the Merkle tree savings; committing separately is preferable.
- *General N, M with $\tau \leq \mu$.* The merge saves the $\tilde{t}$ commitment entirely (one fewer Merkle tree root and PCS opening proof), at the cost of one additional variable in the merged oracle. The net savings in proof size scale with $\min(\tau, \mu) + \nu$: roughly, eliminating the $\tilde{t}$ PCS proof saves $\approx 3(\tau+\nu)$ field elements from PCS folding, while the extra variable adds ≈ 3 field elements. Thus h' is beneficial whenever $\tau+\nu \geq 2$, which is the generic case.
- $\tau = \mu$ *(square matrices).* The two oracles have equal dimension. The merge is always worthwhile, saving one full PCS opening at the cost of one extra round.

3.1.5 Interactive oracle reductions

After the two sumchecks and any holographic oracle queries, the verifier holds one or more evaluation claims on the prover's committed polynomial(s). Before invoking the PCS, we apply interactive oracle reductions (IORs) to consolidate these claims. We follow the framework of Baweja, Chiesa, and Spooner [**BCS24**][fix cite key – Psi], who give IORs for linear-time-encodable codes in the IOPP model; we describe the two reductions we use and verify their compatibility with random foldable codes (in the BaseFold setting) and with DP24 packing.

Multi-point to single-point IOR. Given two evaluation claims $h'(z_0) = v_0$ and $h'(z_1) = v_1$ on the same multilinear polynomial h' of dimension d, the standard reduction proceeds as follows. The verifier sends a challenge $r \leftarrow\$ \mathbb{K}$ and both parties compute the convex combination $z^* = (1-r) \cdot z_0 + r \cdot z_1$ and $v^* = (1-r) \cdot v_0 + r \cdot v_1$. Since h' is multilinear, the univariate restriction $g(t) := h'((1-t) \cdot z_0 + t \cdot z_1)$ has degree at most d in t, and satisfies $g(0) = v_0$, $g(1) = v_1$. The prover sends the coefficients of g (at most $d-1$ additional field elements, since $g(0)$ and $g(1)$ are known), and the verifier checks consistency and accepts the reduced claim $h'(z^*) = g(r)$.

In the special case arising from the h' construction of section 3.1.4, the two evaluation points $z_0 = (0, \beta, \gamma)$ and $z_1 = (1, (\alpha, \beta_{\tau+1:\mu}), \gamma)$ differ in at most $1 + \tau$ coordinates (the selector bit and the first τ entries of $\mathbf{X}$). When $\tau = 0$ (the $N = 1$ case), the points differ in the first coordinate only, and the restriction g is linear: $g(t) = (1-t)v_0 + t \cdot v_1$, so the prover sends nothing and the verifier simply evaluates $g(r)$. The IOR is free.

For general τ, the prover sends τ field elements (the intermediate coefficients of g), and the reduced claim is $h'(z^*) = g(r)$ at a single point $z^* \in \mathbb{K}^{1+\mu+\nu}$.

Multi-polynomial to single-polynomial IOR. When h' is not used and the prover commits to $\tilde{s}$ and $\tilde{t}$ separately, the verifier holds evaluation claims on distinct polynomials at distinct points: $\tilde{s}(\beta, \gamma) = e'_s$ and $\tilde{t}(\alpha, \gamma) = e'_t$. To reduce these to a single PCS invocation, we first pad both polynomials to a common dimension $d^* = \max(\tau, \mu) + \nu$ by treating the lower-dimensional polynomial as constant in the extra variables. The verifier sends a batching challenge $\zeta \leftarrow\$ \mathbb{K}$ and defines the combined polynomial $f = \tilde{s} + \zeta \cdot \tilde{t}_{\text{pad}}$. The two evaluation claims become a single claim on f at a single point (obtained by a further multi-point reduction if the evaluation points differ). The prover commits to f implicitly by providing the combined codeword; the PCS then opens f at the required point.

When additionally A is not holographic and $[[\tilde{A}]]$ is prover-committed, its evaluation claim $\tilde{A}(\alpha, \beta, \gamma) = e'_A$ is absorbed into the same batching: $f = \tilde{s} + \zeta \cdot \tilde{t}_{\text{pad}} + \zeta^2 \cdot \tilde{A}$.

Compatibility with random foldable codes and DP24. Both IOR reductions operate at the PIOP level, producing a single evaluation claim before the PCS is invoked. The BaseFold

PCS then proves this claim using the foldable code's proximity test. The critical property is that the IOR does not impose structural requirements on the code beyond the ability to evaluate the committed polynomial at an arbitrary extension-field point, which random foldable codes support by construction (the encoding is $\mathbb{F}_p$-linear, and evaluation at $\mathbb{K}$-points is obtained by $\mathbb{K}$-linear extension of the codeword).

DP24 packing (section 3.1.6) operates *after* the IOR: the single evaluation claim $f(z^*) = v^*$ on a polynomial of dimension d^* is reduced by the packing sumcheck to a claim on a packed polynomial $\hat{f}$ of dimension $d^* - s$, where $s = \log_2 \kappa$. The IOR's output is a standard evaluation claim, which is exactly the input format that the packing reduction expects. There is no interaction between the IOR mechanics and the packing sumcheck's tower-based decomposition, so compatibility is immediate.

3.1.6 DP24 packing

We apply the packing technique of Diamond and Posen [**DP24**][fix cite key – Psi] to reduce the PCS oracle dimension by $s = \log_2 \kappa$ variables, where $\kappa = [\mathbb{K} : \mathbb{F}_p]$ is the extension degree. This reduces the depth of the Merkle tree and the number of PCS folding rounds, yielding significant concrete proof-size savings. We describe the mechanism, its dependencies, and — crucially — why the CRT consistency check (Sumcheck 2) must remain an explicit sumcheck rather than being absorbed into the PCS when packing is in use.

Packing overview. Let $f \in \mathbb{K}^{\mathsf{ml}}[X_1, \dots, X_R]$ be the multilinear polynomial to be opened at a point $z \in \mathbb{K}^R$ (this is the single evaluation claim produced by the IOR of section 3.1.5). The DP24 packing partitions the R variables into s "packed" variables $X_1, \dots, X_s$ and $R - s$ "free" variables $X_{s+1}, \dots, X_R$. Fixing a $\mathbb{F}_p$-basis $\{e_\rho\}_{\rho=1}^{\kappa}$ of $\mathbb{K}$ with dual basis $\{e_\rho^*\}$, define the packed polynomial $\hat{f} \in \mathbb{K}^{\mathsf{ml}}[X_{s+1}, \dots, X_R]$ by

$$\hat{f}(x_{s+1}, \dots, x_R) := \sum_{b \in \{0,1\}^s} \widetilde{\mathsf{eq}}(z_{1:s}, b) \cdot f(b, x_{s+1}, \dots, x_R).$$

The evaluation $f(z) = \hat{f}(z_{s+1:R})$ follows by multilinearity. The prover commits to $\hat{f}$ (an oracle of dimension $R - s$) and proves $\hat{f}(z_{s+1:R}) = f(z)$ via the BaseFold PCS using a random foldable code over $\mathbb{K}$.

The packing reduction itself is an s-round sumcheck that verifies the relationship between f and $\hat{f}$. Each round "lifts" one tower level, exploiting the tower of quadratic extensions $\mathbb{F}_p \subset \mathbb{F}_{p^2} \subset \cdots \subset \mathbb{F}_{p^\kappa} = \mathbb{K}$: at level i, the sumcheck challenge lives in $\mathbb{F}_{p^{2^i}}$, and the round polynomial has degree 1 in the current variable. After s rounds, the claim reduces to a single evaluation of $\hat{f}$ at an extension-field point, which the BaseFold PCS proves.

Dependencies. The packing reduction requires:

- A *tower of quadratic extensions* $\mathbb{F}_p \subset \mathbb{F}_{p^2} \subset \cdots \subset \mathbb{K}$ with explicit embedding and projection maps. For odd primes p, we construct such a tower in **??**[fix cref – Psi].
- *Linear-time encodable random foldable codes* over each tower level $\mathbb{F}_{p^{2^i}}$, $i = 0, \ldots, s$. We describe our construction based on [**BaseFold**][fix cite – Psi] in **??**[fix cref – Psi].

Transcript costs. The s-round packing sumcheck contributes $3s$ field elements to the PIOP transcript (one degree-1 univariate per round, costing 2 coefficients, plus 1 claimed evaluation — but since the constant term is determined by the claim, each round sends 1 new field element; the total is $2s$[double-check this count against DP24 – Psi]). The BaseFold PCS then runs $R - s$ folding rounds instead of R, saving $3s$ field elements from PCS round messages. The net transcript-size change is approximately zero, but the Merkle tree depth decreases by s, which saves $s \cdot \lambda$ bits of hash proof per query (where λ is the hash output length). For $s = 4$ and $\lambda = 256$, this is a saving of 128 bytes per query repetition.

Incompatibility of CRT absorption with packing. A natural question is whether Sumcheck 2 (the CRT consistency check) can be absorbed into the PCS as a weighted-sum constraint, eliminating it as an explicit sumcheck and saving $\sim 3\nu$ field elements. We now show that this absorption is *incompatible* with DP24 packing, and therefore Sumcheck 2 must remain explicit whenever packing

is used.

Proposition 3.1.2. *Let $\hat{f}$ be the packed polynomial obtained from f via the DP24 reduction with packing parameter $s \geq 1$. Let $\varphi_\rho : \mathbb{K} \to \mathbb{K}$ denote the $\mathbb{F}_p$-linear dual-basis extraction map $\varphi_\rho(x) = \langle e^*_\rho, x \rangle_{\mathbb{F}_p}$ (i.e., φ_ρ extracts the ρ-th $\mathbb{F}_p$-coordinate of x). If the CRT consistency check is absorbed into the PCS, the resulting constraint on $\hat{f}$ takes the form*

$$\sum_y w(y, \rho) \cdot \varphi_\rho(\hat{f}(y)) = \sigma_\rho$$

for verifier-computable weights $w(\cdot)$ and each basis index ρ. This constraint is $\mathbb{F}_p$-linear but not $\mathbb{K}$-linear in the evaluations of $\hat{f}$.

The BaseFold folding step replaces $\hat{f}$ with $\hat{f}_{\text{new}}(y') = \hat{f}(0, y') + r \cdot \hat{f}(1, y')$ for a challenge $r \in \mathbb{K}$. For the constraint to be maintained through folding, we would need

$$\varphi_\rho(\hat{f}(0, y') + r \cdot \hat{f}(1, y')) = \varphi_\rho(\hat{f}(0, y')) + r \cdot \varphi_\rho(\hat{f}(1, y'))$$

for all $r \in \mathbb{K}$ and all evaluations. But φ_ρ is $\mathbb{F}_p$-linear, not $\mathbb{K}$-linear: for general $r \in \mathbb{K}$ $\mathbb{F}_p$, $\varphi_\rho(r \cdot x) \neq r \cdot \varphi_\rho(x)$. Hence the absorbed constraint cannot be tracked through $\mathbb{K}$-challenge folding.

Proof. It suffices to exhibit a concrete failure. Let p be an odd prime and $\mathbb{K} = \mathbb{F}_{p^2}$ with basis $\{1, \theta\}$ where $\theta^2 = g$ for a non-square $g \in \mathbb{F}_p$. Then $\varphi_1(a + b\theta) = a$ and $\varphi_2(a + b\theta) = b$. Take $r = \theta$ and $x = 1$. Then $\varphi_1(\theta \cdot 1) = \varphi_1(\theta) = 0$, but $\theta \cdot \varphi_1(1) = \theta \cdot 1 = \theta \neq 0$. The constraint breaks at the first folding step. □

By contrast, standard evaluation claims $\hat{f}(z) = v$ are trivially compatible with packing: the verifier holds v and applies φ_ρ post-hoc if needed, without requiring the PCS to track an $\mathbb{F}_p$-linear constraint through its folds.

Correct protocol architecture. The incompatibility above dictates the following structure when all optimizations are combined:

1. **Sumcheck 1** (μ rounds, or $\tau + \mu$ rounds if $\mathbf{I}$ is nonempty): the matrix-vector product check in the CRT domain, producing evaluation claims on A_C, $\tilde{s}$, and $\tilde{t}$.

2. **Sumcheck 2** (ν rounds): the explicit CRT consistency check, reducing CRT-domain claims to coefficient-domain evaluation claims on the committed oracles. When A is holographic, only $\tilde{s}$ and $\tilde{t}$ (or $\tilde{s}$ alone, if t is also public) participate.

3. **IOR reduction** (section 3.1.5): consolidates the resulting evaluation claims into a single claim $f(z) = v$ on a single polynomial of dimension d^*. When the h' merge is used, this is a multi-point IOR; otherwise, a multi-polynomial IOR.

4. **DP24 packing sumcheck** (s rounds): reduces the claim to $\hat{f}(z') = v$ on a packed polynomial of dimension $d^* - s$.

5. **BaseFold PCS** ($d^* - s$ folding rounds): proves the evaluation of $\hat{f}$ using a random foldable code over $\mathbb{K}$, with linear-time encoding (**??**[fix cref – Psi]).

The total PIOP transcript size is $3(\mu + \nu) + O(\tau) + 2s + 3(d^* - s)$ field elements, plus the IOR messages. The prover complexity is $O(NMn)$ for the sumchecks plus $O(2^{d^*})$ for the PCS encoding; the verifier complexity is $O(n + \log NM)$ field operations.

Batch verification across MSIS instances. A common setting in lattice-based cryptography is the need to verify k independent Module-SIS relations $As_i = t_i$ for $i = 1, \dots, k$ with a shared public matrix A. Since the indexed oracle $[[A_C]]$ is the same across instances, the indexing cost is paid once. The verifier batches the k instances by sampling $\eta_1, \dots, \eta_k \leftarrow\$ \mathbb{K}$ and checking the combined relation

$$A\Big(\sum_{i=1}^{k} \eta_i s_i\Big) = \sum_{i=1}^{k} \eta_i t_i.$$

This reduces k protocol executions to a single execution on the combined witness $s^* = \sum_i \eta_i s_i$ and target $t^* = \sum_i \eta_i t_i$, at the cost of the prover computing the random linear combination of witness vectors. If each s_i has dimension M, the combination costs $O(kMn)$ ring-element operations. The PCS can additionally batch the k oracle openings using a single multi-polynomial IOR (treating

each $\tilde{s}_i$ as a separate polynomial), or the prover can commit to $\tilde{s}^*$ directly if the combination is performed before commitment. The latter is preferable when k is known in advance (e.g., batch signature verification), as it requires a single oracle of the same dimension as each individual $\tilde{s}_i$.

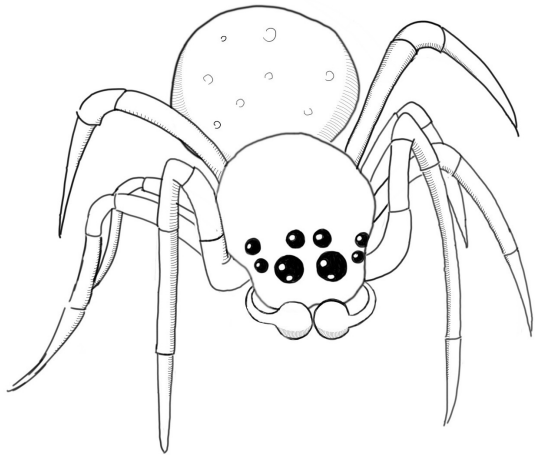
Chapter 4

Orbweaver: Succinct Linear Functional Commitments from Lattices

4.1 Overview

Over the last decade there has been tremendous progress in the development succinct non-interactive argument of knowledge protocols, called SNARKs [**ITCS:BCCT12**]. For a public arithmetic circuit C and a public input x, these SNARKs enable a prover to convince a verifier that the prover knows a witness w such that $C(\mathsf{x}, \mathsf{w}) = 1$, where:

- the proof is short, its length is at most $\mathrm{poly}(\log|C|,\ |\mathsf{x}|,\ \lambda)$;
- the proof can be verified in time $\mathrm{poly}(\log|C|,\ |\mathsf{x}|,\ \lambda)$; and
- generating the proofs takes quasilinear time in $|C|$.



Lattice Orbweaver (Araneus thaddeus) by Jackie P (CC BY 4.0)

Here $|C|$ is the number of gates in C, and λ is a security parameter. The logarithmic dependence on $|C|$ makes these proofs remarkably short and fast to verify. Since the verifier needs to at least read C, there is also a one-time pre-processing phase applied to the circuit C. Once pre-processed, the prover can provide proofs for many x.

Beyond knowledge of a witness, many of these proof systems can be efficiently extended to also provide zero-knowledge. This progress generated considerable real-world interest, and there are now implemented SNARKs and zkSNARKs capable of handling statements involving many millions of arithmetic gates that are now deployed in real-world applications (e.g., [**SP:BCGGMT14**]).

There are multiple techniques for constructing a pre-processing SNARK. Some require a trusted setup [**AC:Groth10a**; **TCC:Lipmaa12**; **STOC:BCCT13**; **EC:GGPR13**; **SP:PHGR13**; **TCC:BCIOP13**; **EC:Groth16**; **C:GroMal17**; **CCS:MBKM19**; **EPRINT:GabWilCio19**; **EC:CHMMVW20**] where a trusted party must honestly generate public parameters. This setup is called *universal* if it is not specific to the circuit C, i.e., it can be done once to generate parameters that can be used to preprocess any circuit C in a publicly verifiable way. Other systems, called transparent SNARKs, require no trusted setup [**C:BBHR19**; **EPRINT:BunFisSze19**; **EPRINT:ChiOjhSpo19**; **EPRINT:BowGriHop19**].

Many of the existing SNARKs make use of a commitment scheme called a *polynomial commitment* [**AC:KatZavGol10**], or PC. A polynomial commitment enables a prover to commit to a polynomial $f \in \mathbb{F}[x]$ of degree w using a short commitment. Later, given two public values $\alpha, \beta \in \mathbb{F}$, the prover can convince a verifier that the committed polynomial f satisfies $\beta = f(\alpha)$ and that f has degree at most w. Ideally, the prover can verifiably open the polynomial at any point $\alpha \in \mathbb{F}$ using a short proof that can be efficiently checked. In fact, it has been shown that given polynomial commitment scheme with proof size $S(w)$ and verification time $T(w)$ it is possible to construct SNARKs with the same complexity characteristics in w (the length of the witness), and additional complexity dependent only on a security parameter λ [**EC:BunFisSze20**; **EC:CHMMVW20**]. This compilation is in the random oracle model and relies on the Fiat-Shamir transform.

Polynomial commitment schemes are a special case of *linear functional vector commitments*, where the prover has a commitment $C = \mathsf{com}(\mathbf{x})$ to a vector $\mathbf{x} \in \mathbb{Z}_p^w$ and is able to open any

[0]Named for the lattice orbweaver spider (araneus thaddeus).

linear form $f(\mathbf{x}) = \sum_{i=1}^{w} \mathbf{x}_i f_i \bmod p$. Polynomial commitments, and the more general linear vector commitments, have been built from bilinear pairings [**AC:KatZavGol10**], groups of unknown order [**EPRINT:BunFisSze19**], proofs of proximity for Reed-Solomon codes [**ICALP:BBHR18**], and to some degree from lattice-based assumptions [**C:ACLMT22**]. However, thus far all lattice-based constructions have had significant drawbacks, and none have achieved asymptotically (in the polynomial's degree) a quasilinear prover time with logarithmic proof size and logarithmic verification time. There are lattice-based generalizations of Bulletproofs [**EC:BCCGP16**; **SP:BBBPWM18**] based on the Ring-SIS assumption, which achieve quasilinear prover time and a polylogarithmic proof size, but have linear verification time [**C:BLNS20**; **C:AttCraKoh21**; **C:AlbLai21**]. Recently, [**C:ACLMT22**] constructed general degree-d polynomial map vector commitments (which include linear functions $d = 1$ as a special case), which achieve logarithmic proof size and verification time, but have a CRS size and prover runtime that is at least $\Omega(w^{2d})$, and thus quadratic in the vector length for linear functions. This construction requires a trusted setup using lattice trapdoor sampling [**EC:MicPei12**], and is based on k-R-ISIS, a new family of lattice-based knowledge assumptions related to Ring-SIS. Another recent work, LaBRADOR [**BS22**], uses recursion to achieve very compact proof sizes, but has verification time linear in w.

Polynomial commitment schemes based on codes and lattice assumptions are of particular importance due to their plausible post-quantum security. Constructions based on Reed-Solomon codes have quasilinear prover time and both polylogarithmic proof size and verifier time. So far they outperform any lattice-based construction not only asympotically, but also concretely by orders of magnitude in overall size and verification time, even when compared with recent lattice-based constructions that sacrifice prover performance for shorter proofs. Moreover, in the random oracle model, code-based constructions use weaker assumptions than lattice-based constructions. Nonetheless, we are optimistic that the additional structure lattices provide vs. generic (i.e., hash and code-based) constructions can be exploited such that lattice SNARKs eventually surpass these results. As a point of reference, hash-based signatures were originally more efficient, but after over a decade of development lattice-based signatures are an order of magnitude smaller and two faster.

A primary motivation of recent lattice-based constructions [**C:ACLMT22**; **WW22**] of vector commitments supporting higher degree polynomial map openings is that, unlike linear function commitments, they can be used to build SNARKs more directly (e.g., by opening an R1CS form) without invoking the Fiat-Shamir transform. Unfortunately, current approaches to supporting higher degree polynomial maps seem to fundamentally require a quadratic prover time. Given the additional structure of lattices compared with code-based proof systems that rely only on hash functions, one might expect that it would be possible to obtain smaller proof sizes and faster verification times. The recent work LaBRADOR [**BS22**] was the first lattice-based system to achieve proof sizes smaller (both concretely and asymptotically) than code-based systems, but this has not yet been done for combined proof size and verification time. The results of this paper make progress in this direction.

4.1.1 Our results

Building off of the techniques in [**C:ACLMT22**], we present the first functional commitment for linear relations from lattices that asymptotically has quasilinear prover time, logarithmic proof size, and polylogarithmic verifier time. The scheme supports commitments to vectors $\mathbf{x} \in \mathcal{R}^w$ over a cyclotomic ring $\mathcal{R}$ of degree n and openings of the form $\sum_{i=1}^{w} x_i \cdot f_i$ for any $\mathbf{f} \in \mathcal{R}^w \bmod q$ for a prime $q \gg \alpha$, where both $\mathbf{x}$ and $\mathbf{f}$ have norm bounded by α. In particular, proofs are $O(\log w\alpha)$ and verification time is $O(\log w\alpha \cdot \log\log w\alpha \cdot \log\log\log w\alpha)$.

We present several extensions to Orbweaver and show how it can be used to build other primitives including:

- A public proof aggregation protocol achieving $O(\log t)$ size aggregate proofs for t linear maps evaluations where each linear map being evaluated and committed input may be distinct.
- An inner product argument.
- A linear functional commitment for the non-cyclotomic ring $\mathcal{R} = \mathbb{Z}$ that combines ternary decomposition and proof aggregation to achieve sublogarithmic proofs in the input norm of size $O(\log w + \log\log \alpha)$.
- Univariate and multilinear polynomial commitments. For degree d univariate polynomials we

achieve $O(\log d)$ openings and polylogarithmic verifier time.

We also provide concrete proof size comparisons to previous lattice-based proof systems. While succinct hash-based arguments provide better proof sizes for smaller statements, the most efficient of these systems still have $O(\log^2 n)$ proofs. Especially in settings where the input norm is small, Orbweaver provides better proof sizes for larger-but-practical circuits than state-of-the-art hash-based arguments.

4.1.2 Related work

Efficient lattice-based proof systems have been constructed in the designated verifier setting [**GMNO18**; **ISW21**]. In the publicly verifiable setting, there are many results that achieve succinct proof sizes but require linear verifier time [**BBCPGL18**; **C:BLNS20**; **C:AttCraKoh21**; **BCS21**; **BS22**]. There are also works focusing on practical lattice-based ZK proofs, which have proof size linear in the witness size, but are concretely efficient for small statements [**ESLL19**; **ALS20**; **ENS20**; **ESLR22**; **LNP22**].

Recently, Albrecht et al. [**C:ACLMT22**] constructed extractable lattice-based vector commitments supporting arbitrary degree polynomial maps. For degree d maps their prover time and CRS length are $\tilde{O}(w^{2d})$, while proof size and verifier time are $O(d\log(w))$. Wee and Wu [**WW22**] construct non-extractable lattice-based vector commitments for linear functions including polynomial commitments with $\mathrm{polylog}(w)$ openings and $\tilde{O}(w^2)$ CRS size and prover time. Castro and Peikert [**CP22**] construct non-extractable vector commitments for functions of bounded complexity based on the standard SIS assumption. Their polynomial commitments achieve proof size $O(\log^4 w)$. Balbás, Catalano, Fiore, and Lai construct extractable lattice-based vector commitments supporting arithmetic circuits of width w and depth d that achieve proof size $O(d\log^2 w)$, but have a CRS of size $O(w^5)$ [**EPRINT:BCFL22**]. LaBRADOR [**BS22**] designs a SNARK for quadratic relations with quasilinear prover and verifier time, but via a complex recursive argument achieve $O(\log^2 w)$ proofs that concretely surpass the best results from hash-based proof systems.

There is a much longer history of succinct arguments constructed from Merkle hashes that begins with [**Kilian92**] and more recently includes FRI [**C:BBHR19**], Ligero [**Ligero**], Aurora

[**Aurora**], Brakedown [**Brakedown**], and Orion [**XZS22**] among others.

4.1.3 Technical overview

At a high level, our construction uses an assumption from the k-R-ISIS assumption family introduced in [**C:ACLMT22**] to translate the knowledge-of-exponent (KEA) based linear vector commitment scheme sketched in [**ac20**] (which is in turn based on [**AC:Groth10a**], and also independently similar to ideas in [**ICALP:LibRamYun16**]) to the lattice setting. The construction should enable a prover to commit to a vector $\mathbf{x}$ and later open any linear function $f(\mathbf{x}) = \langle [f_i]_{i=1}^w,\ [x_i]_{i=1}^w \rangle = \sum_{i=1}^w f_i x_i$.

As observed in [**ac20**], given a function $x(v) = \sum_{i=1}^w x_i v^i$, and a function $f(v) = \sum_{i=0}^{w-1} f_i v^{-i}$, then $h(v) = x(v)f(v) = \sum_{i=-w+1}^{w-1} a_i v^i$, where $a_0 = \sum_{i=1}^w f_i x_i$. The idea is that the prover sends $c = x(v)$, $y = f(\mathbf{x})$, and $\pi = \sum_{i=-w+1, i\neq 0}^{w-1} a_i v^i$, and then the verifier computes $f(v) = \mathsf{ck}_f$ and checks $c \cdot \mathsf{ck}_f - y = \pi$.

In order to achieve both binding and succinctness, we situate this abstract protocol in lattice setting using the techniques of [**C:ACLMT22**]. A trusted setup generates a universal (i.e., the same setup works for any linear function up to a given size) structured reference string (SRS) $(\mathbf{a}, v, \mathbf{u}_{-w+1}, \dots, \mathbf{u}_{-1}, \mathbf{u}_1, \dots, \mathbf{u}_{w-1}) \in \mathcal{R}_q \times \mathcal{R}_q^\times \times (\mathcal{R}_q)^{2w-2}$, where for each $\mathbf{u}_i$ it holds that $\langle \mathbf{a},\ \mathbf{u}_i \rangle \equiv v^i \bmod q$ and $\mathbf{u}_i$ is "short" relative to q. Note that no preimage $\mathbf{u}_0$ is given out such that $\langle \mathbf{a},\ \mathbf{u}_0 \rangle \equiv 1 \bmod q$. Our k-R-ISIS assumption states that it should be hard to find short $(s, \mathbf{u}) \neq (0, \mathbf{0}) \in \mathcal{R} \times \mathcal{R}$ such that $\langle \mathbf{a},\ \mathbf{u} \rangle \equiv s \bmod q$ even given these preimages for other powers of v. In general, it should be hard to find the preimage of any target not in the linear span of $[v_i]_{i=-w+1, i\neq 0}^{w-1}$.

Let α be a bound on the ℓ-infinity norm of both the witness and function. The computation of commitment c, commitment key ck_f, and y are exactly as in the abstract protocol, but taken mod q for prime $q \gg \alpha$. The prover now computes

$$\pi_0 := \sum_{i=-w+1, i\neq 0}^{w-1} a_i \mathbf{u}_i \bmod q \ ,$$

and the verifier checks

$$\langle \mathbf{a},\ \pi_0 \rangle \equiv c \cdot \mathsf{ck}_f - y \bmod q \ , \tag{4.1}$$

and that π_0 and y have small norm. When correctly executing the protocol it follows from the fact that $\alpha \ll q$ and $\beta \ll q$, where β is a norm bound on the $\mathbf{u}_i$, that the proof and output will have small norm. Fixing $\mathcal{R}_q$ with respect to the security parameter, one can see that the size of π_0 is of size $O(\log w + \log \alpha)$. We also achieve (almost) the same verifier time [1] in the preprocessing setting: while computing the commitment key $\mathsf{ck}_f = \sum_{i=1}^{w} f_i v^{-i}$ takes linear time, it may be computed once in advance then subsequently used to verify openings with respect to any c and y.

Our scheme thus far achieves evaluation binding: an adversary who can open a commitment c to two different outputs for the same function can be used to break k-R-ISIS. To see this, we subtract one verifying eq. (4.1) from the other to obtain

$$\langle \mathbf{a},\ \pi_0 - \pi_0' \rangle \equiv y' - y \bmod q$$

Since both proofs and outputs are small, this gives k-R-ISIS solution $s = y' - y, \mathbf{u} = \pi_0 - \pi_0'$.

Achieving extractability. While this construction is evaluation binding, this is insufficient to guarantee that the prover knows some witness consistent with the commitment. For example, the prover can randomly sample a short proof π_0, and compute the commitment $c = \langle \mathbf{a},\ \pi_0 \rangle / (\sum_{i=1}^{w} f_i v^{-i})$. In this case, the prover can pass the verifier checks without knowledge of the input.

To achieve extractability we require an additional knowledge assumption from [**C:ACLMT22**], one from the knowledge k-R-ISIS assumption family. Our knowledge k-R-ISIS assumption states that there is an extractor that extracts short $\mathbf{x}^*$ s.t. $c = \sum_{i=1}^{w} x_i^* v^i$ from any prover who, given $[\mathbf{u}_i']_{i \in [w]}$ s.t. $\langle \mathbf{a},\ \mathbf{u}_i' \rangle \equiv v^i \cdot t \bmod q$ (for some t drawn from a special subset of $\mathcal{R}$), outputs commitment c and a short knowledge proof π_1 s.t. $\langle \mathbf{a},\ \pi_1 \rangle \equiv c \cdot t \bmod q$.

Using this assumption, it suffices to include as part of the commitment an additional value $\pi_1 = \sum_{i=1}^{w} x_i \mathbf{u}_i$ in order to achieve extractability. Observe that if for a verifying (c, y, π_0, π_1) it held that $f(\mathbf{x}^*) \neq y$, then $\mathbf{x}^*$ could be used to create verifying $(c, y' = f(\mathbf{x}^*), \pi_0', \pi_1')$, which violates binding and could be used to break plain k-R-ISIS as explained above. Therefore, the extracted witness must be consistent with the evaluation.

[1] Our verifier time is $O(\log(w \cdot \alpha) \log\log(w \cdot \alpha))$ in the preprocessing setting.

Lastly, we note our extracted witness has "stretch", i.e., the best norm bound we can hope to enforce on $\mathbf{x}^*$ is the norm bound on π_1, which in order to achieve correctness is necessarily bigger than the norm bound we impose on $\mathbf{x}$.

Public proof aggregation. An untrusted prover, input t verifying tuples $(c_i, \pi_{0,i}, \pi_{1,i}, y_i)$ tuples, can combine the evaluation proofs into a single proof $\pi_0 := \sum_{i=1}^{t} h_i \pi 0_i$ using some short challenges h_i. The verifier then checks

$$\langle \mathbf{a},\ \pi_0 \rangle \equiv \mathsf{ck} \cdot \sum_{i=1}^{t} (h_i c_i) - \sum_{i=1}^{t} (h_i y_i) \bmod q \ .$$

Since the h_i and $\pi_{0,i}$ are short, π_0 will be as well. If we consider the norm bound on the h_i to be dependent on the security parameter, then we can see the bit length $|\pi_0|$ is $\log t$ times the bit length of the original proofs $|\pi_{0,i}|$.

Without modifying our knowledge assumption, we cannot take linear combinations of the knowledge proofs. So we only get partial aggregation.

Polynomial commitments for integers. Furthermore, for the special case of polynomial commitments, where $f_i = z^{i-1} \bmod p$ for $i \in [1, w]$ and w is a power of 2 and $q > p^{\log w}$, $\mathsf{ck}_f = \prod_{i=0}^{\log w - 1}(z_i + v^{-2^i})v^{-(w+1)} - z_{\log w} \bmod q$ where $z_i = z^{2^i} \bmod p$, which can be computed in time $O(\log w)$ operation over $\mathcal{R}_q$. Given this value the verifier can compute $c \cdot \mathsf{ck}_f = (\sum_{i=1}^{w} x_i v^i)(\sum_{i=1}^{w} f_i v^{-i}) = \sum_{i=1}^{w} f_i x_i + \sum_{i=-w+1}^{w-1} a_i v^i = f(\mathbf{x}) + \sum_{i=-w+1}^{w-1} a_i v^i$, for some ring elements a_i. The opening proof will include information that allows the verifier to cancel out the term dependent on the a_i values. To prevent the prover from cheating, we invoke the k-R-ISIS assumption by involving $\mathbf{a}$ in the opening. More specifically, the opening is computed as $\pi_0 = \sum_{i \in \mathbb{Z}(w)\{0\}} a_i \mathbf{u}_i$, where a_i's can be computed by the prover. The verifier of the opening checks $\langle \mathbf{a},\ \pi_0 \rangle = c \cdot \mathsf{ck}_f - y$, and that f, y, π_0 are short (i.e., their norm is bounded). The k-R-ISIS hardness assumption guarantees that the commitment is evaluation binding when f and x are both short.

4.2 Preliminaries

Let $\mathbb{Z}(b) := (-b, b) \cap \mathbb{Z}$ and $[a] := [1, a] \cap \mathbb{Z}$. For a ring $\mathcal{R}$ of degree n, let $\mathsf{vec}(r) \in \mathbb{Z}^n$ denote the coefficient vector of $r \in \mathcal{R}$ in the integral basis.

For $m \in \mathbb{N}$, let $\zeta_m \in \mathbb{C}$ be any fixed primitive m-th root of unity. Let $\mathcal{R} = \mathbb{Z}[\zeta_m]$ denote its ring of integers, called a cyclotomic ring. We have $\mathcal{R} \cong \mathbb{Z}[x]/\langle \Phi_m(x) \rangle$, where $\Phi_m(x)$ is the m-th cyclotomic polynomial. If m is a power of 2, we call $\mathcal{R}$ a power-of-2 cyclotomic ring. In this paper, we exclusively use power-of-2 cylotomic rings. Let $q \in \mathbb{N}$ be a prime number, we let $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$ and let $\mathcal{R}_q^\times$ denote all invertible elements in $\mathcal{R}_q$. For any $f \in \mathcal{R}$, let $\mathsf{ct}(f)$ denote the constant term of f (i.e., $\mathsf{ct}(f) = \mathsf{vec}(f)_0$).

For $x \in \mathcal{R}$, let $\|x\|$ denote the ℓ-infinity norm of its coefficient vector, i.e., $\|x\| = \max_{i \in [n]} \mathsf{vec}(x)$. We use $\|\cdot\|_p$ for the ℓ_p-norm (e.g., $\|\cdot\|_2$ for the ℓ_2 norm).

Definition 4.2.1 (Ring expansion factor)**.** Let $\mathcal{R}$ be a ring. The *expansion factor* of $\mathcal{R}$ is defined as $\gamma_{\mathcal{R}} := \max_{a,b \in \mathcal{R}} \frac{\|a \cdot b\|}{\|a\| \cdot \|b\|}$.

Theorem 4.2.2 ([**C:AlbLai21**])**.** *If $\mathcal{R} = \mathbb{Z}[\zeta_m]$ is a power-of-2 cyclotomic ring, then $\gamma_{\mathcal{R}} \leq n$.*

Theorem 4.2.3 ([**C:ACLMT22**])**.** *Let $q = \omega((w \cdot f)^{f/\phi(m)})$ be a rational prime such that $\mathcal{R}_q$ splits into f fields each of size $q^{\phi(m)/f}$. For $v \leftarrow\$ \mathcal{R}_q^w$, we have $v \in (\mathcal{R}_q^\times)^w$ with probability $(1 - 1/q^{\phi(m)/f})^{w \cdot f}$.*

Subsequently in this work, we set q large enough so that uniformly random $v \leftarrow\$ \mathcal{R}_q$ satisfies $v \in \mathcal{R}_q^\times$ with non-negligible probability.

The conjugation automorphism. The cyclotomic ring $\mathcal{R}$ has a group of automorphisms $\sigma_{-1}(\mathcal{R})$ that is isomorphic to $\mathbb{Z}_{2n}^\times$,

$$i \to \sigma_i : \mathbb{Z}_{2n}^\times \to \sigma_{-1}(\mathcal{R})$$

We make use of the following property: given two vectors $\mathsf{vec}(a) = (a_0, \dots, a_{kn-1}) \in \mathbb{Z}^{kn}$, $\mathsf{vec}(b) = (b_0, \dots, b_{kn-1}) \in \mathbb{Z}^{kn}$ where $k \geq 1$, let $c \leftarrow \sum_{i=0}^{k-1} \sigma_{-1}(\sum_{j=0}^{n-1} a_{i \cdot n+j} X^j) \cdot (\sum_{j=0}^{n-1} b_{i \cdot n+j} X^j) \in \mathcal{R}$; then the constant coefficient denoted as $\mathsf{ct}(c)$ has the property that $\mathsf{ct}(c) = \langle \mathsf{vec}(a),\ \mathsf{vec}(b) \rangle$ (adapted from [**LNP22**], see also [**ENS20**]).

4.2.1 Exceptional challenge sets

For a cyclotomic ring $\mathcal{R}$ of degree n, let $\mathcal{H} \subset \mathcal{R}$ be the set of ring elements with c plus or minus one coefficients and $n - c$ zero coefficients. Then $|\mathcal{H}| = 2^c\binom{n}{c}$, and if $\mathcal{R}_q$ is a power-of-two cyclotomic, the operator norm of $\mathcal{H}$ with respect to the ℓ-infinity norm is c, i.e., $\mathsf{op}(\mathcal{H}) = c$ (lemma 4.2.5). By [**ls20**] for n a power of two and $q \equiv 2n + 1 \bmod 4n$ prime the $2n$-th cyclotomic polynomial $X^n + 1$ splits completely into linear factors modulo q.[2] Further, infinitely many such primes q exist and for any $h \in \mathcal{R}$ that satisfies $0 < \|h\|_2 < q^{1/n}$ has an inverse in $\mathcal{R}_q$. We wish to ensure that the difference between any two distinct elements in the challenge set is not a zero divisor, i.e., $\mathcal{H}$ is *exceptional*, in order to invoke the following theorem.

Theorem 4.2.4 (Generalized Alon–Füredi Theorem [**BCPS18**])**.** *Let $\mathcal{R}$ be a ring and let $\mathcal{H}$ be an exceptional, non-empty, finite subset of $\mathcal{R}$. Then for any non-zero polynomial $f \in \mathcal{R}[Y_1, \dots, Y_t]$ such that* $\deg(f) < |H|$ *it holds that*

$$\Pr[f(h_1, \dots, h_t) = 0 \mid h_1, \dots, h_t \leftarrow\$ \mathcal{H}] \leq \frac{\deg(f)}{|\mathcal{H}|} \quad .$$

Concretely, for $\deg(f) = 1$ and $c = 19$ the right-hand of side of the inequality above is $< 2^{-132}$. We know that differences $c_i - c_j$ for $c_i \neq c_j \in \mathcal{H}$ with 19 ± 2 coefficients can be reduced to the invertibility of 2 and polynomials with 18 ± 1 coefficients. Differences with 18 ± 2 coefficients and 2 ± 1 coefficients have ℓ_2 norm $\sqrt{74}$. We must then have $q \geq 2^{1590} > \sqrt{74}^{512}$. It turns out even larger q are optimal for proof size for concrete instantiations of our scheme capable of handling large circuits.

Lemma 4.2.5 (Operator norm of sparse ternary sets)**.** *For power-of-two n, let $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ and $1 \leq c \leq n$. The set*

$$\mathcal{H} = \left\{h \in \mathcal{R}_q \;\middle|\; \|h\|_\infty = 1 \wedge \|h\|_1 = c\right\}.$$

[2] A close reading of the proof reveals for the fully splitting case that the simpler condition $q \equiv 1 \bmod n$ suffices.

has operator norm

$$\mathsf{op}_\infty(\mathcal{H}) = \max_{h\in\mathcal{H}, r\in\mathcal{R}} \frac{\|hr\|_\infty}{\|h\|_\infty \|r\|_\infty} = c$$

Proof. Let $\mathbf{r}, \mathbf{c}$ be the coefficients of r and c. Let $g = rc$ with coefficient vector $\mathbf{g}$. Then

$$g_k = \sum_{i+j=k} r_i c_j - \sum_{i+j=n+k} r_i c_j$$

Fixing any $j, k \in \{0, \dots, n-1\}$, there is exactly one $i \in \{0, \dots, n-1\}$ such that either $i + j = n$ or $i + j = n + k$. Therefore, each coefficient g_k can be bounded by $c\|r\|_\infty$. □

4.2.2 Functional commitments

Definition 4.2.6 (Functional commitment)**.** A (pre-processing non-interactive) functional commitment (FC) scheme is parameterized by a function family

$$\mathcal{F} \quad := \{\mathcal{F}_w \subseteq \{f : \mathcal{X}^w \to \mathcal{Y}\}\}_{w\in\mathbb{N}}$$

over a ring $\mathcal{R}$ for input alphabet $\mathcal{X} \subseteq \mathcal{R}$ and image space $\mathcal{Y} \subseteq \mathcal{R}$, where w is the dimension of the secret (committed) input. The FC scheme is defined by a 5-tuple of PPT algorithms ($\mathsf{Setup}, \mathsf{Com}, \mathsf{Open}, \mathsf{PreVerify}, \mathsf{Verify}$), working as follows:

- $\mathsf{Setup}(1^\lambda, 1^w) \to (\mathsf{ck}, \mathsf{vk})$: Input (in unary) security parameter λ and secret input dimension w, samples commitment key ck and verification key vk.
- $\mathsf{Com}(\mathsf{ck}, \mathbf{x}) \to (c, \pi_1)$: Input commitment key ck and secret input $\mathbf{x} \in \mathcal{X}^w$, computes commitment c and a proof of knowledge π_1 of vector $\mathbf{x}$ such that $c = \mathsf{Com}(\mathsf{ck}, \mathbf{x})$.
- $\mathsf{Open}(\mathsf{ck}, f, \mathbf{x}) \to \pi_0$: Input commitment key ck, function $f \in \mathcal{F}_w$, and secret input $\mathbf{x} \in \mathcal{R}^w$, computes opening proof π_0 for the evaluation $f(\mathbf{x})$.
- $\mathsf{PreVerify}(\mathsf{ck}, f) \to \mathsf{vk}_f$: Input verification key vk and function $f \in \mathcal{F}_w$, computes preprocessed commitment key vk_f. Preprocessing only needs to be performed once per function and allows Verify to run in time sublinear in f.
- $\mathsf{Verify}(\mathsf{vk}_f, c, y, \pi_1, \pi_0) \to \{0, 1\}$: Input preprocessed verification key vk_f, commitment c, output

$y \in \mathcal{Y}$, and proofs π_1 and π_0, the verifier returns 1 if the proofs convince them the verifier knows some $\mathbf{x} \in \mathcal{X}^w$ such that $\mathsf{Com}(\mathsf{ck}, \mathbf{x}) = c$ and $f(\mathbf{x}) = y$ (else 0).

We require that functional commitments satisfy correctness, extractability, and succinctness as defined below.

Definition 4.2.7 (Correctness)**.** An FC scheme for $(\mathcal{F}, \mathcal{X}, \mathcal{Y})$ is *correct* if for any $\lambda, w \in \mathbb{N}$, any $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda, 1^w)$, and for any $(f, \mathbf{x}, y) \in \mathcal{F} \times \mathcal{X}^w \times \mathcal{Y}$ satisfying $f(\mathbf{x}) = y$, any $(c, \pi_1) \leftarrow \mathsf{Com}(\mathsf{ck}, \mathbf{x})$, any $\pi_0 \leftarrow \mathsf{Open}(\mathsf{ck}, f, \mathbf{x})$, and any $\mathsf{vk}_f \leftarrow \mathsf{PreVerify}(\mathsf{ck}, f)$, it holds that $\Pr[\mathsf{Verify}(\mathsf{vk}_f, c, y, \pi_1, \pi_0) = 1] = 1$.

At a high level, extractability of an FC scheme requires that if an adversary can produce a commitment c and a valid opening (π_1, π_0) for some function f and some evaluation y, it must know $\mathbf{x} \in \mathcal{X}^w$ satisfying $\mathsf{Com}(\mathsf{ck}, \mathbf{x}) = c$ and $f(\mathbf{x}) = y$. Note that extractability implies *weak binding* (meaning that it is not possible to open a commitment to two different evaluations for the same function).

Definition 4.2.8 (Extractability)**.** A FC scheme for $(\mathcal{F}, \mathcal{X}, \mathcal{Y})$ is $(\kappa, \mathcal{X}^*)$-*extractable* if for any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}_{\mathcal{A}}$ that, input $(\mathsf{ck}, \mathsf{vk})$ and given black-box access to $\mathcal{A}$ and any randomness it uses, returns $\mathbf{x}^* \in (\mathcal{X}^*)^w$ such that

$$\Pr\left[\begin{array}{rl} & \mathsf{Verify}(\mathsf{vk}_f, c, y, \pi_1, \pi_0) = 1 \\ \wedge & \left(\begin{array}{rl} & \mathbf{x} \notin (\mathcal{X}^*)^w \\ \vee & c \neq \mathsf{Com}(\mathsf{ck}, \mathbf{x},) \\ \vee & f(\mathbf{x}) \neq y \end{array}\right) \end{array} \middle| \begin{array}{r} (\mathsf{ck}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^w) \\ (f, c, y, \pi_1, \pi_0) \leftarrow \mathcal{A}(\mathsf{ck}, \mathsf{vk}) \\ \mathbf{x} \leftarrow \mathcal{E}_{\mathcal{A}}(\mathsf{ck}, \mathsf{vk}) \\ \mathsf{vk}_f \leftarrow \mathsf{PreVerify}(\mathsf{vk}, f) \end{array}\right] \leq \kappa(\lambda, w) \ .$$

We say the scheme is $\mathcal{X}^*$-extractable if its *knowledge error* $\kappa(\lambda, w)$ is negligible in λ for $w = \mathrm{poly}(\lambda)$.

Definition 4.2.9 (Succinctness)**.** Let Π be a VC scheme for the alphabet $\mathcal{X} = \{r \in \mathcal{R} \mid \|r\| \leq \alpha\}$. We say Π is *succinct* if it satisfies the following properties:

- *Proof succinctness*: $|c + \pi_1 + \pi_0| = \mathrm{poly}(\log w + \log \alpha)$.
- *Verifier succinctness*: Verify runs in time $\mathrm{poly}(\log w + \log \alpha)$.

4.2.3 Sampling Algorithm

The following relies on the Leftover Hash Lemma over rings to generate some vector $\mathbf{a}$ that is indistinguishable from a uniformly randomly sample vector, and some trapdoor that makes it possible to easily generate vectors in the kernel of $\mathbf{a}$. We let $\mathsf{lhl}(\mathcal{R}_q, \beta)$ denote an algorithm that outputs the minimum $\ell \in \mathbb{N}$, which ensures that the resulting distribution of the vector $\mathbf{a}$ is indistinguishable from the uniform distribution. It is formally defined as follows, adapted from [**GPV08**; **EC:MicPei12**; **GM18**; **C:ACLMT22**]:

A sampling algorithm has the following three PPT algorithms (taking 1^λ as input implicitly):

- $(\mathbf{a}, \mathsf{td}) \leftarrow \mathsf{TrapGen}(1, q, \mathcal{R}, \beta)$: takes dimension $\ell \in \mathbb{N}$, a modulus $q \in \mathbb{N}$, a ring $\mathcal{R}$, and a norm bound $\beta \in \mathbb{R}$, and outputs a vector $\mathbf{a} \in \mathcal{R}_q^\ell$ and a trapdoor td. For any $n \in \text{poly}(\lambda), \ell \geq \mathsf{lhl}(\mathcal{R}_q, b)$ where $b = O(\beta)$, the distribution of $\mathbf{a}$ is within $\text{negl}(\lambda)$ statistical distance to $U(\mathcal{R}_q^\ell)$.
- $\mathbf{u} \leftarrow \mathsf{SampD}(1^\ell, \mathcal{R}_q, \beta)$: for $\ell \geq \mathsf{lhl}(\mathcal{R}_q, \beta)$, outputs $\mathbf{u}$ such that $\|\mathbf{u}\| \leq \beta$ and the distribution of $\langle \mathbf{a},\ \mathbf{u} \rangle \bmod q$ is withing $\text{negl}(\lambda)$ statistical distance to $U(\mathcal{R}_q)$.
- $\mathbf{u} \leftarrow \mathsf{SampPre}(\mathsf{td}, v, \beta)$: for $\ell \geq \mathsf{lhl}(\mathcal{R}_q, \beta)$ and $v \in \mathcal{R}_q$, outputs $\mathbf{u} \in \mathcal{R}_q^\ell$ satisfying $\|\mathbf{u}\| \leq \beta$, $\langle \mathbf{a},\ \mathbf{u} \rangle \equiv v \bmod q$, and that distribution of $\mathbf{u}$ is within $\text{negl}(\lambda)$ statistical distance the distribution of $v' \leftarrow \mathsf{SampD}(1^\lambda, 1^\ell, \mathcal{R}_q, \beta)$ conditioned on $\langle \mathbf{a},\ \mathbf{u} \rangle \equiv v' \bmod q$.

4.2.4 Cryptographic assumptions

The Short Integer Solution (SIS) problem was first introduced in [**Ajt96**], which asks to find a short element (of ℓ_2 norm) in the kernel of a random matrix over the ring $\mathbb{Z}_q$. An inhomogeneous version, ISIS, instead asks for a short solution to some linear equation system [**Mic07**]. It has been shown that the SIS and ISIS problems are equivalent.

We define the ring version of SIS (R-SIS) from [**Mic07**] as follows.

Definition 4.2.10 (R-SIS [**Mic07**])**.** Let $\mathcal{R}, q, , \beta$ be parameters depending on λ. The R-SIS problem states the following: for $\mathbf{a} \leftarrow\$ \mathcal{R}_q$ sampled uniformly at random, and $t = 0$, find $\mathbf{u} \neq \mathbf{0} \in \mathcal{R}$ such that $\|\mathbf{u}\| \leq \beta$ and $\langle \mathbf{a},\ \mathbf{u} \rangle \equiv t \bmod q$.

When $t \leftarrow\$ \mathcal{R}_q$ this becomes the ring inhomogeneous SIS (R-ISIS) assumption, which is known to be equivalent. For appropriate parameters, there are no known efficient algorithms for solving R-SIS for cyclotomic rings.

k-R-ISIS assumptions

We define a family of assumptions over rings, k-R-ISIS, introduced in [**C:ACLMT22**]. k-R-ISIS assumptions can be trivially broken if some basic conditions are not satisfied, so we begin by defining those via the notion of k-R-ISIS admissibility:

Definition 4.2.11 (k-R-ISIS admissible)**.** Let $g \in \mathcal{R}(X)$ be a Laurent monomial, i.e., $g(X) := \mathbf{X}^{\mathbf{e}} := \prod_{i \in [y]} X_i^{e_i}$ for some exponent vector $\mathbf{e} \in \mathbb{Z}^y$. Let $\mathcal{G} \subset \mathcal{R}(X)$ be a set of Laurent monomials with $k := |\mathcal{G}|$. Let $g^* \in \mathcal{R}(X)$ be a *target* Laurent monomial. We say a *monomial family* $(\mathcal{G}, g^*)$ is k-R-ISIS admissible if the following conditions are satisfied:

1. All $g \in \mathcal{G}$ and g^* have constant degree
2. All $g \in \mathcal{G}$ are distinct
3. $0 \notin \mathcal{G}$
4. $g^* \notin \mathcal{G}$

Remark 4.2.12. Condition 1 rules out the monomials that depend on $\mathcal{R}$. Condition 2 rules out that trivial linear combinations of preimages give a preimage for the target. Condition 3 rules out a trivially producing multiple preimages of the same target. Condition 4 rules out the trivial solution to get the preimage of the target.

We then define the k-R-ISIS assumption as follows:

Definition 4.2.13 (k-R-ISIS assumption)**.** Let $\ \in \mathbb{N}$. Let q be a rational prime, $\mathcal{R}$ a cyclotomic ring, and $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$. Let $\mathcal{G} \subset \mathcal{R}(X)$ be a set of y-variate Laurent monomials and let $g^* \in \mathcal{R}(X)$ be a target Laurent monomial such that $(\mathcal{G}, g^*)$ is k-R-ISIS-admissible. Let $\beta, \beta^* \geq 1$ be reals. For $g \in \mathcal{G}$, $\ \geq \mathsf{lhl}(\mathcal{R}_q, \beta)$, $\mathbf{a} \in \mathcal{R}_q$, and $\mathbf{v} \in (\mathcal{R}_q^\times)^y$, let $\mathcal{D}_{g,\mathbf{a},\mathbf{v}}$ be a distribution over

$$\{\mathbf{u}_g \in \mathcal{R} \mid \langle \mathbf{a},\ \mathbf{u}_g \rangle \equiv g(\mathbf{v}) \bmod q \ \wedge\ \|\mathbf{u}_g\| \leq \beta\} \ .$$

Let $\mathcal{D} := \left\{\mathcal{D}_{g,\mathbf{a},\mathbf{v}} : \in \mathbb{N},\ g \in \mathcal{G},\ \mathbf{a} \in \mathcal{R}_q,\ \mathbf{v} \in (\mathcal{R}_q^\times)^y\right\}$ be the family of these distributions. Define $\mathsf{pp} := (\mathcal{R}_q, , y, \mathcal{G}, g^*, \mathcal{D}, \beta, \beta^*)$. The k-R-$\mathsf{ISIS}_{\mathsf{pp}}$ assumption states that for any PPT adversary $\mathcal{A}$, we have $\mathsf{Adv}^{k\text{-}R\text{-}\mathsf{ISIS}}_{\mathsf{pp},\mathcal{A}}(\lambda) \leq \mathrm{negl}(\lambda)$, where $\mathsf{Adv}^{k\text{-}R\text{-}\mathsf{ISIS}}_{\mathsf{pp},\mathcal{A}}(\lambda)$ is the following probability:

$$\Pr\left[\begin{array}{ll} & \langle \mathbf{a},\ \mathbf{u}_{g^*}\rangle \equiv s^* \cdot g^*(\mathbf{v}) \bmod q \\ \wedge & \|s^*\| \leq \beta^* \\ \wedge & \left\|\mathbf{u}_{g^*}\right\| \leq \beta^* \\ \wedge & (s^* \cdot g^*, \mathbf{u}_{g^*}) \neq (0, \mathbf{0}) \end{array} \middle| \begin{array}{r} \mathbf{a} \leftarrow\$\ \mathcal{R}_q \\ \mathbf{v} \leftarrow\$\ (\mathcal{R}_q^\times)^y \\ \mathbf{u}_g \leftarrow\$\ \mathcal{D}_{g,\mathbf{a},\mathbf{v}},\ \ \forall g \in \mathcal{G} \\ (s^*, \mathbf{u}_{g^*}) \leftarrow \mathcal{A}(\mathbf{a}, t, [\mathbf{u}_g]_{g\in\mathcal{G}}, \mathbf{v}) \end{array}\right] .$$

Remark 4.2.14. For simplicity, we set t in [**C:ACLMT22**] to be fixed to 1 and thus have $\langle\{t\}\rangle = \mathcal{R}_q$. The k-R-ISIS assumption requires v to be in $\mathcal{R}_q^\times$. Otherwise, the scheme can be insecure. For example, for a power-of-two $\mathcal{R}$, if $q = 2^k$, the ideal $q\mathcal{R}$ splits into $\mathcal{J}^{k\cdot\phi(n)}$ for some ideal $\mathcal{J}$ with norm 2. Then, for $v \leftarrow\$\ \mathcal{R}_q$, $\Pr[v = 0 \mod \mathcal{J}] = 1/2$. Thus, $v^{k\cdot\phi(n)} = 0 \bmod q$. Therefore, we have $p(X) = X^{k\cdot\phi(n)}$ being a solution to any R-V-SIS instance over $\mathcal{R}_q$. Further, we require $1/|\mathcal{R}_q^\times|$ to be negligible such that SIS attacks are not possible in the image space. theorem 4.2.3 shows how to pick parameters to guarantee this.

In [**C:ACLMT22**] they also introduce the following meta assumption:

Assumption 1 (k-R-ISIS meta assumption)**.** *For any k-R-ISIS admissible $(\mathcal{G}, g^*)$, k-R-$\mathsf{ISIS}_{\mathcal{R}_q,,y,\mathcal{G},g^*,\mathcal{D},\beta,\beta^*}$ is hard if R-$\mathsf{ISIS}_{\mathcal{R}_q,,\beta^*}$ is hard and if R-$\mathsf{SIS}_{\mathcal{R}_q,k,\beta^*}$ is hard.*

To achieve extractability we also require an additional knowledge assumption, because by [**GW11**], we need an unfalsifiable assumption to prove the extractability, as follows:

Definition 4.2.15 (Knowledge k-R-ISIS)**.** Let the parameters $(\mathcal{R}_q, , y, \mathcal{G}, \beta, \beta^*)$ be defined as in definition 4.2.13. Let $\alpha^* \geq 1$ be a real. Let $\mathcal{T} \subset \mathcal{R}_q$ be such that for all $t \in \mathcal{T}$ it holds that:

1. $|\langle t\rangle|/|\mathcal{R}_q| = \mathrm{negl}(\lambda)$, and
2. finding $s' \in \mathcal{R}_q$ satisfying $s' \cdot t \equiv 0 \bmod q$ and $0 < \|s'\| \leq \alpha^*$ is hard.

For $g \in \mathcal{G}$, $\geq \mathsf{lhl}(\mathcal{R}_q, \beta)$, $\mathbf{a} \in \mathcal{R}_q$, $t \in \mathcal{T}$, and $\mathbf{v} \in (\mathcal{R}_q^\times)^y$, let $\mathcal{D}_{g,\mathbf{a},t,\mathbf{v}}$ be a distribution over

$$\{\mathbf{u}_g \in \mathcal{R} \mid \langle \mathbf{a},\ \mathbf{u}_g\rangle \equiv g(\mathbf{v}) \cdot t \bmod q\ \wedge\ \left\|\mathbf{u}_g\right\| \leq \beta\}\ .$$

Let $\mathcal{D} := \left\{\mathcal{D}_{g,\mathbf{a},t,\mathbf{v}} : \in \mathbb{N},\ g \in \mathcal{G},\ \mathbf{a} \in \mathcal{R}_q,\ t \in \mathcal{T},\ \mathbf{v} \in (\mathcal{R}_q^\times)^y\right\}$ be the family of these distributions.

Define $\mathsf{pp} := (\mathcal{R}_q, , y, \mathcal{G}, \mathcal{D}, \mathcal{T}, \alpha^*, \beta, \beta^*)$. The knowledge k-R-$\mathsf{ISIS}_{\mathsf{pp}}$ assumption states that for any PPT adversary $\mathcal{A}$ there exists a PPT extractor $\mathcal{E}_{\mathcal{A}}$ s.t. $\mathsf{Adv}^{k\text{-}R\text{-}\mathsf{ISIS}}_{\mathsf{pp},\mathcal{A}} \leq \mathrm{negl}(\lambda)$, where $\mathsf{Adv}^{k\text{-}R\text{-}\mathsf{ISIS}}_{\mathsf{pp},\mathcal{A}}$ is the following probability:

$$\Pr\left[\begin{array}{ll} & \langle \mathbf{a}, \mathbf{u} \rangle \equiv c \cdot t \bmod q \\ \wedge & \|\mathbf{u}\| \leq \beta^* \ \wedge \ (c, \mathbf{u}) \neq (0, \mathbf{0}) \\ \wedge & \neg \left(\begin{array}{ll} & c = \sum_{g \in \mathcal{G}} x_g \cdot g(\mathbf{v}) \bmod q \\ \wedge & \|x_g\| \leq \alpha^*, \ \forall g \in \mathcal{G} \end{array} \right) \end{array} \middle| \begin{array}{r} \mathbf{a} \leftarrow\$ \mathcal{R}_q \bmod q; \ t \leftarrow\$ \mathcal{T}; \ \mathbf{v} \leftarrow\$ (\mathcal{R}_q^\times)^w \\ \mathbf{u}_g \leftarrow\$ \mathcal{D}_{g,\mathbf{a},\mathbf{v}}, \ \forall g \in \mathcal{G} \\ ((c, \mathbf{u}), [x_g]_{g \in \mathcal{G}}) \leftarrow (\mathcal{A} \| \mathcal{E}_{\mathcal{A}})(\mathbf{a}, t, [\mathbf{u}_g]_{g \in \mathcal{G}}, \mathbf{v}) \end{array}\right].$$

To understand the first restriction on $\mathcal{T}$, consider an adversary who samples a random short $\mathbf{u}$ and then checks if $\langle \mathbf{a}, \ \mathbf{u} \rangle \in \langle t \rangle$, setting the commitment to be c s.t. $\langle \mathbf{a}, \ \mathbf{u} \rangle \equiv c \bmod q$ if so. Since $\langle \mathbf{a}, \ \mathbf{u} \rangle$ is close to uniformly distributed over $\mathcal{R}_q$, restriction 1 ensures such an adversary succeeds with negligible probability. To understand the second restriction, consider an adversary who finds s' as above and outputs $(c, \mathbf{u}) = (s', s' \cdot \mathbf{u}_g)$ for an arbitrary $g \in \mathcal{G}$. Observe that $\mathbf{u}$ is short since s' and $\mathbf{u}_g$ are, and since $\langle \mathbf{a}, \ \mathbf{u}_g \rangle \equiv g(v) \cdot t \bmod q$ it follows $\langle \mathbf{a}, \ \mathbf{u} \rangle \equiv s' \cdot g(v) \cdot t \equiv 0 \equiv c \cdot t \bmod q$. However, we have that $s' \cdot g(\mathbf{v}) \not\equiv c \equiv s' \bmod q$ unless $g(\mathbf{v}) \equiv 1 \bmod q$, which should only happen with negligible probability.

In addition to the two contraints on $\mathcal{T}$ in our knowledge k-R-ISIS definition, in [**C:ACLMT22**] they give a third. Namely, that $1/|\langle t \rangle| = \mathrm{negl}(\lambda)$. We have omitted this constraint given it is implied by constraint 2. Let $\phi : \mathcal{R}_q \mapsto \langle t \rangle$. Then $|\mathsf{ker}(\phi)|/|\mathcal{R}_q| = 1/|\langle t \rangle|$. If $1/|\langle t \rangle|$ is not negligible, then $s' \leftarrow\$ \mathcal{R}_q$ satisfies $s' \cdot t \equiv 0 \bmod q$ with non-negligible probability.

We believe this assumption is suitable as it can be used to prove the extractability of our scheme as in the proof of theorem 4.4.2, and is not trivially broken due to all the restrictions above. As in [**C:ACLMT22**], we can set $\mathcal{T}$ to contain all the elements t such that exactly half of the elements in the NTT representation of t are zero. Note this is only defined when $\langle q \rangle$ is not a prime ideal in $\mathcal{R}$; in this case $\mathcal{R}_q \simeq \mathbb{Z}_{q^n}$, which only has one NTT coefficient. Our restriction that $q \equiv 2n + 1 \bmod \ 4n$ ensures that $\langle q \rangle$ is not a prime ideal, and in particular that $\mathcal{R}_q$ is isomorphic to the direct product of n copies of $\mathbb{Z}_q$.

The *k-P-R-ISIS* assumption

Our construction relies on a particular assumption in the k-R-ISIS assumption family and its knowledge counterpart, parameterized by the monomial sets $[X^i]_{i\in\mathbb{Z}(w)\{0\}}$ and $[X^i]_{i\in[w]}$, respectively. This assumption could be seen as a lattice analogue of the k-Strong-Diffie-Hellman (k-SDH) problem, and we refer to it as the k-Powers R-ISIS (k-P-R-ISIS) assumption.

Assumption 2 (k-P-R-ISIS assumption)**.** *Define* k-P-R-$\mathsf{ISIS}_{\mathcal{R}_q,,\mathcal{D},\beta,\beta^*} := k$-$R$-$\mathsf{ISIS}_{\mathcal{R}_q,,y,\mathcal{G},g^*,\mathcal{D},\beta,\beta^*}$ *with fixed* $\mathcal{G} = [X^i]_{i\in\mathbb{Z}(k)\{0\}}$, $g^*(X) = 1$, *and consequently* $y = 1$. *The* k-P-R-ISIS *assumption is hard if* R-$\mathsf{ISIS}_{\mathcal{R}_q,,\beta^*}$ *is.*

In the discrete log setting, this is akin to asking an adversary to compute g given $(g^{x^{w-1}}, \dots, g^{x^{-1}}, g^x, \dots, g^{x^w})$. Since $(\mathcal{G}, g^*)$ is k-M-ISIS admissible (definition 4.2.11), our assumption follows from the much broader meta assumption of Lai et al. (Assumption 1). We also require the following knowledge assumption:

Assumption 3 (Knowledge k-P-R-ISIS assumption)**.** *Define knowledge* k-P-R-$\mathsf{ISIS}_{\mathcal{R}_q,,\mathcal{D},\mathcal{T},\alpha^*,\beta,\beta^*}$ *as knowledge* k-R-$\mathsf{ISIS}_{\mathcal{R}_q,,y,\mathcal{G},\mathcal{D},\mathcal{T},\alpha^*,\beta,\beta^*}$ *with* $\alpha^* \geq \beta^* \geq 1$, *fixed* $\mathcal{G} = [X^i]_{i\in[k]}$, *and consequently* $y = 1$. *The knowledge* k-P-R-ISIS *assumption holds if* R-$\mathsf{ISIS}_{\mathcal{R}_q,,\beta^*}$ *does.*

4.3 Cryptanalysis of k-P-R-ISIS

In this section, we consider how the k-P-R-ISIS assumption may be attacked. We begin by formulating a special case of R-SIS assumption when $\mathbf{a}$ is generated by the powers of a single ring element, which we call ring Vandermonde SIS (R-V-SIS). Then we show how an attacker who efficiently solves R-V-SIS can break the security of Orbweaver.

Definition 4.3.1. (R-V-SIS) Let $\mathcal{R}, q, \beta_V$ be parameters depending on λ and w. The R-V-SIS problem asks the following: given $\mathbf{v} := [v^i]_{i\in\mathbb{Z}(w)\{0\}} \in \mathcal{R}_q^{2w-3}$ for $v \leftarrow\$ \mathcal{R}_q^\times$, find $(s, \mathbf{z}) \neq (0, \mathbf{0}) \in \mathcal{R} \times \mathcal{R}^{2w-3}$ such that $0 < \|s\| \leq \beta_V, \|\mathbf{z}\| \leq \beta_V$, and $\langle \mathbf{v},\ \mathbf{z}\rangle \equiv s \bmod q$.

We note that similar assumptions have been proposed by prior works (see [**BSS22**] for a

summary). Thus far, there are no known algorithms which solve these variants faster than their non-Vandermonde counterparts.

We now show how an adversary who breaks R-V-SIS can break k-P-R-ISIS. Since we are given $\mathbf{a} \cdot \mathbf{u}_i \equiv v^i \bmod q$ for all $i \in \mathbb{Z}(w) \ \{0\}$, and our goal is to find short $(s^*, \mathbf{u}_{g^*}) \neq (0, \mathbf{0})$ such that $\mathbf{a} \cdot \mathbf{u}_{g^*} \equiv s^* \cdot v^0 \bmod q$. We first solve R-V-SIS, obtaining $s, \mathbf{z}$ satisfying the constraints in definition 4.3.1. Then, we let $\mathbf{u}_{g^*} = \sum_{i \in \mathbb{Z}(w)\{0\}} z_i \cdot \mathbf{u}_i$ and $s^* = s$. We observe that $\langle \mathbf{a},\ \mathbf{u}_{g^*} \rangle \equiv \sum_{i \in \mathbb{Z}(w)\{0\}} z_i \cdot \langle \mathbf{a},\ \mathbf{u}_i \rangle \equiv s^* v^0 \bmod q$.

Notice, however, we do get some blowup in the norm, where now $\|\mathbf{u}_{g^*}\| \leq (2w-3) \cdot \beta_V \cdot \beta \cdot \gamma_{\mathcal{R}}$. So we must solve R-V-SIS for a β_V such that $(2w-3) \cdot \beta_V \cdot \beta \cdot \gamma_{\mathcal{R}} \leq \beta^*$.

In [**C:ACLMT22**], they generalize this attack, requiring the attacker find (not necessarily short) $\mathbf{z}$ and short s^* satisfying $\langle \mathbf{a},\ \mathbf{u}_{g^*} \rangle \equiv s^* v^0 \bmod q$ and $\|\mathbf{u}_{g^*}\| \leq \beta^*$ for $\mathbf{u}_{g^*} = \sum_{i \in \mathbb{Z}(w)\{0\}} z_i \cdot \mathbf{u}_i$. This attack applies to our scheme as well. See Section 4.1 of their paper for details.

Setting parameters according to the direct SIS attack. Given there are no specific algorithms for the above problems, when picking parameters we consider the cost of a direct SIS attack that ignores the preimages and their algebraic dependencies. The following analysis is standard. We first reduce R-ISIS to R-SIS: recall that to break the assumption, the attacker needs to find some $\mathbf{u}_g$ such that $\langle \mathbf{a},\ \mathbf{u}_g \rangle \equiv s^* \bmod q$; instead, we ask the attacker to find $\mathbf{u}'_g$ such that $\langle \mathbf{a}',\ \mathbf{u}'_g \rangle \equiv 0 \bmod q$, where $\mathbf{a}' := \mathbf{a} \| -1$. Then, here the last entry of $\mathbf{u}'_g$ becomes s^*.

We then view this as a SIS instance: for $\mathbf{A} \in \mathbb{Z}_q^{n \times n(+1)}$ find $\mathbf{u} \in \mathbb{Z}_q^{n(+1)}$ such that $\mathbf{A} \cdot \mathbf{u} \equiv 0 \bmod q$. Let $\mathbf{L} \in \mathbb{Z}_q^{n(+1) \times n}$ be a basis for the right kernel of $\mathbf{A}$. Then solving this SIS instance is equivalent to finding a short vector in $\Lambda(\mathbf{L})$. Observe that

$$\mathbf{L}' = \left[\begin{array}{c|c} L & \begin{matrix} 0 \\ qI_n \end{matrix} \end{array}\right]$$

is an equivalent basis for this lattice. If we compute the Gram–Schmidt format of $\mathbf{L}'$, we get the form $\left[\begin{array}{c|c} I & \begin{matrix} 0 \\ qI_n \end{matrix} \\ \tilde{\mathbf{L}} & \end{array}\right]$ for some $\tilde{\mathbf{L}} \in \mathbb{Z}_q^{n(+1) \times n}$.

Thus, it is easy to compute the volume $\mathsf{Vol}(\Lambda) = \prod_{i \in [n(+1)]} \|b_i\|_\infty = q^n$, where $[b_1, \ldots, b_{n(+1)}] = \tilde{\mathbf{L}}$. Thus, the goal is to find a vector in $\Lambda(\mathbf{L})^\top$ in a $d \leq n \cdot (\ell + 1)$ dimensional lattice with volume q^n.

We thus consider the adversary wins if it finds a vector with ℓ_2 norm of $\sqrt{d} \cdot \beta^*$, where β^* is the ℓ_∞ norm we allow in our hardness assumption.

Recall that lattice reduction with lattice parameter n (i.e., our ring degree) returns a vector with ℓ_2 norm $\approx \delta^d \cdot \mathsf{Vol}(\Lambda)^{1/d}$, where δ is the root Hermite factor. With the state-of-the-art lattice reduction algorithm [**SODA:BDGL16**], to achieve $\delta \approx (\frac{n}{2\pi e})^{1/(2n)}$, it takes roughly $2^{0.292n+o(n)}$ time. When $d \approx \sqrt{n \cdot log(q)/\log(\delta)}$, the norm of the vector found (i.e., $\delta^d \cdot \mathsf{Vol}(\Lambda)^{1/d}$) is minimized [**MR09**].

Thus, we can simply set $\delta^d \cdot \mathsf{Vol}(\Lambda)^{1/d} \geq \sqrt{d} \cdot \beta^*$ for the d that minimizes the left hand side, and $2^{0.292n+o(n)} \geq 2^\lambda$. Thus, when we set parameters for R-SIS problem with parameter $\mathcal{R}, q, \ell, \beta^*$ (where $\mathcal{R}$ has ring degree n), essentially, we are requiring that the adversary cannot find a vector with ℓ_2 norm $\leq \sqrt{n \cdot log(q)/\log(\delta)} \cdot \beta^*$ in $\leq 2^\lambda$ time using the attack above, except with neglgible probability. [3]

4.4 Orbweaver: linear functional commitments for rings

We present Orbweaver, a vector commitment (VC) for linear functions that is non-interactive, publicly verifiable, preprocessing, and structure-preserving. Together, these features enable efficient recursive composition of our construction.

Orbweaver supports opening committed vectors with respect to the function family $\mathcal{F} = \{\mathcal{F}_w\}_{w \in \mathbb{N}}$, where $\mathcal{F}_w = \mathcal{X}^w$ and $f \in \mathcal{F}_w$ is computed as $f(\mathbf{x}) \equiv \langle [f_i]_{i=1}^w, \mathbf{x} \rangle \bmod q$. The input alphabet $\mathcal{X} = \{x \in \mathcal{R} \mid \|x\| \leq \alpha\}$ is specified by the ring $\mathcal{R}$ and norm bound α. Let $\delta_M = w \cdot \alpha^2 \cdot \gamma_\mathcal{R}$, where $\gamma_\mathcal{R}$ is the expansion factor of $\mathcal{R}$ (see definition 4.2.1). The image space $\mathcal{Y}$ is then $\mathcal{Y} = \{y \in \mathcal{R} \mid \|y\| \leq \delta_M\}$.

Let $P := \mathbb{Z}(w) \setminus \{0\}$. We use the monomial set $\mathcal{G}_0 := [g_i(X) = X^i]_{i \in P}$ to generate our opening proof $[\mathbf{u}_{0,i}]_{i \in P}$, the monomial set $\mathcal{G}_1 := [g_i(X) = X^i]_{i \in [w]}$ to generate our knowledge proof SRS $[\mathbf{u}_{1,i}]_{i \in [w]}$, the target monomial $g^*(X) := 1$, and the distribution $\mathcal{T}$ satisfying the requirements of Assumption 3.

[3]Note that there are known poly-time attacks against some parameter selection for R-SIS, e.g., [**EC:PXWC21**]. Thus, we also need to avoid those parameter selections. In more detail, we follow what is suggested in [**C:ACLMT22**] and pick q such that $\mathcal{R}_q$ fully splits and pick t as specified in definition 4.2.15, which makes the attack from [**EC:PXWC21**] not work.

Table 4.1: A list of parameters used in Orbweaver.

$w \in \mathbb{N}$		Dimension of secret input $\mathbf{x}$
$n \in \mathbb{N}$		Degree of $\mathcal{R}$
$\alpha \in \mathbb{R}$	$\text{poly}(\lambda)$	Norm bound for $\mathbf{x}$ and f
$\beta_0 \in \mathbb{R}$	$\text{poly}(\lambda)$	Norm bound for public preimages $\mathbf{u}_{0,\cdot}$ for π_0
$\beta_1 \in \mathbb{R}$	$\text{poly}(\lambda)$	Norm bound for public preimages $\mathbf{u}_{1,\cdot}$ for π_1
$\delta_0 \in \mathbb{R}$	$w^2 \cdot \alpha^2 \cdot \beta_0 \cdot \gamma_{\mathcal{R}}^2$	Norm bound for opening proof π_0
$\delta_1 \in \mathbb{R}$	$w \cdot \alpha \cdot \beta_1 \cdot \gamma_{\mathcal{R}}$	Norm bound for commitment knowledge proof π_1
$\delta_M \in \mathbb{R}$	$w \cdot \alpha^2 \cdot \gamma_{\mathcal{R}}$	Norm bound for evaluation of a linear function with coefficients of norm bounded by α at a point of norm bounded by α
$q \in \mathbb{R}$		Modulus for $\mathcal{R}_q$
$_i \in \mathbb{N}$	$\geq \mathsf{lhl}(\mathcal{R}_q, \beta_i)$	Number of ring elements in $\mathbf{a}$
$\gamma_{\mathcal{R}}$		Ring expansion factor of $\mathcal{R}$
$\mathcal{X}$	$\{x \in \mathcal{R} : \lVert x \rVert \leq \alpha\}$	$\mathcal{R}$ elements with norm bound α
$\mathcal{F}$		family of w-variate linear functions over $\mathcal{X}$

$\underline{\mathsf{Setup}(1^\lambda, 1^w) \to \mathsf{ck}}$
$v \leftarrow\$ \mathcal{R}_q^\times;\ \ t \leftarrow\$ \mathcal{T}$
$(\mathbf{a}_0, \mathsf{td}_0) \leftarrow \mathsf{TrapGen}(1, 1, \mathcal{R}_q, \beta_0);\ \ (\mathbf{a}_1, \mathsf{td}_1) \leftarrow \mathsf{TrapGen}(1, 1, \mathcal{R}_q, \beta_1)$
$\mathbf{u}_{0,i} \leftarrow \mathsf{SampPre}(\mathsf{td}_0, v^i, \beta_0),\ \ \forall i \in P;\ \ \mathbf{u}_{1,i} \leftarrow \mathsf{SampPre}(\mathsf{td}_1, v^i \cdot t, \beta_1),\ \ \forall i \in [w]$
Return $\mathsf{ck} := (\mathbf{a}_0, [\mathbf{u}_{0,i}]_{i \in P}, \mathbf{a}_1, t, [\mathbf{u}_{1,i}]_{i \in [w]}, v)$

$\underline{\mathsf{Com}(\mathsf{ck}, \mathbf{x}) \to (c, \pi_1)}$
$c := \sum_{i=1}^{w} x_i v^i \bmod q$
$\pi_1 := \sum_{i=1}^{w} x_i \cdot \mathbf{u}_{1,i} \bmod q$
Return (c, π_1)

$\underline{\mathsf{Open}(\mathsf{ck}, f, \mathbf{x}) \to \pi_0}$
Let $a_{-w+1}, ..., a_{w-1}$ denote the $2w-1$ coefficients of $\mathbf{x}(v)f(v) := \left(\sum_{i=1}^{w} x_i v^i\right)\left(\sum_{i=1}^{w} f_i v^{-i}\right)$
Return $\pi_0 := \sum_{i \in P} a_i \cdot \mathbf{u}_{0,i} \bmod q$

$\underline{\mathsf{PreVerify}(\mathsf{ck}, f) \to \mathsf{vk}_f}$
If $\lVert f \rVert > \alpha$, abort
Return $\mathsf{vk}_f := \sum_{i=1}^{w} f_i \cdot v^{-i} \bmod q$

$\underline{\mathsf{Verify}(\mathsf{vk}_f, c, \pi_1, y, \pi_0) \to \{0, 1\}}$
Output 1 if the following conditions all hold (else 0):
$\lVert y \rVert_\infty \leq \delta_M;\ \ \lVert \pi_1 \rVert_\infty \leq \delta_1;\ \ \lVert \pi_0 \rVert_\infty \leq \delta_0$
$\langle \mathbf{a}_1,\ \pi_1 \rangle \equiv c \cdot t \bmod q$
$\langle \mathbf{a}_0,\ \pi_0 \rangle \equiv \mathsf{vk}_f \cdot c - y \bmod q$

We next prove that Orbweaver satisfies correctness, extractability, and succinctness.

Theorem 4.4.1. *Orbweaver is correct (definition 4.2.7) for $\delta_1 \geq w \cdot \alpha \cdot \beta_0 \cdot \gamma_{\mathcal{R}}$ and $\delta_0 \geq w^2 \cdot \alpha^2 \cdot \beta_1 \cdot \gamma_{\mathcal{R}}^2$.*

Proof. We begin by proving the norm bound checks hold. $y = f(\mathbf{x}) = \sum_{i=1}^{w} f_i \cdot x_i \bmod q$, where $\|f_i \cdot x_i\| \leq \alpha^2 \cdot \gamma_{\mathcal{R}}$ for all i. Therefore, $\|y\| \leq w \cdot \alpha^2 \cdot \gamma_{\mathcal{R}} = \delta_M$. Next, $\pi_1 = \sum_{i=1}^{w} x_i \cdot \mathbf{u}_{1,i} \bmod q$, where $\|x_i \cdot \mathbf{u}_{1,i}\| \leq \alpha \cdot \beta_1 \cdot \gamma_{\mathcal{R}}$ for all i. It follows $\|\pi_1\| \leq w \cdot \alpha \cdot \beta_1 \cdot \gamma_{\mathcal{R}} = \delta_1$. Last, $\pi_0 = \sum_{i \in P} a_i \cdot \mathbf{u}_{0,i} \bmod q$, where $a_{-w+1}, ..., a_{w-1}$ denote the $2w-2$ coefficients of $\mathbf{x}(v)f(v) = \left(\sum_{i=1}^{w} x_i v^i\right)\left(\sum_{i=1}^{w} f_i v^{-i}\right)$. Since $\|\mathbf{u}_{0,i}\| \leq \beta_0$ for all i, $\|\pi_0\| \leq \beta_0 \cdot \gamma_{\mathcal{R}} \cdot \sum_{i \in P} a_i$. Then $\left\|a_{|i|}\right\| \leq (w-i) \cdot \alpha^2 \cdot \gamma_{\mathcal{R}}$ for $i \in [w-1]$. Thus, $\|\pi_0\| \leq w^2 \cdot \alpha^2 \cdot \beta_0 \cdot \gamma_{\mathcal{R}}^2 = \delta_0$.

Next, we show the verification equations hold:

$$\begin{aligned}
\langle \mathbf{a}_1,\ \pi_1 \rangle &\equiv \langle \mathbf{a}_1,\ \textstyle\sum_{i=1}^{w} x_i \cdot \mathbf{u}_{1,i} \rangle \bmod q \\
&\equiv \textstyle\sum_{i=1}^{w} x_i \cdot \langle \mathbf{a}_0,\ \mathbf{u}_{1,i} \rangle \bmod q \\
&\equiv \textstyle\sum_{i=1}^{w} x_i \cdot v^i \cdot t \bmod q \\
&\equiv c \cdot t \bmod q
\end{aligned}
\qquad
\begin{aligned}
\langle \mathbf{a}_0,\ \pi_0 \rangle &\equiv \langle \mathbf{a}_0,\ \textstyle\sum_{i \in P} a_i \cdot \mathbf{u}_i \rangle \bmod q \\
&\equiv \textstyle\sum_{i \in P} a_i \cdot v^i \bmod q \\
&\equiv \left(\textstyle\sum_{i=1}^{w} x_i v^i\right)\left(\textstyle\sum_{i=1}^{w} f_i v^{-i}\right) - a_0 \cdot v^0 \bmod q \\
&\equiv \mathsf{vk}_f \cdot c - \textstyle\sum_{i=1}^{w} f_i \cdot x_i \equiv \mathsf{vk}_f \cdot c - y \bmod q\ .
\end{aligned}$$

□

Theorem 4.4.2. *Let $\mathcal{X}^* := \{x \in \mathcal{R} \mid \|x\| \leq \alpha^*\}$. Orbweaver is $\mathcal{X}^*$-extractable if*

$$\begin{aligned}
&_0 \geq \mathsf{lhl}(\mathcal{R}_q, b_0),\ \ _1 \geq \mathsf{lhl}(\mathcal{R}_q, b_1) \\
&\alpha^* \geq \beta_1^* \geq \delta_1 \\
&\beta_0^* \geq 2 \cdot w^2 \cdot \alpha \cdot \alpha^* \cdot \beta_0 \cdot \gamma_{\mathcal{R}}^2
\end{aligned}$$

where $b_0 = O(\beta_0), b_1 = O(\beta_1)$, and the k-R-$\mathsf{ISIS}_{\mathcal{R}_q,0,1,\mathcal{G}_0,g_0^*,\mathcal{D}_0,\beta_0,\beta_0^*}$ (i.e., k-P-R-ISIS) and knowledge k-R-$\mathsf{ISIS}_{\mathcal{R}_q,1,1,\mathcal{G}_1,\mathcal{D}_1,\mathcal{T},\alpha^*,\beta_1,\beta_1^*}$ (i.e., knowledge k-P-R-ISIS) assumptions hold, where $\mathcal{D}_0$ and $\mathcal{D}_1$ are such that

$$\left\{ (\mathbf{a}_0, [\mathbf{u}_{0,g}]_{g \in \mathcal{G}_0}, v) \;\middle|\; \begin{array}{l} \mathbf{a}_0 \leftarrow\$\ \mathcal{R}_q^{\ell_0};\ \ v \leftarrow\$\ \mathcal{R}_q^{\times} \\ \mathbf{u}_{0,g} \leftarrow\$\ \mathcal{D}_{g,\mathbf{a}_0,v},\ \ \forall i \in P \end{array} \right\}$$
$$\approx \left\{ (\mathbf{a}_0, [\mathbf{u}_{0,g}]_{g \in \mathcal{G}_0}, v) \;\middle|\; \begin{array}{l} \mathbf{a}_0 \leftarrow\$\ \mathcal{R}_q^{\ell_0};\ \ v \leftarrow\$\ \mathcal{R}_q^{\times} \\ \mathbf{u}_{0,g} \leftarrow\$\ \mathsf{SampD}(1^{\ell_0}, \mathcal{R}_q, \beta_0) : \langle \mathbf{a}_0,\ \mathbf{u}_{0,g} \rangle \equiv v^i \bmod q,\ \ \forall i \in P \end{array} \right\}$$

and

$$\left\{ (\mathbf{a}_1, t_i, [\mathbf{u}_{1,g}]_{g \in \mathcal{G}_1}, v) \;\middle|\; \begin{array}{l} \mathbf{a}_1 \leftarrow\$\ \mathcal{R}_q^{\ell_1};\ \ t \leftarrow\$\ \mathcal{T};\ \ v \leftarrow\$\ \mathcal{R}_q^{\times} \\ \mathbf{u}_{1,g} \leftarrow\$\ \mathcal{D}_{g,\mathbf{a}_1,t,v},\ \ \forall i \in [w] \end{array} \right\}$$
$$\approx \left\{ (\mathbf{a}_1, t_i, [\mathbf{u}_{1,g}]_{g \in \mathcal{G}_1}, v) \;\middle|\; \begin{array}{l} \mathbf{a}_1 \leftarrow\$\ \mathcal{R}_q^{\ell_1};\ \ t \leftarrow\$\ \mathcal{T};\ \ v \leftarrow\$\ \mathcal{R}_q^{\times} \\ \mathbf{u}_{1,g} \leftarrow\$\ \mathsf{SampD}(1^{\ell_1}, \mathcal{R}_q, \beta_1) : \langle \mathbf{a}_1,\ \mathbf{u}_{1,g} \rangle \equiv v^i \cdot t \bmod q,\ \ \forall i \in [w] \end{array} \right\}.$$

Proof. Suppose there exists a PPT adversary $\mathcal{A}$ that, on input $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda, 1^w)$, outputs (f, c, y, π_1, π_0) such that $\mathsf{PreVerify}(\mathsf{ck}, f)$ does not abort and $\mathsf{Verify}(\mathsf{vk}_f, c, y, \pi_1, \pi_0) = 1$ with non-negligible probability in λ over any randomness it uses and the choice of ck. We construct an extractor $\mathcal{E}_\mathcal{A}$ that, input ck and given black-box access to $\mathcal{A}$ and any randomness it uses, returns $\mathbf{x}^* \in (\mathcal{X}^*)^w$ such that $c = \mathsf{Com}(\mathsf{ck}, \mathbf{x}^*)$ and $f(\mathbf{x}^*) = y$ with all but neglibible probability in λ when $\mathcal{A}$ succeeds.

For commitment key $\mathsf{ck} = (\mathbf{a}_0, [\mathbf{u}_{0,i}]_{i \in P}, \mathbf{a}_1, t, [\mathbf{u}_{1,i}]_{i \in [w]}, v)$, define $\mathsf{ck}_0 := (\mathbf{a}_0, [\mathbf{u}_{1,i}]_{i \in [w]}, v)$ and $\mathsf{ck}_1 := (\mathbf{a}_1, t, [\mathbf{u}_{1,i}]_{i \in [w]}, v)$. Let $\mathcal{B}_\mathcal{A}$ be an adversary with black-box access to $\mathcal{A}$ that runs on input ck_1' sampled according to the knowledge k-R-$\mathsf{ISIS}_{\mathcal{R}_{q'},1,\mathcal{G}_1,\mathcal{D}_1,\mathcal{T},\alpha^*,\beta_1,\beta_1^*}$ definition (definition 4.2.15). First, $\mathcal{B}_\mathcal{A}$ samples ck_0' according to the k-R-$\mathsf{ISIS}_{\mathcal{R}_{q'},0,1,\mathcal{G}_0,g_0^*,\mathcal{D}_0,\beta_0,\beta_0^*}$ definition (definition 4.2.13), except fixing $v' = v$. Then $\mathcal{B}_\mathcal{A}$ runs $\mathcal{A}$ on $(\mathsf{ck}_0', \mathsf{ck}_1)$ (passing its own random input tape) to obtain (f, c, y, π_1, π_0), which it parses to output (c, π_1).

Let $\mathcal{E}_{\mathcal{B}_\mathcal{A}}$ be the extractor promised by our knowledge assumption for $\mathcal{B}_\mathcal{A}$. Extractor $\mathcal{E}_\mathcal{A}$, on input $\mathsf{ck} = (\mathsf{ck}_0, \mathsf{ck}_1)$, runs $\mathcal{E}_{\mathcal{B}_\mathcal{A}}(\mathsf{ck}_1)$ to obtain $\mathbf{x}^*$. Next consider the following hybrid experiments:

1. Hyb_0: the real experiment, where $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda, 1^w)$, $(f, c, y, \pi_1, \pi_0) \leftarrow \mathcal{A}(\mathsf{ck})$, and $\mathbf{x}^* \leftarrow \mathcal{E}_\mathcal{A}(\mathsf{ck})$.
2. Hyb_1: the same as Hyb_0, but $\mathsf{ck} = (\mathsf{ck}_0, \mathsf{ck}_1')$, where ck_1' is sampled according to the knowledge k-R-$\mathsf{ISIS}_{\mathcal{R}_{q'},1,\mathcal{G}_1,\mathcal{D}_1,\mathcal{T},\alpha^*,\beta_1,\beta_1^*}$ definition, conditioned on $v' = v$.
3. Hyb_2: the same as Hyb_1, but $\mathsf{ck} = (\mathsf{ck}_0'', \mathsf{ck}_1')$, where ck_1' is sampled as in Hyb_1, but ck_0'' is sampled according to the k-R-$\mathsf{ISIS}_{\mathcal{R}_{q'},0,1,\mathcal{G}_0,g_0^*,\mathcal{D}_0,\beta_0,\beta_0^*}$ definition, again conditioned on $v'' = v'$.

By our assumption on $\mathcal{D}_1$ (resp., $\mathcal{D}_0$) hybrids 0 and 1 (resp., 1 and 2) are statistically close. Therefore, they are all statistically close. This implies if a property holds with respect to the output of one hybrid with a certain probability, it will hold with respect to the output of the others with statistically close probability.

From our knowledge assumption, we know that in Hyb_1 the extractor outputs $\mathbf{x}^*$ satisfying $c \equiv \sum_{i=1}^w x_i \cdot v^i \bmod q$ and $\|x_i\| \leq \alpha^*$ for all $i \in [w]$ with probability all but negligible in λ. Now consider the output in Hyb_2 satisfies $c \equiv \sum_{i=1}^w x_i \cdot v^i \bmod q$ for $\mathbf{x}^* \in \mathcal{X}^*$, but $f(\mathbf{x}^*) \neq y$. When this happens we can break k-R-$\mathsf{ISIS}_{\mathcal{R}_{q'},0,1,\mathcal{G}_0,g_0^*,\mathcal{D}_0,\beta_0,\beta_0^*}$ as follows. Let $y^* := f(\mathbf{x}^*)$ and

$\pi_0^* \leftarrow \mathsf{Open}(\mathsf{ck}, f, \mathbf{x}^*)$. Then we obtain the solution $(y^* - y, \pi_0 - \pi_0^*)$ for k-R-ISIS, where

$$\langle \mathbf{a}_0,\ \pi_0 - \pi_0^* \rangle \equiv \mathsf{vk}_f \cdot c - y - (\mathsf{vk}_f \cdot c - y^*) \equiv (y^* - y) \cdot 1 \equiv (y^* - y) \cdot g^*(v) \bmod q\ .$$

To see that the norm bounds hold, we can bound $\delta_M^* := \|y^*\| \leq w\alpha^*\alpha\gamma_{\mathcal{R}}$ and $\delta_0^* := \|\pi_0^*\| \leq w^2\alpha^*\alpha\beta_0\gamma_{\mathcal{R}}^2 \leq \beta_0^*/2$ by modifying the calculations of δ_M and δ_0 in the correctness proof. Therefore, this can only happen with negligible probability in λ. Together, these two properties about the results of Hyb_1 and Hyb_2 show that $\mathsf{Orbweaver}$ is $\mathcal{X}^*$-extractable. □

Theorem 4.4.3. *Orbweaver is succinct (definition 4.2.9).*

Proof. We first show $\mathsf{Orbweaver}$ has $O(\log w + \log \alpha)$ size commitments and proofs. From our construction we have that $|c| = n\lceil \log(q) \rceil$, $|\pi_0| = n_0 \lceil \log \delta_0 \rceil$, and $|\pi_1| = n_1 \lceil \log \delta_1 \rceil$. We set a base $b_0 = O(\beta_0), b_1 = O(\beta_1)$ for trapdoor generation and let ${}_0 := \mathsf{lhl}(\mathcal{R}_q, b_0) = O(\log(q)/\log(\beta_0)))$, ${}_1 := \mathsf{lhl}(\mathcal{R}_q, b_1) = O(\log(q)/\log(\beta_1))$. We consider the ring degree n to be a function of the security parameter and treat it as a constant for the following analysis.

As implicated by our extractability theorem (theorem 4.4.2) together with our assumptions relating the hardness of plain and knowledge k-P-R-ISIS to R-SIS (Assumptions 2 and 3), we must have that $R\text{-}\mathsf{SIS}_{\mathcal{R}_q,\ell,\beta_0^*}$ and $R\text{-}\mathsf{SIS}_{\mathcal{R}_q,\ell,\delta_1}$ are hard. Recall $\beta_0^* = 2w^2\alpha\alpha^*\beta_0\gamma_{\mathcal{R}}^2$. Using $\alpha^* = \delta_1 = w\alpha\beta_1\gamma_{\mathcal{R}}$, we can write $\beta_0^* = 2w^3\alpha^2\beta_0\beta_1\gamma_{\mathcal{R}}^3$. Since $\delta_1 < \beta_0^*$, we can just pick q s.t. $R\text{-}\mathsf{SIS}_{\mathcal{R}_q,\ell,\beta_0^*}$ is hard.

In particular, we let $q \approx \beta_0^* n \log(n)$ such that $\log(q) = O(\log \beta_0^*) = O(\log(w) + \log(\alpha) + \log(\beta_0) + \log(\beta_1))$ since for power-of-two cyclotomic rings $\gamma_{\mathcal{R}} \leq n$ (theorem 4.2.2). Setting $\beta_0 := \max\{w, \alpha\}$ and $\beta_1 := \max\{w, \alpha\}$, we obtain logarithmic commitment and proof sizes:

$$
\begin{aligned}
|c| &= n \lceil \log(q) \rceil \\
&\approx n \log(n \log(n) \beta_0^*) \\
&= O(\log \beta_0^*) \\
&= O(\log w + \log \alpha)
\end{aligned}
$$

$$
\begin{aligned}
|\pi_1| &= n_0 \lceil \log \delta_1 \rceil \\
&= 2 \lceil \log(q) / \log(\beta_1) \rceil \, n \lceil \log \delta_1 \rceil \\
&\approx 2n \log(\beta_0^* n \log n) \log \delta_1 / \log \beta_1 \\
&= O(\log \beta_0^* \cdot \log \delta_1 / \log \beta_1) \\
&= O(\log w + \log \alpha)
\end{aligned}
$$

$$
\begin{aligned}
|\pi_0| &= n_0 \lceil \log \delta_0 \rceil \\
&= 2 \lceil \log(q) / \log(\beta_0) \rceil \, n \lceil \log \delta_0 \rceil \\
&\approx 2n \log(\beta_0^* n \log n) \log \delta_0 / \log \beta_0 \\
&= O(\log(\beta_0^*) \cdot \log(\delta_0) / \log \beta_0) \\
&= O(\log w + \log \alpha)
\end{aligned}
$$

Next, we show that the Orbweaver verifier runs in time $O(\log(w\alpha) \log\log(w\alpha))$ time. The verifier work is dominated by computing the inner products $\langle \mathbf{a}_1, \ \pi_1 \rangle$ and $\langle \mathbf{a}_0, \ \pi_0 \rangle$, where $|\mathbf{a}_i| = \ell_i$. This work is dominated by $\ell_0 + \ell_1$ multiplications in the ring $\mathcal{R}_q$ of degree n. Using the NTT, this can be computed with $(\ell_0 + \ell_1)(n \log n)$ multiplications in $\mathbb{Z}_q$. Though since we've defined n in terms of λ, which itself is treated as a constant in our asymptotic analysis, we can simplify this to $O(\ell_0 + \ell_1) = O(\log q / \log \beta_0 + \log q / \log \beta_1) = O(\log \beta_0^* / \log \beta_0 + \log \beta_0^* / \log \beta_1) = O((\log w + \log \alpha) / \log \beta_0 + (\log w + \log \alpha) / \log \beta_1) = O(1)$ operations over the ring $\mathbb{Z}_q$, where elements have bit length $\log q = O(\log w + \log \alpha)$. Asymptotically, $\mathbb{Z}_q$ multiplications can be performed in $\log(q) \log\log(q)$ time [**Harvey2021IntegerMI**], giving verifier complexity is $O(\log(w\alpha) \log\log(w\alpha))$. [4] □

4.4.1 Extensions

This section discuss extensions to Orbweaver including support for proof aggregation, an inner product argument, and recursive proof composition. Even if the reader is only interested in using Orbweaver just for ring relations, we suggest looking at the section on linear functional commitments for $\mathbb{Z}_M$ (section 4.4.2) anyway as many of the optimizations introduced there including ternary decomposition can be applied in the ring setting as well.

Universal SRS. In order to support all instances of length $w \leq W$, observe it suffices to run the protocol without modification using only powers in $\mathbb{Z}(w)$. Since parameters are set such that R-SIS

[4] We note [**Harvey2021IntegerMI**] is a galactic algorithm, i.e., it is only efficient at instance sizes so large it is never used in practice. For the integer sizes we care about (several thousand bits) recent works have shown NTT-based approaches, with complexity $O(\log w \log\log w \log\log\log w)$, concretely most efficient [**bhkpy22**].

is hard for β_0^* based on W, it is not an issue that the extractor outputs an input of length W.

Public proof aggregation. Orbweaver can be extended to support public proof aggregation (i.e., the aggregator can be an untrusted third party) by taking advantage of the linearly homomorphic property of the opening proofs. Using short challenges $[[h_{i,j}]_{i=1}^{t}]_{j=1}^{T[i]}$ we can check that commitments $[c_i]_{i=1}^{t}$, evaluated at function sets $[f_j]_{j=1}^{T[i]}$, output values $[[y_{i,j}]_{i=1}^{t}]_{j=1}^{T[i]}$, respectively. The verifier checks the opening proof equation

$$\langle \mathbf{a},\ \pi_0 \rangle \equiv \sum_{i=1}^{t} \sum_{j=1}^{T[i]} h_{i,j} \cdot \left(c_i \cdot \mathsf{ck}_{f_{i,j}} - y_{i,j} \right) \bmod q \ , \text{ where } \ \pi_0 = \sum_{i=1}^{t} \sum_{j=1}^{T[i]} h_{i,j} \pi_{0,i,j} \mod q$$

We give an instantiation of $\mathcal{H}$ from exceptional sets over $\mathcal{R}_q$ in section 4.2.1. With this challenge set, the protocol becomes interactive: the aggregator sends $[(c_i, \pi_{1,i}, [y_{i,j}]_{j=1}^{T[i]})]_{i=1}^{t}$, the verifier samples $[h_{i,j}]_{i=1,j=1}^{t,\ T[i]} \leftarrow\$ \mathcal{H}$, and the aggregator responds with π_0. Using an R-SIS instance as a fixed challenge string as introduced in [**C:ACLMT22**] provides a way to maintain non-interactivity, while using an exceptional set provides better performance.

To see how security holds, after using our knowledge assumption extractor to obtain the $[\mathbf{x}_i^*]_{i=1}^{t}$, we have by k-P-R-ISIS that

$$\sum_{i=1}^{t} \sum_{j=1}^{T[i]} h_{i,j} \cdot \left(\langle f_{i,j},\ \mathbf{x}_i^* \rangle - y_{i,j} \right) \equiv 0 \bmod q$$

When interactively sampling challenges from an exceptional set $\mathcal{H}$ over $\mathcal{R}_q$, we obtain that $\langle f_{i,j},\ \mathbf{x}_i^* \rangle = y_{i,j}$ for all i, j except with probability $1/|\mathcal{H}|$ by theorem 4.2.4. If instead we use a fixed R-SIS instance $\mathbf{h} \leftarrow\$ \mathcal{R}_p$ for $\alpha \ll p \ll q$, then if $\langle f_{i,j},\ \mathbf{x}_i^* \rangle \neq y_{i,j}$ for some i, j, we obtain a non-trivial R-SIS solution.

Observe that because we extract the $\mathbf{x}_i^*$ directly from the knowledge proofs, we do not need to extract the hypothetical $\pi_{0,i,j}$ inside π_0 for our security reduction. This lets use an exceptional set over $\mathcal{R}_q$, which can be exponential in size (see section 4.2.1 for an explicit construction). We can then invoke the Alon–Füredi Theorem theorem 4.2.4 to say the adversary succeeds with. On the other hand, when extraction is required current techniques rely on using exceptional sets over $\mathcal{R}$,

and there's evidence such sets are all polynomial-sized for rings of interest [**C:AlbLai21**].

When using exceptional sets our interactive protocol is public-coin and we may therefore apply the Fiat–Shamir transform to once again obtain a non-interactive protocol. The interactive protocol is round-by-sound since Σ-protocols are, and since its relation (finding a non-zero polynomial such that the hash of its commitment is one of its zeros) is sparse, the non-interactive version is secure in the random oracle model [**canettichlrr18**]. Security in the standard model is also possible if there is a correlation intractable hash function family for the relation.

As introduced in [**C:ACLMT22**], it's also possible to use a public R-SIS vector $\mathbf{h}$ over a $\mathcal{R}_{c'}$ for $c' \ll q$ as a fixed challenge vector rather than sampling a fresh $\mathbf{h}$ each time from an exceptional set as above. This requires no interaction, but comes at the cost of a much larger operator norm $c = c' \cdot \gamma_{\mathcal{R}}$ versus the exceptional sets approach.

Private aggregation. If the aggregator is the prover itself, they may compute the linear combination of the $a_{i,j,k}$ values first to obtain a vector $\mathbf{a}'$ and then compute the aggregate proof $\pi_0 = \sum_i a'_i \cdot \mathbf{u}_{0,i}$ thereby reducing the amount of costly ring operations.

Inner product argument. We can easily construct an inner product argument (IPA) using $\mathsf{Orbweaver}$, allowing a prover to prove the inner product $y := \langle \mathbf{x},\ \mathbf{x}' \rangle \bmod q$ between two committed vectors. The prover runs commit twice to produce c, c', π_1, and π'_1, and runs $\mathsf{Open}(\mathsf{ck}, \mathbf{x}', \mathbf{x})$ to obtain π_0. The verifier checks the norm bounds of all three proofs, verifies both knowledge proofs as in Verify, and then checks the opening proof satisfies $\langle \mathbf{a},\ \pi_0 \rangle \equiv c' \cdot c - y \bmod q$.

It should be noted that this requires a modification of the norm bound β_0^* for which R-SIS must be hard. In the modified proof of theorem 4.4.2 the extractor would generate π_0^* using extracted values $\mathbf{x}^*$ and f^*, both of norm α^* (whereas in the linear function case they use f of norm α). Then we must set $\beta_0^* = w^2(\alpha^*)^2\beta_0\gamma_{\mathcal{R}}^2 = w^4\alpha^2\beta_1^2\beta_0\gamma_{\mathcal{R}}^4$.

Recursive proof composition. Observe that $\mathsf{vk}_f \cdot \mathsf{ck} - y \bmod q$ and $c \cdot t \bmod q$ are Ajtai R-SIS commitments to π_0 and π_1 for respective public vectors $\mathbf{a}_0$ and $\mathbf{a}_1$. Using any proof-succinct argument of knowledge for R-SIS commitment preimages (e.g., [**BS22**]), we can achieve better proof sizes by instead sending an AoK of π_0 and π_1. The composed scheme will still have log-time verification even if the outer argument has a linear-time verifier.

One may also recursively compose Orbweaver with itself by sending commitments to the proofs and proving they satisfy the (linear) verification equations. To deal with the norm bound checks, the only non-structure-preserving part of verification, one could send a random Johnson-Lindenstrauss projection, a linear operation which requires only constant communication, then prove it was computed correctly with respect to the committed proofs. At the 128-bit security level, an optimized version of Orbweaver achieves witness compression starting at sizes a little under 668KiB, or 165KiB if we forego extractability and only require evaluation binding; achieving smaller proof sizes will require switching to recursion using another (possibly even linear verifier) scheme at this point, as described above.

Linear functional commitments for integers. We can also extended Orbweaver to support the input alphabets $\mathcal{X} = \{z \in \mathbb{Z} \mid |z| < \alpha\}$ and, using the centered representation, $\mathcal{X} = \{z \in \mathbb{Z} \mid 0 \leq z \leq 2\alpha\}$. One way to do this is to use the coefficient embedding to encode $\hat{f}, \hat{\mathbf{x}} \in \mathbb{Z}^N$ as $\mathbf{x}, f' \in \mathcal{R}^w$ where $N = wn$. We further set $f = [\sigma_{-1}(f'_i)]_{i \in [w]}$, where $\sigma_{-1} \in \mathsf{aut}(\mathcal{R}_q)$ is the Galois automorphism corresponding to $\sigma_{-1}(X) = X^{-1}$ (see section 4.2). We then have that $\langle \hat{f},\ \hat{\mathbf{x}} \rangle \equiv \mathsf{ct}(\langle f,\ \mathbf{x} \rangle) \bmod q$, where ct returns the constant coefficient. We run Orbweaver with commitments to $f, \mathbf{x}$ and with $y = \langle f,\ \mathbf{x} \rangle$. While y is part of the instance over $\mathcal{R}$, only $\mathsf{ct}(y)$ is in the instance over $\mathbb{Z}$ in this shim protocol, while the non-constant coefficients of y are instead part of the proof for the relation over $\mathbb{Z}$.

4.4.2 Sublogarithmic proofs in the input norm

We present a functional commitment based on Orbweaver that achieves $O(\log w + \log\log \alpha)$ commitment and proof size by combining ternary decomposition and proof aggregation. The construction below is restricted to the input alphabets consisting of bounded-size balanced integers $\mathcal{X} = \{z \in \mathbb{Z} \mid |z| < \alpha\}$ and (using the centered representation) unbalanced integers $\mathcal{X} = \{z \in \mathbb{Z} \mid 0 \leq z \leq 2\alpha\}$. While our techniques should extend to subsets of cyclotomic rings more generally, we leave this to future work.

Let $t = \log_3(\alpha)$. Ternary decomposition allows us express a linear map over length-N (where $N = wn$, a notation switch that will soon be helpful) vectors of norm α as $2t - 1$ linear maps of

length-(Nt) vectors of norm 1. Using the aggregation techniques in section 4.4.1, we can create a single compact aggregate proof for k separate linear maps of size $O(\log(N\alpha k))$. For $k = 2t - 1$ and $\alpha = 1$ this gives $O(\log N + \log t)$ proofs. In terms of proof norms, we trade a power of α for a higher power of $\log_3(\alpha)$. Similarly, committing to the ternary decomposition of the witness results in a $O(\log(Nt) + \log \alpha) = O(\log N + \log t)$ sized commitment. Experimentally, at the 128-bit security level decomposition and aggregation performed better starting around $\alpha = 2^{16}$ for all input lengths of interest for this scheme, i.e., for input vectors of length at least 2^{16} where we begin to achieve witness compression.

We preface our result below with a few remarks. The requirement $\alpha = 3^t$ is not strict and the proceeding construction works if we set $t = \lceil \log_3(\alpha) \rceil$. This is more than just with respect to correctness: since Orbweaver proves $\langle f,\ \mathbf{x} \rangle = y$ over the integers, it will also hold $\bmod\ \alpha$. It is always possible to include in a commitment some elements of norm larger than correctness is guaranteed for. When taking the result mod α by committing to these values the prover is effectively committing to their coset representative.

Theorem 4.4.4. *Let $\hat{\mathcal{X}} = \{z \in \mathbb{Z} \mid |z| \leq \alpha = 3^t\}$, and for $N = wn$ let $\hat{f}, \hat{\mathbf{x}} \in \hat{\mathcal{X}}^N$. We obtain a succinct functional commitment for $\langle \hat{f},\ \hat{\mathbf{x}} \rangle = y$ where the size of c, π_1, and π_0 are all $O(\log N + \log\log \alpha)$.*

Proof (informal). We can write

$$\langle \hat{f},\ \hat{\mathbf{x}} \rangle = \sum_{i=0}^{2t-2} 3^i \langle f'_i,\ \mathbf{x}' \rangle$$

where $[f'_i]_{i=1}^t, \mathbf{x}' \in \{-1, 0, 1\}^{Nt}$. We then combine the techniques from "private aggregation" and "linear functional commitments for bounded integers" above to compute this using Orbweaver.

If we view each f'_i as a column vector and, abusing notation, let $f'_{i,j}$ refer to j-th (for $j \in \{0, \dots, N-1\}$) sequential t-length subvector of f'_i we see the following form

$$[f'_{0,j} \cdots f'_{2t-1,j}] = \begin{bmatrix} b_0 & b_1 & b_2 & \cdots & b_{t-1} & 0 & 0 & \cdots & 0 \\ 0 & b_0 & b_1 & \cdots & b_{t-2} & b_{t-1} & 0 & \cdots & 0 \\ \vdots & & & & & & & & \\ 0 & 0 & \cdots & 0 & b_0 & b_1 & b_2 & \cdots & b_{t-1} \end{bmatrix} \in \{-1, 0, 1\}^{t \times (2t-1)}$$

where $[b_k]_{k=0}^{t-1}$ is the ternary decomposition of $\hat{f}_j$. For simplicity, let $n = rt$ (otherwise, padding may be applied) for $r \in Z^+$, thus supporting decomposition for norm bounds up to 3^{512} (where, again, modifications enable more generality). Let $f_i \in \mathcal{R}^{wt}$ denote the coefficient embedding of f'_i. Then we have for

$$\tilde{f} = \sum_{i=0}^{2t-2} h_i f_i$$

that $\left\|\tilde{f}_j\right\| \leq \mathsf{min}\{c, r\} \cdot t^2 \leq ct^2$ for all $j \in [wt]$, where c is the operator norm of the challenge set $\mathcal{H}$ used. This follows from the fact we are taking the sum over $2t-1$ vectors $h_i f_i$, where when averaging over the i the norm bound of each element is $\mathsf{min}\{c, r\} \cdot t^2/(2t-1)$. The norm of $\tilde{f}$ is now larger than $\mathbf{x} \in \mathcal{R}^{wt}$, and for a more tight result we recompute the norm bounds with this in mind.

First, recall $y_i = \langle f_i,\ \mathbf{x}\rangle$. For $i \in \{0, \dots, t-1\}$ we then obtain

$$\|y_i\| \leq (i+1)rwt = (i+1)N = \delta_{M,i}$$

and for $i \in \{t, \dots, 2t-1\}$ we have

$$\|y_i\| \leq (2n - (i+1)r)wt = (2t-i-1)N = \delta_{M,i}$$

Next, $\pi_1 = \sum_{i=1}^{w} x_i \mathbf{u}_{1,i} \bmod q$, where $\left\|x_i \mathbf{u}_{1,i}\right\| \leq \beta_1 \gamma_{\mathcal{R}}$ for all i. It follows

$$\|\pi_1\| \leq wt\beta_1\gamma_{\mathcal{R}} = Nt\beta_1 = \delta_1$$

Then, $\pi_0 = \sum_{k \in P} a_k \cdot \mathbf{u}_{0,k} \bmod q$, where $a_{-wt+1}, \dots, a_{wt-1}$ denote the $2wt-2$ coefficients of $\mathbf{x}(v)\tilde{f}(v) = \left(\sum_{j=1}^{wt} x_j v^j\right)\left(\sum_{j=1}^{wt} \tilde{f}_j v^{-j}\right)$ and $P := \{-wt+1, \dots, wt-1\}\ \{0\}$. Since $\left\|\mathbf{u}_{0,k}\right\| \leq \beta_0$ for all k, $\|\pi_0\| \leq \beta_0 \gamma_{\mathcal{R}} \sum_{k \in P} a_k$. Then $\left\|a_{|k|}\right\| \leq (wt-k)ct^2\gamma_{\mathcal{R}}$ for $k \in [w-1]$. We then have

$$\sum_{k \in P} a_k = 2ct^2\gamma_{\mathcal{R}} \sum_{\kappa=1}^{wt-1} (wt - \kappa) = ct^2\gamma_{\mathcal{R}}(w^2t^2 - wt)$$

Thus,

$$\begin{aligned}\|\pi_0\| &\leq \beta_0 ct^2\gamma_{\mathcal{R}}^2((N/n)^2t^2-(N/n)t) = N^2ct^4\beta_0 - Nct^3\beta_0 n \\ &= \delta_0 \\ &\leq (N^2-N)ct^4\beta_0 \leq N^2ct^4\beta_0\end{aligned}$$

Last, we consider δ_0^*. Replacing, $\mathbf{x}$ with the extracted $\mathbf{x}^*$ in the analysis above, where $\|x_j^*\| \leq \delta_1 = Nt\beta_1$ for all $j \in [wt]$, gives

$$\|\pi_0^*\| \leq Nct^3\beta_0\beta_1\gamma_{\mathcal{R}}^2((N/n)^2t^2-(N/n)t) = N^3ct^5\beta_0\beta_1 - N^2ct^4\beta_0\beta_1 n$$

Finally,

$$\begin{aligned}\|\pi_0-\pi_0^*\| &\leq N^2ct^4\beta_0 - Nct^3\beta_0 n + N^3ct^5\beta_0\beta_1 - N^2ct^4\beta_0\beta_1 n \\ &= \delta_0^* \\ &\leq N^3ct^5\beta_0\beta_1\end{aligned}$$

□

4.4.3 Polynomial commitments for integers

We show how to construct $O(\log N)$ proof size and $O(\log N \log\log N)$ verifier time polynomial commitment (PC) for both degree-N univariate polynomials and N-linear polynomials over $\mathbb{Z}$ using Orbweaver. How to do this while preserving a quasilog-verifier is not immediate. In a PC protocol based on Orbweaver, the prover commits to the coefficients of the polynomial and the function f becomes the powers of a challenge evaluation point z. Previously we considered the preprocessing setting, where we assume the same function will be evaluated repeatedly on different commitments. In this amortized setting the cost to run PreVerify once is considered a constant. On the other hand, in the PC setting f is defined by z, which is usually a fresh challenge sampled uniformly at random from $\mathbb{Z}_M$—a setting in which preprocessing no longer makes sense.

We present the univariate degree-N PC. Omitting a couple details, by replacing z^{2^i} with z_i for all i below we obtain the N-linear version. Again, we choose $\alpha \geq (M-1)/2$ and assume wlog that $\alpha = 3^t$, $wn = N$, and w is a power of 2. We show how to compute, in log-time, a commitment key $\mathsf{ck}_z' \equiv \mathsf{ck}_z \bmod M$, where ck_z is the commitment key corresponding to running PreVerify on the function $f = (f_1, ..., f_w) \in \mathbb{Z}_M^w$, where $\sigma_{-1}(f_i) \in \mathcal{R}_q$ is equal to the coefficient embedding of the vector $z^{n(i-1)} \cdot (z^0, z^{1+n(i-1)}, \dots, z^{n-1}) := z^{n(i-1)} \cdot f_\heartsuit \in \mathbb{Z}_M^n$. This choice of f follows the techniques

we used in section 4.4.2 such that the evaluation of the committed polynomial at z will be equal to the constant coefficient of y. The verifier computes

$$\mathsf{ck}'_z := f_\heartsuit \cdot v^{-w} \cdot \prod_{i=0}^{\log(w)-1} ((z^{n \cdot 2^i} \bmod M) + v^{2^i}) = \sum_{i=1}^{w} f_\heartsuit \cdot \hat{z}_{i-1} \cdot v^{-i} \equiv \mathsf{ck}_z \bmod M$$

In particular this holds because each $\hat{z}_i \equiv z^{in} \bmod M$. The downside of computing the key this way is that it corresponds to a commitment key for the function $\hat{f} := f_\heartsuit \cdot \hat{z} \in \mathcal{R}_q^w$, where $\left\|\hat{f}\right\| \leq \alpha^{\log w+1}$ since the norm of $\hat{z}_i$ is bounded by α to the power of the Hamming weight of i. Roughly speaking, computing the commitment key this way comes at a cost of a $\log \alpha$ times larger opening proof.

4.5 Evaluation

We used SageMath to compute proof and CRS sizes for an optimized version Orbweaver for integers in section 4.5.2, and in section 4.5.2 we compare an unoptimized version for cyclotomic rings to previous work [**C:ACLMT22**]. Before getting there, we describe the additional optimizations in section 4.5.1.

4.5.1 Implementation optimizations

Sending only π_1 and not c. We can slightly reduce communication and both prover and verifier complexity by only sending π_1 and not c. The verifier first computes $\langle \mathbf{a}_1,\ \pi_1 \rangle = c_t$, then checks $c_t \in \langle t \rangle$,[5] and verifies

$$\langle \mathbf{a}_0,\ \pi_0 \rangle \cdot t = c_t \cdot \mathsf{ck}_f - z \cdot t$$

as well as the standard norm checks. This works because

$$t(\langle \mathbf{a},\ \pi_0 \rangle - c \cdot \mathsf{ck}_f + z) \equiv 0 \bmod q$$

[5]Recall that we pick t by setting half its NTT coefficients t_i to 0 and picking the rest at random from $\mathbb{Z}_q$. The condition $b \in \langle t \rangle$ is equivalent to $b_i \in \langle t_i \rangle$ (in the NTT basis), which can be verified by checking $b_i = 0$ for all i such that $t_i = 0$ since $b_i, t_i \in \mathbb{F}_q$.

implies

$$\langle \mathbf{a}_0,\ \pi_0 \rangle = c \cdot \mathsf{ck}_f - z$$

by assumption (2) Since c is a deterministic function of π_1, an extractor can still obtain $\mathbf{x}^*$ given π_1. To see the evaluation proof check still holds, note that $t \leftarrow\$ \mathcal{T}$ is sampled such that finding $s' = 0$ and $s't = 0$ is hard. This implies that if $lt = rt$, then $l - r = 0$.

Distinct vs. equal a_0 and a_1. We choose to fix a single public R-SIS vector $\mathbf{a}$ with respect to which preimages for v^i and $v^i t$ are generated, as this enables better verifier complexity. We note that with distinct trapdoor SIS vectors $\mathbf{a}_0, \mathbf{a}_1$, we can further optimize proof size, but only very slightly in our experiments.

The most expensive part for the verifier is computing $\langle \mathbf{a}_0,\ \pi_0 \rangle$ and $\langle \mathbf{a}_1,\ \pi_1 \rangle$, which requires $O(n \log n \cdot (\ell_0 + \ell_1))$ multiplications between the arbitrary $\mathcal{R}_q$ elements in the $\mathbf{a}_i$ and the bounded norm elements in the π_i. For concrete parameters, we believe these multiplications might be computed most efficiently using an NTT-based algorithm [**bhkpy22**]. Either way, by fixing a single $\mathbf{a} = \mathbf{a}_0 = \mathbf{a}_1$ the verifier runtime can be nearly halved using the verification equation

$$\langle \mathbf{a},\ (\mathsf{ck}_f \cdot \pi_1 - t \cdot \pi_0) \rangle = z \cdot t$$

Computationally indistinguishable trapdoor matrix. Recall that trapdoor public RSIS vectors in [**EC:MicPei12**] are of the form $\mathbf{a} = [\hat{\mathbf{a}} | \mathbf{g} - \mathbf{r}\hat{\mathbf{a}}]$, where $\hat{\mathbf{a}} \leftarrow\$ \mathcal{R}_q^\ell$, $\mathbf{g}$ is the public gadget vector and $\mathbf{r}$ is the trapdoor/ In prior work [**C:ACLMT22**], $\ell = \lceil \log_\beta q \rceil$ was chosen, where it can be shown using the leftover hash lemma the resulting $\mathbf{a}$ is indistinguishable from random. We can instead use $\ell = 2$, where one can see the resulting $\mathbf{a}$ is an RLWE sample, $[\hat{\mathbf{a}}, \mathbf{g} - \hat{A}\mathbf{r}]$ where $\hat{\mathbf{a}} = [1 | \hat{\mathbf{a}}^\top \leftarrow \mathcal{R}_q^{\log_b q}] \in \mathcal{R}_q^{\log_b q \times 2}$, for some $b = O(\beta)$ and $\mathbf{r} \leftarrow\$ \mathcal{D}$ where $\mathcal{D}$ is some distribution for the RLWE secret and error. Experimentally, we find the best distribution, b, β such that the proof size is minimized and the RLWE problem has 128-bit hardness.

Other trapdoor optimizations. We only consider one optimization to the computationally indistinguishable variant of [**EC:MicPei12**]: the heuristic from [**gmpw20**] for computing s_1. However, works focused on improving lattice signature sizes have introduced optimizations we

estimate together could reduce our proof sizes by 5–10× [**cgm19**; **dsh21**; **jht22**]. We leave parameterizing such variants as future work.

4.5.2 Proof and CRS sizes

In the tables below we present results for our optimized bounded integer maps for input alphabets $\mathcal{X}_1 = \{0, 1, 2\}$ and $\mathcal{X}_{32} = \{0, \ldots, 2^{32} + 1\}$, both with and without extractability, at the 128 bit classical security level. Our proof sizes are competitive with other non-recursive post-quantum arguments, especially for witnesses of length $> 2^{25}$. The verifier keys remain very small even up to 2^{30}, while for larger instances the prover key grows quite large.

Sparse inputs. We note that the proof sizes below are actually upper bounds. When working with sparse f and/or $\mathbf{x}$, as is common in practice, the proof size will be dependent on the number of non-zero entries

$\mathcal{X}_{32}$, extractable	2^{18}	2^{19}	2^{20}	2^{21}	2^{22}	2^{23}	2^{24}	2^{25}	2^{26}	2^{27}	2^{28}	2^{29}	2^{30}
commitment (B)	147	167	154	163	173	188	185	200	211	227	243	236	252
total proof size (KiB)	845	904	977	1,028	1,081	1,137	1,197	1,257	1,315	1,376	1,438	1,505	1,571
witness compression	1.2×	2.3×	4.2×	8.0×	15×	28×	54×	104×	199×	381×	729×	1,394×	2,670×
verifier key size (MiB)	28	29	31	32	33	34	36	37	38	40	41	42	43
prover key size (GiB)	0.3	0.7	1.3	2.8	5.9	12.8	25.3	53.2	111	234	494	982	2,070

$\mathcal{X}_{32}$, binding	2^{17}	2^{18}	2^{19}	2^{20}	2^{21}	2^{22}	2^{23}	2^{24}	2^{25}	2^{26}	2^{27}	2^{28}	2^{29}	2^{30}
commitment (B)	51	56	57	60	65	66	71	75	76	82	83	92	93	95
total proof size (KiB)	264	278	294	311	327	344	360	379	398	415	434	453	474	494
witness compression	1.9×	3.7×	7.0×	13×	25×	48×	91×	173×	329×	632×	1207×	2314×	4427×	8483×
verifier key size (MiB)	1.7	1.9	2.1	2.2	2.4	2.6	2.8	3.0	3.2	3.4	3.7	3.9	4.2	4.4
prover key size (GiB)	< 0.1	0.1	0.2	0.3	0.7	1.5	3.1	6.6	13.5	28.2	57.7	124	253	517

$\mathcal{X}_1$, extractable	2^{22}	2^{23}	2^{24}	2^{25}	2^{26}	2^{27}	2^{28}	2^{29}	2^{30}
commitment (B)	106	118	129	145	162	188	166	180	178
total proof size (KiB)	668	712	757	805	862	931	986	1,038	1,096
witness compression	1.2×	2.3×	4.3×	8.1×	15×	28×	53×	100×	190×
verifier key size (MiB)	2.6	2.8	3.0	3.2	3.4	3.7	3.9	4.2	4.4
prover key size (GiB)	3.8	8.3	17.7	38.4	83.7	187	360	764	1,545

$\mathcal{X}_1$, binding	2^{20}	2^{21}	2^{22}	2^{23}	2^{24}	2^{25}	2^{26}	2^{27}	2^{28}	2^{29}	2^{30}
commitment (B)	35	38	42	42	49	50	51	55	59	63	65
total proof size (KiB)	165	178	189	202	215	229	243	256	272	286	302
witness compression	1.2×	2.3×	4.3×	8.0×	15×	28×	54×	101×	191×	364×	689×
verifier key size (MiB)	1.0	1.1	1.3	1.4	1.5	1.7	1.8	1.9	2.1	2.3	2.5
prover key size (GiB)	0.2	0.5	1.0	2.1	4.5	9.3	19.1	40.5	85.5	180	371

Comparison to [C:ACLMT22]

For sake of better comparison with [**C:ACLMT22**], in Figure 4.2 we consider an unoptimized version of Orbweaver (as trit decomposition in section 4.4.2, better trapdoor sampling, and many other types of optimizations in section 4.5.1 can also be applied to [**C:ACLMT22**]). We also note that due to errors in their code, they (1) set a root Hermite factor much smaller than currently achievable (resulting in a security level higher than 128 bits); and (2) overestimate the desired preimage quality, i.e., the shortness/norm bound of the preimages. In the comparison below, we apply the same errors that "oversecure" our scheme for $\lambda = 128$.

In order to make the comparison as fair as possible, we also modified the code of [**C:ACLMT22**] to remove the overhead induced by their scheme's support for batching and higher degree polynomial maps. While our unoptimized proof sizes are about $2 - 3\times$ smaller, perhaps even more important practically was our reduction of the CRS size and prover time from quadratic to quasilinear in w and α. The Orbweaver CRS size is calculated as $3wn \log \beta$ bits, as it consists of $3w$ vectors of ring elements, each of which with bit length bounded by by $\log(\beta)$. In contrast, [**C:ACLMT22**] requires about $w^2 \ell n \log \beta$ bits for the SRS.

Acknowledgments

We would like to thank Martin Albrecht and Russell W.F. Lai for answering questions about [**C:ACLMT22**]. We would also like to thank Nicholas Genise for answering our questions about lattice trapdoors.

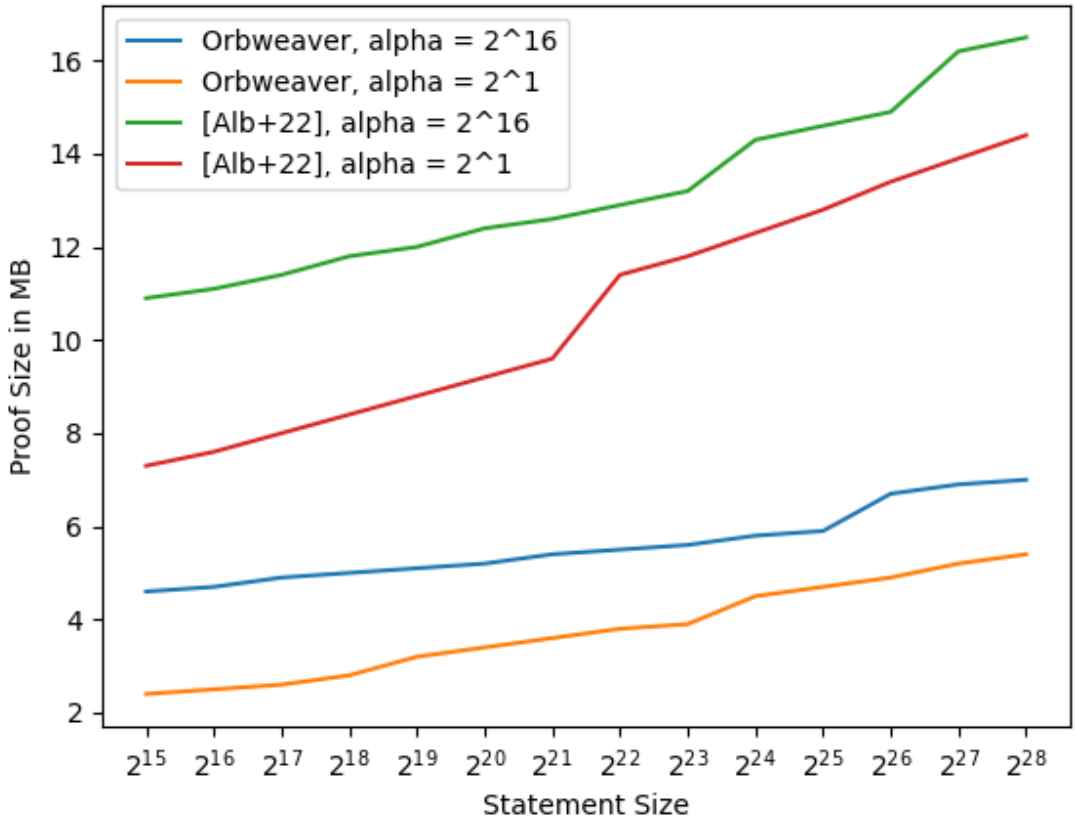


Figure 4.2: Comparison of combined commitment and proof size for an unoptimized version of Orbweaver and [**C:ACLMT22**] with $n \geq 2^9$. We have purposely left errors in the code when producing the data in this figure for better comparison with [**C:ACLMT22**] (see details at the start of section 4.5.2).

.1 Sumcheck algorithms

This section includes an optimized sumcheck protocol in Figure 3 and a linear time and space prover algorithm for it in Figure 4, both from from [**BagadDaoDombThaler25**]. We include these both for self-containedness and because these particular protocols provide a launching off point for some further sumcheck optimizations introduced in our paper.

In the following version of sumcheck the polynomial s_i in each round i is specified by its evaluation on the $d+1$ points $\widehat{U}_d = \{\infty\} \cup \{0, \ldots, d\}$ (where the point ∞ is explained just below).

The evaluation $s_i(1)$ can actually be omitted, using the equation $s_i(1) = \sigma_{i-1} - s_i(0)$.

For a degree d univariate polynomial $s(X) = a_d X^d + \cdots$ we use a variant of Lagrange interpolation where the highest degree coefficient a_d is sent in addition to evaluations of the polynomial $t(X) = s(X) - a_d \cdot \prod_{k=0}^{d-1}(X - x_k)$ over the domain $\{0, \dots, d\}$. We note that $t(X)$ is of degree $d-1$, and equal to $s(x)$ over $\{0, \dots, d\}$. Defining $s(\infty) = a_d$, we recover s via evaluations of

$$s(X) = a_d \cdot \prod_{k=1}^{d}(X - x_k) + \sum_{k=1}^{d} t(x_k) \cdot \mathcal{L}_{\{0,\dots,d\},k}(X)$$

over $\widehat{U}_d$, where $\mathcal{L}_{\{0,\dots,d\},k}$ is the kth Lagrange basis polynomial over the interpolation set $\{0, \dots, d\}$.

If $s(X) = \prod_{i=1}^{d} p_i(X)$ is a product of linear polynomials, then

$$s(\infty) = \prod_{i=1}^{d} p_i(\infty) = \prod_{i=1}^{d}(p_i(1) - p_i(0)) \quad .$$

More generally, when $s(X) = G(p_1(X), \dots, p_d(X))$ for some polynomial G and linear polynomials $p_1, \dots, p_d$, we can calculate $s(\infty)$ by evaluating only the terms of G of highest total degree. Then, like $s(1)$, the prover can generally omit sending the evaluation $s(\infty)$, instead computing it using the equation above.

A linear time and space prover for this sumcheck version may be implemented as follows. The prover maintains d arrays $P_1, \dots, P_d$, which initially store all evaluations of $p_1, \dots, p_d$ over $\{0,1\}^v$; we have $P_k[x] = p_k(x)$ for all $k \in [d]$ and $x \in \{0,1\}^v$. The prover halves the size of the arrays after each round i, via "binding" the arrays $[P_k]_{k \in [d]}$ to the challenge α_i received from the verifier.

Figure 3: The sumcheck protocol

Setup: Assume that the verifier has oracle access to the polynomial g, and knows the value σ_0 claimed to equal

$$\sum_{x \in \{0,1\}^v} g(x) = \sigma_0.$$

For each round $i = 1, \dots, v$:

1. The prover P sends the univariate polynomial $s_i(X)$ claimed to equal

$$\sum_{(x_{i+1},\dots,x_v) \in \{0,1\}^{v-i}} g(\alpha_1, \dots, \alpha_{i-1}, X, x_{i+1}, \dots, x_v).$$

   P does so by sending the evaluations $\{s_i(u) : u \in \widehat{U}_d\}$ to the verifier V.

2. The verifier V sends a random challenge $\alpha_i \leftarrow \mathbb{F}$ to P.

3. V derives $s_i(1) := \sigma_{i-1} - s_i(0)$, then sets $\sigma_i := s_i(\alpha_i)$.

After v rounds, we reduce to the claim that

$$g(\alpha_1, \dots, \alpha_v) = \sigma_v.$$

V checks this claim by making a single oracle query to g.

Figure 4: A linear time and space prover implementation of sumcheck

Setup: Length-2^v arrays $P_1^{(0)}, \dots, P_d^{(0)}$ such that

$$P_k^{(0)}[x] = p_k(x) \text{ for all } k = 1, \dots, d \text{ and } x \in \{0,1\}^v.$$

For each round $i = 1, \dots, v$:

1. For $u \in U_d^b$, compute $s_i(u)$ using the formula

$$s_i(u) := \sum_{x' \in \{0,1\}^{v-i}} \prod_{k=1}^{d} \Big((P_k^{(i)}[1, x'] - P_k^{(i)}[0, x']) \cdot u + P_k^{(i)}[0, x'] \Big).$$

2. Send $\{ s_i(u) : u \in U_d^b \}$ to the verifier.
3. Receive challenge $\alpha_i \in F$ from the verifier.
4. For $k = 1, \dots, d$ and $x' \in \{0,1\}^{v-i}$, update arrays:

$$P_k^{(i+1)}[x'] := (P_k^{(i)}[1, x'] - P_k^{(i)}[0, x']) \cdot \alpha_i + P_k^{(i)}[0, x'].$$

`// Invariant:` $P_k^{(i)}[x'] = p_k(r[1:i], x')$ for all $k \in [d]$, $i \in [0, v]$, $x' \in \{0,1\}^{v-i}$.